

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Ingeniería

Carrera de Ingeniería en Software

SGED

Sistema de Gestión para la
Escuela Deportiva ProFútbol

Informe de la Entrega Final

Proyecto Fin de Curso — Aplicaciones Web

Asignatura: Aplicaciones Web (Quinto nivel)

Docente: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.

Periodo: PPA 2026-2027

Etiqueta: v1.0.0

Commit del corte: el *commit* al que apunta la etiqueta v1.0.0

DOI del software: [10.5281/zenodo.22714477](https://doi.org/10.5281/zenodo.22714477) (corte v1.0.0, publicado el 2026-09-11)

DOI del dataset: [10.5281/zenodo.22422305](https://doi.org/10.5281/zenodo.22422305)

Integrantes y ORCID:

Arcalle Grefa Darwin Orlando ORCID: <https://orcid.org/0009-0005-8002-1649>

Pallo Pinto Alejandro Daniel ORCID: <https://orcid.org/0009-0009-0717-4250>

Velez Lopez Ricardo Elias ORCID: <https://orcid.org/0009-0005-9463-6847>

Repositorio:

https://github.com/DarwinSM21/SGED_APPWEB

Despliegue público:

App: <https://sged-frontend-r2rs.onrender.com>

API y /actuator/health: <https://sged-backend-2p05.onrender.com>

Quevedo — Los Ríos — Ecuador — 2026

Nota de versión (honest, no cosmética): esta entrega queda cerrada en la etiqueta v1.0.0, cuyo commit consta arriba. Los tres ORCID ya están registrados y verificados contra la API pública de orcid.org. El DOI del dataset de mediciones está publicado en Zenodo ([10.5281/zenodo.22422305](https://doi.org/10.5281/zenodo.22422305)) bajo licencia CC BY 4.0, como depósito separado del software.

Índice

Lista de siglas y acrónimos	5
Resumen	6
Abstract	7
1. Introducción	8
1.1. Contexto y motivación	8
1.2. Preguntas de investigación	8
1.3. Objetivo general y objetivos específicos	9
1.4. Contribuciones	9
1.5. Organización del documento	10
1.6. Pila tecnológica	10
1.7. Estado de la entrega: declaración previa	10
2. Marco teórico	12
2.1. Ingeniería de requisitos	12
2.2. Calidad de un requisito bien formado (INCOSE v4)	13
2.3. Historias de usuario y casos de uso	13
2.4. Modelo arquitectónico C4	13
2.5. Calidad de software: ISO/IEC 25010	13
2.6. Arquitecturas orientadas a recursos (REST)	13
2.7. Autenticación stateless: JWT y su ubicación en cookie HttpOnly	13
2.8. OWASP Top 10	15
2.9. Patrones de acceso a datos y modelos de consistencia	15
3. Trabajos relacionados	15
3.1. Estrategia de búsqueda	15
3.2. Síntesis comparativa	16
3.3. Brecha identificada	17
4. Ingeniería de requisitos	18
4.1. Contexto y stakeholders	18
4.2. Técnicas de elicitación aplicadas	18
4.3. Requisitos funcionales y no funcionales: categorización y prioridad	19
4.4. Modelado de casos de uso y de historias de usuario	19
4.5. Validación de requisitos	20
4.6. Gestión del cambio de requisitos	20
4.7. Métricas de calidad de requisitos	20
5. Resolución de las observaciones previas (Bloque 0)	21
6. Reestructuración de paquetes y reconciliación de documentación	22
6.1. Los paquetes reservados vacíos: por qué se eliminaron	23

7. Arquitectura	24
7.1. Vista de contexto (C4 nivel 1)	24
7.2. Vista de contenedores (C4 nivel 2)	25
7.3. Vista de componentes (C4 nivel 3)	25
7.4. Autenticación: JWT en cookie <code>HttpOnly</code>	26
8. Estrategia híbrida de acceso a datos	26
8.1. Un defecto real: <code>FUNCTION</code> frente a <code>PROCEDURE</code>	26
8.2. Ausencia de SQL dinámico	28
9. Implementación	28
9.1. Aislamiento de capas	28
9.2. Manejo global de errores	29
9.3. Autenticación en el <i>frontend</i> : cookies, no <i>localStorage</i>	29
9.4. Caché de consultas de listado	29
9.5. Configuración por entorno	30
10. Requisitos y modelo de datos	30
10.1. Resumen de requisitos	30
10.2. Catálogo de la API REST	31
10.3. Modelo de datos	32
11. Materiales y métodos	33
11.1. Enfoque metodológico	33
11.2. Instrumentos de medición	33
11.3. Enfoque de métricas: un GQM completo	33
11.4. Población, muestreo y participantes	33
11.5. Análisis estadístico declarado a priori	34
11.6. Entorno de ejecución	35
11.7. Procedimiento por bloque	35
11.8. Determinismo, semillas y trazabilidad	35
12. Evidencia empírica	37
12.1. Rendimiento (Bloque C.1)	37
12.2. Seguridad: OWASP Top 10 (Bloque C.4)	38
12.2.1. Una auditoría que no auditaba lo que decía auditar	39
12.2.2. El defecto que A01 destapó: control de acceso ausente	39
12.3. Dos defectos de funcionamiento encontrados al ejecutar el sistema	40
12.4. Análisis estático y escaneo automático (Bloque A.1/A.2.3)	40
12.5. Cobertura de pruebas	42
12.6. Calidad web: Lighthouse (Bloque C.5 / A.1)	45
12.6.1. Por qué el SEO de 63 es el resultado correcto	45
12.6.2. Medición pública en el despliegue real (2026-09-08)	46
12.7. Usabilidad: SUS (Bloque C.3)	46
13. Reproducibilidad	48
14. Amenazas a la validez	48
14.1. Validez de constructo	48
14.2. Validez interna	49
14.3. Validez externa	49
14.4. Validez de conclusión	49

15.Consideraciones éticas	50
15.1. Generación de retroalimentación con un modelo de lenguaje externo	51
16.Declaración de contribuciones (CRedit)	51
16.1. Roles por integrante	51
16.2. Cobertura de los catorce roles CRedit	52
16.3. Dos advertencias sobre la medida	52
17.Trazabilidad y honestidad académica	53
18.Discusión	53
19.Trabajo futuro	55
20.Conclusiones	55
21.Declaraciones obligatorias	56
Anexos	62
Anexo A — Observaciones acumuladas	62
Anexo B — Cadena de búsqueda completa (Capítulo 3)	62
Anexo C — Protocolo detallado de la encuesta SUS	62
Anexo D — Ejecución del pipeline de integración continua	64
Anexo E — Checklist del estándar empírico (Ralph et al., 2021)	65
Anexo F — Checklist INCOSE v4 de calidad de requisitos	66
Anexo G — Matriz de trazabilidad completa	66

Índice de figuras

1. Diagrama de flujo PRISMA 2020 de la revisión de trabajos relacionados.	16
2. C4 nivel 1 — contexto del sistema SGED, generado desde docs/arquitectura/workspace.dsl.	24
3. C4 nivel 2 — contenedores de SGED, generado desde docs/arquitectura/workspace.dsl.	25
4. C4 nivel 3 — componentes del contenedor API Spring Boot, generado desde docs/arquitectura/workspace.dsl.	27

Índice de cuadros

1. Pila tecnológica de SGED.	10
2. Escenarios de calidad ISO/IEC 25010, priorizados y vinculados a la estrategia arquitectónica y a la evidencia reproducible.	14
3. Trabajos relacionados y su diferencia frente a SGED.	16
4. Interesados de SGED por anillo (<i>stakeholder onion</i>).	18
5. Requisitos funcionales de SGED por categoría MoSCoW, declarada explícitamente en el SRS.	19
6. Métricas de calidad de requisitos de SGED.	21
8. Reparto de responsabilidades entre ORM y procedimientos almacenados.	26
9. Procedimientos almacenados de SGED por categoría funcional (Bloque A.2.1).	28
12. Esquemas del modelo de datos de SGED.	32
13. Instrumentos de medición usados en la evaluación empírica.	34
14. Ejemplo de Goal-Question-Metric para RQ1.	34
15. Entorno de ejecución de las mediciones.	35
16. Procedimiento de medición por bloque de evaluación.	36

17.	Caché cálida: resultados de las corridas de rendimiento (<code>k6, docs/mediciones/perf/k6-run*.json</code>). 37	37
18.	Caché fría: resultados de las corridas de rendimiento (<code>k6, docs/mediciones/perf/k6-frio*.json</code>). 37	37
19.	Contraste inferencial entre caché cálida y caché fría (Mann-Whitney).	38
20.	Resultado de los seis controles OWASP verificados.	38
21.	Criterio de acceso aplicado por recurso tras corregir el hallazgo H-08.	40
22.	Clases de prueba JUnit y qué verifica cada una (primera corrida completa, 2026-07-30; el estado actual se reporta más abajo).	43
23.	Cobertura actual por capa arquitectónica.	43
24.	Cobertura por subdominio (agregando <code>controller/dto/service/entity/etc.</code>), recalculada desde <code>jacoco.csv</code>	44
25.	Resultados de Lighthouse por categoría y perfil.	45
26.	Resultados de Lighthouse en el despliegue público, por perfil y ruta (medias de 3 corridas; Lighthouse v13.4.1).	46
27.	Estadísticos agregados de la encuesta SUS.	47
28.	Puntuación SUS promedio por perfil de usuario.	47
29.	Objetivos del Makefile para reproducción end-to-end.	48
30.	Evidencia cuantitativa de autoría por integrante (<code>git log</code> sobre <code>main</code> , medido el 2026-09-11). La misma medición está en <code>CONTRIBUTORS.md</code> . Arcalle Grefa suma los commits de sus dos correos vinculados a la misma cuenta de GitHub (<code>darcalleg@uteq.edu.ec</code> y <code>darwinarcalle@gmail.com</code>).	52
31.	Cobertura de los catorce roles de la taxonomía CRediT.	53
32.	Cuestionario SUS de diez ítems aplicado (español).	63
33.	Interpretación de la puntuación SUS (Bangor, Kortum y Miller, 2009; Sauro, 2011). 63	63
34.	Tres corridas de CI consecutivas y verdes tras cerrar el gate de cobertura, verificadas contra la API de GitHub Actions.	64
35.	Checklist del estándar empírico Engineering Research (Ralph et al., 2021).	65
36.	Características de requisitos individuales, INCOSE v4 (C1–C9).	66
37.	Características del conjunto de requisitos, INCOSE v4 (C10–C15).	66
38.	Matriz de trazabilidad completa (requisito, historia, caso de uso, estado).	67

Listings

1.	Emisión de la cookie de sesión (<code>AuthController.java</code>)	26
2.	Invocación real desde el repositorio (<code>academico/estudiante/repository/EstudianteRepository.java</code>) 28	28
3.	Manejador global de excepciones (<code>backend/.../common/exception/GlobalExceptionHandler.java</code>) 29	29
4.	Interceptor de autenticación (<code>frontend/src/app/auth/auth.interceptor.ts</code>)	29
5.	Respuesta al sexto intento de autenticación	38
6.	Arranque reproducible	48

Lista de siglas y acrónimos

Sigla	Expansión (primera aparición)
ADR	<i>Architecture Decision Record</i> — registro de decisión arquitectónica
API	<i>Application Programming Interface</i>
CI	Integración continua (<i>Continuous Integration</i>)
CRediT	<i>Contributor Roles Taxonomy</i>
CRUD	Crear, Leer, Actualizar, Borrar (<i>Create, Read, Update, Delete</i>)
CSP	<i>Content Security Policy</i>
DOI	<i>Digital Object Identifier</i>
DSR	<i>Design Science Research</i>
FAIR	<i>Findable, Accessible, Interoperable, Reusable</i>
GQM	<i>Goal-Question-Metric</i>
HU	Historia de usuario
IC 95 %	Intervalo de confianza al 95 %
INCOSE	<i>International Council on Systems Engineering</i>
IMRaD	<i>Introduction, Methods, Results and Discussion</i>
JPA	<i>Java Persistence API</i>
JTI	<i>JWT ID</i> — identificador único de un token JWT
JWT	<i>JSON Web Token</i>
LOPD	Ley Orgánica de Protección de Datos Personales (Ecuador)
MoSCoW	<i>Must, Should, Could, Won't</i> — escala de priorización de requisitos
ORM	<i>Object-Relational Mapping</i>
OWASP	<i>Open Web Application Security Project</i>
PFC	Proyecto Fin de Curso
PRISMA	<i>Preferred Reporting Items for Systematic Reviews and Meta-Analyses</i>
RD	Restricción de diseño
RF	Requisito funcional
RNF	Requisito no funcional
RQ	Pregunta de investigación (<i>Research Question</i>)
SGED	Sistema de Gestión para la Escuela Deportiva (ProFútbol)
SP	Procedimiento almacenado (<i>Stored Procedure</i>)
SRS	<i>Software Requirements Specification</i>
SUS	<i>System Usability Scale</i>
UTEQ	Universidad Técnica Estatal de Quevedo
VU	Usuario virtual (<i>Virtual User</i> , k6)
ZAP	<i>Zed Attack Proxy</i> (OWASP)

Resumen

Contexto y problema. La escuela de fútbol formativo ProFútbol gestionaba manualmente la matrícula de menores de edad, la asistencia, los pagos y el inventario, sin trazabilidad verificable ni control de acceso a datos sensibles. **Objetivo.** Este trabajo desarrolla y evalúa SGED, un sistema web que cubre esos cuatro dominios bajo una estrategia híbrida de acceso a datos (ORM y procedimientos almacenados parametrizados) y un proceso de ingeniería de requisitos conforme a ISO/IEC/IEEE 29148:2018. **Métodos.** Se siguió *Design Science Research* (Peppers *et al.*) sobre una arquitectura C4 (Spring Boot, Angular, PostgreSQL, Redis, Docker Compose), con autenticación JWT en cookie `HttpOnly`. La evaluación combina pruebas de carga (k6), auditoría de los seis controles OWASP Top 10, cobertura de pruebas (JaCoCo), calidad web (Lighthouse) y usabilidad (*System Usability Scale*, $N = 15$), con datos crudos reproducibles. **Resultados principales.** El sistema cumple los umbrales de rendimiento (p95 bajo 200 ms), los seis controles OWASP y la cobertura exigida; el proceso de medición detectó y corrigió tres instrumentos defectuosos, incluida una exposición real de datos de menores de edad (H-08, corregido). La usabilidad agregada cruza apenas el umbral, pero esconde un patrón bimodal por rol de usuario. **Conclusiones.** La estrategia híbrida de acceso a datos es sostenible sin sacrificar seguridad, y reportar los instrumentos defectuosos —no solo sus resultados— es un aporte metodológico. Queda abierto el patrón bimodal y la exposición pública del dominio deportivo restante.

Palabras clave: sistemas de gestión escolar deportiva; ingeniería de requisitos; validación empírica de software; seguridad de aplicaciones web; arquitectura de software; pruebas de usabilidad; reproducibilidad de resultados.

Abstract

Context and problem. The ProFútbol youth football academy managed enrollment of underage students, attendance, payments, and sports inventory manually, with no verifiable traceability and no access control over sensitive personal data. **Objective.** This work develops and evaluates SGED, a web system covering those four domains under a hybrid data-access strategy (ORM plus parametrized stored procedures) and a requirements-engineering process aligned with ISO/IEC/IEEE 29148:2018. **Methods.** The project follows Design Science Research (Peffers *et al.*) over a C4-documented architecture (Spring Boot, Angular, PostgreSQL, Redis, Docker Compose), with JWT authentication in an `HttpOnly` cookie. Empirical evaluation combines load testing (k6), an audit of the six mandatory OWASP Top 10 controls, automated-test coverage (JaCoCo), web-quality auditing (Lighthouse), and usability (System Usability Scale, $N = 15$), with reproducible raw data. **Main results.** The system meets the performance thresholds (p95 under 200 ms), all six OWASP controls, and the required coverage; the measurement process uncovered and fixed three faulty instruments, including a real exposure of underage students' data (H-08, fixed). Aggregate usability barely clears the threshold but hides a bimodal pattern by user role. **Conclusions.** The hybrid data-access strategy is sustainable without sacrificing security, and reporting faulty instruments rather than just their results is a methodological contribution. Open work remains on the bimodal pattern and the public sports-domain exposure.

Keywords: sports school management systems; requirements engineering; empirical software validation; web application security; software architecture; usability testing; reproducibility of results.

1. Introducción

Este informe documenta la Entrega Final del Proyecto Fin de Curso de la asignatura Aplicaciones Web: el sistema **SGED**, una aplicación web para la gestión administrativa y deportiva de la escuela de fútbol formativo ProFútbol.

El eje de este trabajo es la *verificabilidad*. Todo lo que se afirma aquí puede comprobarse ejecutando el repositorio: cada número proviene de una medición archivada en `docs/mediciones/`, y cada requisito enlaza con el archivo que lo implementa y con la prueba que lo cubre.

1.1. Contexto y motivación

La administración de una escuela de fútbol formativo combina cuatro procesos que rara vez conviven en una sola herramienta: matrícula y control académico de menores de edad, seguimiento deportivo (categorías, horarios, sesiones, asistencia, evaluación de desempeño), cobro de mensualidades y control de inventario (uniformes, balones, material de entrenamiento). La literatura de informática deportiva documenta que las herramientas digitales dedicadas siguen siendo la excepción, no la norma, en clubes y escuelas de formación —que típicamente operan con hojas de cálculo, mensajería informal y registro en papel— y que esa carencia se traduce en pérdida de trazabilidad y en decisiones tomadas sin datos consolidados [6]. El mismo patrón se documenta en el deporte de alto rendimiento, donde la fragmentación del flujo de información entre preparadores, cuerpo médico y dirección deportiva es un problema reportado explícitamente, no solo intuitivo [38].

El caso de ProFútbol añade una dimensión que no es solo operativa sino regulatoria: la mayoría de sus estudiantes son menores de edad, y sus datos —incluida la cédula, la asistencia con hora y lugar, y las evaluaciones individuales de desempeño— constituyen información sensible bajo la Ley Orgánica de Protección de Datos Personales de Ecuador [1]. Digitalizar sin diseñar control de acceso desde el inicio no resuelve el problema original: lo traslada a un sistema que puede exponerlo a mayor escala, como evidencia el propio hallazgo H-08 de este trabajo (sección 12.2.1). La magnitud concreta del problema que SGED cubre se dimensiona con las cifras de su propia especificación: 52 requisitos individuales (36 funcionales, 13 no funcionales, 3 restricciones de diseño, sección 4) repartidos en cinco roles de usuario y cuatro dominios funcionales, que antes de este proyecto no tenían ningún soporte de software dedicado.

1.2. Preguntas de investigación

Cuatro preguntas guían el resto de este documento. Cada una es cerrada —admite una respuesta verificable contra la evidencia empírica del Capítulo 12, no una opinión— y cada una se retoma explícitamente en la Discusión (sección 18).

RQ1.

¿El sistema cumple los umbrales de rendimiento definidos ($p95 \leq 200$ ms con caché caliente, $p95 \leq 500$ ms con caché fría) bajo una carga concurrente de 50 usuarios virtuales, tanto en operaciones CRUD servidas por el ORM como en operaciones que invocan procedimientos almacenados?

RQ2.

¿La estrategia híbrida de acceso a datos —ORM parametrizado para el CRUD elemental, procedimientos almacenados con parámetros nombrados para consultas multi-tabla, agregados, reportes, actualizaciones masivas, validaciones cruzadas y generación de códigos— elimina el riesgo de inyección SQL de forma verificable mediante los seis controles OWASP y un análisis estático independiente?

RQ3.

¿La usabilidad percibida (SUS) es homogénea entre los distintos perfiles de usuario del sistema (entrenador, estudiante, recepcionista, representante), o existen diferencias significativas entre roles que una media agregada esconde?

RQ4.

¿La cobertura de pruebas automatizadas y la trazabilidad de requisitos se sostienen en los umbrales exigidos (≥ 70 % líneas y ramas; 100 % de requisitos *Must* verificados) conforme crece el dominio funcional del sistema, o se degradan a medida que se agregan módulos nuevos?

1.3. Objetivo general y objetivos específicos

Objetivo general. Desarrollar y validar empíricamente un sistema web de gestión administrativa y deportiva para una escuela de fútbol formativo, bajo una estrategia híbrida de acceso a datos y con evidencia reproducible de sus atributos de calidad.

OE1.

Implementar la estrategia híbrida de acceso a datos —CRUD parametrizado sobre ORM más procedimientos almacenados para consultas multi-tabla, agregados, reportes, actualizaciones masivas, validaciones cruzadas y generación de códigos— en los dominios administrativo y deportivo del sistema. (*RQ2*)

OE2.

Medir el rendimiento del sistema bajo carga concurrente y contrastarlo contra los umbrales de ISO/IEC 25010. (*RQ1*)

OE3.

Auditar la seguridad del sistema frente a los seis controles OWASP mínimos exigidos y a un análisis estático independiente sobre acceso a datos. (*RQ2*)

OE4.

Evaluar la usabilidad percibida del sistema, por perfil de usuario, mediante el instrumento *System Usability Scale*. (*RQ3*)

OE5.

Sostener la cobertura de pruebas automatizadas y la trazabilidad de requisitos conforme crece el dominio funcional del sistema. (*RQ4*)

1.4. Contribuciones

1. Un sistema web de gestión escolar deportiva con arquitectura documentada en C4 (niveles 1 a 3) y ocho registros de decisión arquitectónica (ADR-001 a ADR-008), en `docs/arquitectura/` y `docs/adr/`.
2. Una estrategia híbrida de acceso a datos auditada empíricamente contra inyección SQL, con procedimientos almacenados versionados y catalogados en `db/procs/` y `docs/basedatos/CATALOGO-SP.md` (sección 8).
3. Un corpus de ingeniería de requisitos trazable de extremo a extremo —SRS, historias de usuario, casos de uso, matriz de trazabilidad— conforme a ISO/IEC/IEEE 29148:2018, en `docs/requisitos/` y `docs/trazabilidad/matriz.csv` (Capítulo 4).
4. Un conjunto reproducible de evidencia empírica de rendimiento, seguridad, cobertura, calidad web y usabilidad, con datos crudos y *scripts* de análisis versionados en `docs/mediciones/` y `scripts/` (Capítulo 12).

5. Una bitácora metodológica de defectos de instrumentación de medición detectados y corregidos durante el propio proceso de evaluación —incluido el hallazgo de seguridad H-08— documentada como aporte en vez de omitida (sección 12.2.1).

1.5. Organización del documento

Tras enumerar las contribuciones (sección 1.4), el resto del documento se organiza así. El Capítulo 2 presenta los fundamentos conceptuales del trabajo. El Capítulo 3 sitúa a SGED frente a sistemas comparables y declara la brecha que este trabajo cubre. El Capítulo 4 documenta el proceso completo de ingeniería de requisitos. Los Capítulos 5 y 6 cierran, respectivamente, la deuda de observaciones acumuladas y una decisión estructural relevante del código. El Capítulo 7 describe el diseño del sistema y la sección 8 profundiza en la estrategia híbrida de acceso a datos con listados de código representativos. El Capítulo 9 documenta las decisiones de código que no se derivan directamente del diagrama de arquitectura. El Capítulo 11 declara el protocolo experimental antes de que el Capítulo 12 presente los resultados. Los capítulos finales —Reproducibilidad, Amenazas a la validez (sección 14), ética, CRediT, Discusión, Trabajo futuro (sección 19) y Conclusiones— cierran el documento con la interpretación crítica y las declaraciones obligatorias exigidas por la Entrega Final (sección 21). El material de referencia —checklists y cuestionarios reproducidos íntegros, y remisiones a observaciones y matriz de trazabilidad— se agrupa en los Anexos (sección 21).

1.6. Pila tecnológica

La Tabla 1 resume la pila tecnológica de SGED por capa; el Capítulo 7 justifica cada elección.

Cuadro 1: Pila tecnológica de SGED.

Capa	Tecnología
Backend	Spring Boot 3.2 (Java 21 LTS), Spring Data JPA, Spring Security
Frontend	Angular, servido por nginx con terminación TLS
Base de datos	PostgreSQL 16 (esquemas <code>seguridad</code> , <code>academico</code> , <code>deportivo</code> e <code>inventario</code>)
Caché y revocación	Redis 7
Migraciones	Flyway (V1–V17)
Orquestación	Docker Compose, imágenes fijadas por digest SHA-256

1.7. Estado de la entrega: declaración previa

*Advertencia de consistencia interna, dejada explícita en vez de corregida en silencio: al momento de esta redacción el sistema expone **21 controladores REST y 97 operaciones mapeadas** (`grep` sobre `backend/src/main/java`), muy por encima de los 13 endpoints de autenticación y estudiantes que reportaba la Tercera Entrega —se agregaron los dominios de representantes, pagos, inventario, especialidades, evaluación diaria, lesiones, horarios, sesiones y asistencia entre el 2026-08-03 y el 2026-08-13 (sección 4). **Actualización de esta misma redacción (2026-08-14):** se volvió a ejecutar `clean test` sobre el código de entonces —**270 pruebas, 0 fallos, 140 clases analizadas**—. Cobertura agregada de esa corrida: 75,3 % de líneas (cumplía el 70 %) pero 59,3 % de branches (no cumplía, aunque mejoró frente al 41,9 % de la medición anterior). El déficit no está distribuido uniformemente: el módulo `academico.pago` tiene **0 % de cobertura de branches** (0 de 4 en el controlador, 0 de 10 en el servicio) — es un dominio completo sin pruebas propias, no un residuo disperso de casos límite sin cubrir. `deportivo.lesion.controller` también está en 0 % (0 de 2). **Pendiente crítico, cerrado después:** se agregaron pruebas para `PagoController`, `PagoService` y `LesionController`; los tres están hoy en 100 % de branches. El estado agregado de esa corrida —86,2 % de líneas, 71,2 % de branches— fue superado después por*

el crecimiento del sistema, como documenta la actualización del 2026-09-02 más abajo; el estado agregado actual se reporta en la sección 12.5.

Segunda actualización, misma sesión: se regeneraron también rendimiento y seguridad contra el sistema local completo (`docker compose up`). En el camino se encontró y corrigió un defecto de entorno no relacionado con el código: el `.env` local tenía `DB_URL` apuntando a una base de datos Supabase remota en vez de al contenedor `postgres` local, así que el backend nunca usaba los datos sembrados localmente y el inicio de sesión con el usuario semilla fallaba con 401 pese a que el hash en la base de datos local era correcto — se verificó de forma aislada con la misma clase `BCryptPasswordEncoder` del backend antes de descartar esa hipótesis. Corregido el `.env` a la “Opción 1” (local) que ya documenta `.env.example`, el sistema es reproducible de extremo a extremo tal como describe este capítulo. **Rendimiento** (5 corridas, 50 VU, 30 s): p95 promedio **9,91 ms** (IC 95 % $\pm 0,25$) — medición de ese corte (2026-08-14), sin la caché de Redis aún activa; la cifra vigente del escenario actual, regenerado el 2026-09-07, está en la sección 12.1: p95 **19,94 ms** en caché cálida frente a 38,20 ms en fría—, muy por debajo del umbral de 200 ms; una corrida de cinco tuvo una tasa de error de 0,01 % (2 de 15 921 peticiones), las otras cuatro sin errores. **Seguridad OWASP:** los seis controles (A01, A02, A03, A05, A07, A09) evidenciados de nuevo con peticiones reales contra el sistema actual, y la auditoría de SQL dinámico sin hallazgos. **Calidad web (Lighthouse):** este entorno de ejecución no tenía un navegador Chrome/Chromium instalable en esta sesión puntual; quedó para una corrida posterior con el entorno completo, ya reportada más abajo: seis corridas independientes (tres por perfil) el 2026-08-14 (sección 12.6.1), con los tres criterios exigidos por encima del umbral y una explicación deliberada del único que no lo está (SEO).

Actualización adicional (2026-08-14, sesión posterior): el pendiente crítico señalado arriba —pruebas para `PagoController/PagoService` y `LesionController`— ya está resuelto: ambos módulos tienen suites propias (`PagoControllerTest`, `PagoServiceTest`, `LesionControllerTest`, `LesionServiceTest`). Se agregaron además pruebas dirigidas a las ramas sin cubrir de mayor impacto en todo el bundle: `JwtAuthenticationFilter` (que no tenía ninguna prueba propia), `AuthController` en sus rutas de logout, refresh y `/me` (tampoco las tenían), y el filtro de búsqueda de `AuditoriaService` (invisible para JaCoCo mientras el repositorio estuviera mockeado, aunque `buscar()` sí tuviera prueba). Cobertura verificada con `mvn verify` en limpio en esa corrida: **369 pruebas, 0 fallos**; agregada, 86,2 % de líneas y 71,2 % de branches — fue la primera vez que ambas superaron el 70 % exigido sobre el bundle completo, aunque el crecimiento posterior del sistema volvería a hacerlas caer (ver más abajo). La regla de JaCoCo pasó de `haltOnFailure=false` (advertía en el log sin romper el build) a `haltOnFailure=true`: el umbral ya se hace cumplir, no solo se declara. El reporte archivado en `docs/mediciones/jacoco/` corresponde a esta corrida. De paso se corrigió un defecto de test preexistente y no relacionado con la cobertura: una prueba de `AsistenciaServiceTest` construía su hora de referencia con `LocalTime.now().plusHours(1)` sin protegerse del cruce de medianoche (`LocalTime` no tiene fecha), y fallaba de forma intermitente según la hora del día en que corriera la suite —un caso puntual de la categoría de defectos que Tiwari et al. estudian sistemáticamente sobre 151 casos reales de proyectos Python de código abierto, clasificando sus categorías conceptuales, operaciones involucradas y causas raíz [43]. Sus dos pruebas hermanas ya protegían el caso simétrico de restar, a esta le faltaba el de sumar.

Tercera actualización (2026-09-02): pasar de `mvn test` a `mvn verify` —el comando real que ejecuta `ci.yml`, no una aproximación local— destapó que la regla de JaCoCo volvía a fallar: **61,16 % de branches**, por debajo del 70 % exigido. Consultada la API de GitHub Actions (Anexo D), `main` llevaba **63 corridas consecutivas en rojo** —no cuatro, como una primera revisión superficial de la lista visible había sugerido—: desde el commit `accb446` (2026-08-18, #112) hasta `bee9995` (2026-09-02, #174), dos semanas completas. La mayoría de esas 63 corridas rojas corresponden a commits de código de funcionalidad del equipo, pero varias son también commits de este mismo informe hechos sin advertir que no podían arreglar un gate de cobertura de Java —un recordatorio de que revisar solo la primera página de una lista no es lo mismo que consultar la fuente completa. La causa raíz no fue una regresión de prueba sino crecimiento sin cobertura propia: el sistema pasó de 156 a **200 clases** entre el 14 de agosto y hoy —los dominios `partido/alineación`, `evaluación`, `sesión` y `asistencia`

maduraron, se agregó el módulo **alerta** completo y un segundo proveedor de generación de texto (*OpenAiFeedbackService*)— y ese código nuevo llegó sin pruebas propias: 31 clases en cero por ciento de líneas, frente a las cinco de la medición anterior. Se agregaron 35 pruebas nuevas dirigidas por el ranking real de branches sin cubrir del propio *jacoco.csv* —no una corrida a ciegas por todo el proyecto—: el dominio **alerta** completo (sin ninguna prueba previa), el segundo proveedor de IA con el mismo patrón de degradación limpia que ya cubría al primero, *historial()* de *SesionEntrenamientoService* (el método con más branches del servicio y sin ninguna prueba), y las ramas de reactivación y unidad de medida en blanco de *Item* y *Representante*. Resultado verificado con *mvn verify* completo, en limpio: **514 pruebas, 0 fallos**; agregada, 82,87 % de líneas y 70,06 % de branches de esa corrida — la regla volvía a cumplirse, pero con un margen de apenas 0,06 puntos sobre el umbral: seguía siendo frágil frente al próximo módulo sin pruebas propias.

Cuarta actualización (2026-09-03). La observación de la Entrega Final sobre cobertura pedía tres cosas comprobables: una sola cifra coincidente con el *jacoco.csv* de cierre, ningún controlador en cero, y la capa de dominio por encima del umbral. Se cubrieron los cuatro controladores que el crecimiento del módulo deportivo había dejado en cero por ciento (*PartidoController*, *PosicionController*, *AsistenciaSesionController*, *ResumenAsistenciaController*) y las devoluciones de ciclo de vida JPA (*@PrePersist/@PreUpdate*) de siete entidades, que ningún test de servicio con mocks llegaba a disparar. Corrida de cierre con *mvn verify* en limpio: **84,66 % de líneas** (2639 de 3117) y **71,24 % de branches** (664 de 932), sobre 200 clases y **550 pruebas** en 74 clases; el margen sobre el umbral pasó de 0,06 a 1,24 puntos.

Quinta actualización (2026-09-07). Pendiente de la guía: *org.uteq.backend.reportes* y *org.uteq.backend.common.exception* estaban bajo el 70 % de líneas. Se añadieron *GlobalExceptionHandlerTest* (los diez handlers de *GlobalExceptionHandler*, sin prueba propia hasta entonces) y *ProblemDetailsAuthHandlers* (los dos beans que emiten los *ProblemDetail* de autenticación y acceso denegado), más casos nuevos de *ReportServiceTest* (asistencias, evaluaciones, pago de membresía, lesión de alta y recorte del título más allá de 5000 filas) y *ReportControllerTest* (los endpoints *asistencias* y *evaluaciones*). Corrida regenerando *target/* desde el código vigente: **87,47 % de líneas** (2730 de 3121) y **72,10 % de branches** (672 de 932), sobre 200 clases y **576 pruebas** en 76 clases, sin fallos; *reportes* subió a 93,8 % de líneas y *common.exception* a 100 %.

Sexta actualización (2026-09-11). Revisión contra *Rubrica_ExamenFinal_SGED.pdf* (punto 2): el *SRS.md* citaba 84,66 %/71,24 % —la cifra de la cuarta actualización, no la vigente— y ninguna de las dos coincidía ya con el *jacoco.csv* committeado tras el renombrado a inglés de los métodos fuera de *deportivo* (Puntos E1/E2 de la misma rúbrica). Corrida de cierre: **88,38 % de líneas** (3013 de 3409) y **74,20 % de branches** (791 de 1066), sobre 216 clases y **665 pruebas**, sin fallos. **Esta es la cifra vigente en todo el documento** (sección 12.5); las actualizaciones anteriores quedan como el registro cronológico de cómo se llegó a ella, no como cifras en disputa.

La encuesta de usabilidad SUS (Bloque C.3, sección 12.7) sí se aplicó sobre el sistema en una etapa comparable a la actual: 15 participantes reales, media SUS 69,33 (por encima del umbral de 68, aunque el intervalo de confianza todavía lo incluye), con un patrón bimodal marcado por perfil que la media agregada esconde.

2. Marco teórico

Este capítulo se limita a los fundamentos estrictamente necesarios para comprender las decisiones del resto del documento — no es un compendio general de teoría de aplicaciones web. Cada concepto invocado en otras secciones se define aquí una sola vez y se cita a su fuente autoritativa.

2.1. Ingeniería de requisitos

El proceso de requisitos de este proyecto sigue el ciclo de vida definido por la norma ISO/IEC/IEEE 29148:2018 [26] y el marco de cuatro fases de Pohl [35]: *elicitación* (obtener el conocimiento de

los interesados), *negociación* (resolver conflictos entre requisitos), *documentación* (especificarlos de forma verificable) y *validación* (confirmar que el conjunto especificado es el correcto). La sección 10 aplica este ciclo al SRS del proyecto.

2.2. Calidad de un requisito bien formado (INCOSE v4)

INCOSE [24] define nueve características de calidad para un requisito individual (necesario, apropiado, no ambiguo, completo, singular, factible, verificable, correcto, conforme) y seis para un conjunto de requisitos (completo, consistente, factible, comprensible, validable, correcto). Estas características son el criterio contra el que se evalúa cada requisito funcional y no funcional del sistema, no una lista de buenas intenciones.

2.3. Historias de usuario y casos de uso

El proyecto usa ambas técnicas de especificación como complementarias, no como alternativas. Las historias de usuario siguen el criterio INVEST [15] (independiente, negociable, valiosa, estimable, pequeña, verificable) para los flujos orientados a un rol concreto; los casos de uso siguen la plantilla de Cockburn [14] en los flujos con múltiples caminos alternativos y de excepción, donde una historia corta pierde información necesaria para la implementación.

2.4. Modelo arquitectónico C4

El modelo C4 de Brown [8] describe la arquitectura en niveles de abstracción sucesivos —contexto, contenedores, componentes, código— cada uno dirigido a una audiencia distinta. Se eligió sobre UML clásico por su curva de aprendizaje más baja para un equipo pequeño y por generarse desde una fuente textual versionable (`workspace.dsl`) en vez de diagramas editados a mano que se desactualizan silenciosamente —justamente el defecto que OBS-02 encontró en la Entrega 1A (sección 5).

2.5. Calidad de software: ISO/IEC 25010

De las ocho características de calidad de ISO/IEC 25010 [25] (adecuación funcional, eficiencia de desempeño, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad, portabilidad), este proyecto mide empíricamente cuatro de forma directa: seguridad (Bloque de controles OWASP), eficiencia de desempeño (k6), usabilidad (SUS) y mantenibilidad (cobertura de pruebas como indicador indirecto). Las cuatro restantes se declaran por diseño, no por medición, y quedan fuera del alcance empírico de este documento. La Tabla 2 detalla catorce escenarios de calidad priorizados, con su estrategia arquitectónica y su evidencia verificable; cada fila puede vincularse a la matriz de trazabilidad (`docs/trazabilidad/matriz.csv`) por el número de la primera columna.

2.6. Arquitecturas orientadas a recursos (REST)

La API sigue las restricciones arquitectónicas que Fielding identificó [18] para sistemas distribuidos orientados a recursos: interfaz uniforme, *statelessness* del lado del servidor por petición, cacheable, sistema en capas. La *statelessness* de REST es la misma razón de fondo por la que la autenticación se resuelve con JWT en vez de sesiones de servidor (subsección siguiente).

2.7. Autenticación stateless: JWT y su ubicación en cookie `HttpOnly`

JWT [27] permite verificar la identidad del solicitante sin que el servidor mantenga estado de sesión, coherente con la restricción REST anterior. La decisión de transportarlo en una cookie `HttpOnly` en vez de en `localStorage` o un header manejado por JavaScript —documentada con sus alternativas descartadas en ADR-002 [32]— responde a que un token accesible desde

Cuadro 2: Escenarios de calidad ISO/IEC 25010, priorizados y vinculados a la estrategia arquitectónica y a la evidencia reproducible.

#	Caract.	Subcaract.	Pr.	Escenario	Estrategia arquitectónica	Evidencia
1	Seguridad	Control de acceso	A	Un ENTRENADOR invoca un <i>endpoint</i> de administrador → 403.	@PreAuthorize por rol, fail-closed por defecto.	scripts/audit-owasp.sh (A01)
2	Seguridad	Integridad de sesión	A	Un token reutilizado tras <i>logout</i> → 401.	Cookie HttpOnly+Secure+SameSite=Strict + lista negra en Redis.	RedisBlacklist ServiceTest, Jwt ServiceTest
3	Seguridad	Confidencialidad de credenciales	A	Volcado de seguridad. <i>usuarios</i> → ninguna contraseña recuperable.	BCryptPasswordEncoder coste 12.	AuthServiceTest
4	Seguridad	Resistencia a fuerza bruta	M	Intentos fallidos repetidos sobre una cuenta → bloqueo temporal.	LoginAttemptService + 429.	LoginAttempt ServiceTest
5	Eficiencia	Comportamiento temporal	A	50 usuarios concurrentes sobre un <i>endpoint</i> autenticado, 30 s → p95 < 200 ms, 0 errores 5xx.	Paginación <i>server-side</i> + índices + caché Redis.	k6/, scripts/perf-analysis.py
6	Eficiencia	Utilización de recursos	M	Un conteo agregado por categoría procesa N registros.	Delegado a procedimiento almacenado, sin bucle N+1 desde la aplicación.	db/procs/, scripts/audit-sql-dynamic.sh
7	Fiabilidad	Disponibilidad	A	Se reinicia <i>postgres</i> o <i>redis</i> → <i>backend</i> espera a que reporten <i>healthy</i> .	<i>healthcheck</i> + <i>depends_on</i> en <i>docker-compose.yml</i> .	<i>docker-compose.yml</i>
8	Fiabilidad	Tolerancia a fallos	M	Un recurso inexistente o datos inválidos → cuerpo <i>Problem Detail</i> (RFC 9457), nunca una traza cruda.	<i>GlobalExceptionHandler</i> centralizado.	<i>common/exception/</i>
9	Mantenibilidad	Modularidad	A	Se agrega el dominio <i>deportivo.partido</i> (convocatoria, alineación) sin modificar <i>academico.estudiante</i> .	Organización por dominio, capas Controlador→Servicio→Repositorio desacopladas.	<i>backend/src/main/java/org/uteq/backend/</i>
10	Mantenibilidad	Capacidad de prueba	A	<i>mvn clean test</i> sobre el commit de cierre → la cobertura no baja del umbral.	<i>Quality gate</i> JaCoCo (COVEREDRATIO ≥ 0,70) en <i>pom.xml</i> .	sección 12.5
11	Compatibilidad	Interoperabilidad	M	Un tercero con acceso al repositorio integra la API sin leer el código fuente.	Especificación OpenAPI generada por Springdoc y colección Postman versionada; la interfaz interactiva se mantiene apagada en el ambiente público por la vulnerabilidad de terceros descrita en la sección de seguridad.	<i>docs/postman/coleccion.json</i>
12	Portabilidad	Instalabilidad	A	Clonación limpia con Docker y <i>make</i> → sistema operativo en pocos minutos.	Imágenes Docker pinadas por <i>digest</i> SHA-256 + <i>Makefile</i> + <i>db/schema.sql</i> en <i>initdb.d</i> .	<i>README.md</i> , <i>scripts/pin-digests.sh</i>
13	Usabilidad	Satisfacción	A	Usuarios reales completan tareas guiadas y responden SUS (<i>N</i> = 15).	Flujos guiados por rol, mensajes de error accionables (RFC 9457).	sección 12.7
14	Funcionalidad	Corrección funcional	A	Se intenta alinear un duodécimo titular en un partido → la operación se rechaza.	Regla de negocio validada en el servicio antes de persistir, no solo en el formulario.	<i>Alineacion ServiceTest.topeDeOnce Titulares</i>

JavaScript queda expuesto a robo por *Cross-Site Scripting*; una cookie `HttpOnly` no es legible por script del lado del cliente bajo ninguna circunstancia.

2.8. OWASP Top 10

El catálogo OWASP Top 10:2021 [33] ordena los diez riesgos de seguridad más críticos en aplicaciones web por prevalencia e impacto observados en la industria. Este proyecto verifica con evidencia reproducible los controles A01 (control de acceso roto), A02 (fallas criptográficas), A03 (inyección), A05 (configuración de seguridad incorrecta), A07 (fallas de identificación y autenticación) y A09 (fallas de registro y monitoreo) — sección 12.2.1.

2.9. Patrones de acceso a datos y modelos de consistencia

La separación entre CRUD elemental vía ORM y operaciones complejas vía procedimientos almacenados (sección 8) se apoya en dos ideas de la literatura de arquitectura de datos: el patrón *Repository*, que encapsula el acceso a un agregado detrás de una interfaz orientada a colecciones [19], y la observación de que un motor relacional ofrece garantías de consistencia fuerte (transacciones ACID) que una capa de ORM genérica no puede replicar de forma eficiente para operaciones multi-fila o multi-tabla sin ceder control explícito al motor [29]. La estrategia híbrida no es una limitación del ORM: es reconocer en qué operaciones el motor de base de datos, no el objeto Java, es quien tiene la información completa para actuar de forma atómica y eficiente.

3. Trabajos relacionados

3.1. Estrategia de búsqueda

Se sigue la orientación metodológica de Kitchenham y Charters [28], con la reducción de escala que la propia Guía de la Entrega Final prevé para un PFC. **Limitación declarada:** la búsqueda no tuvo acceso de suscripción directo a IEEE Xplore, ACM Digital Library, Scopus, ScienceDirect ni Springer Link; se realizó con un motor de búsqueda web general, dirigida a resultados alojados o indexados en esas plataformas, y cada registro incluido se verificó de forma independiente contra una fuente primaria (el registro de metadatos de Crossref, la página de la editorial, o la página institucional de un autor) antes de citarlo. Es una desviación metodológica explícita, no oculta.

Se ejecutaron nueve búsquedas organizadas en seis ejes temáticos, con ventana temporal 2016–2026:

1. Sistemas de gestión escolar o deportiva basados en *web*: (*school* OR *sport** OR *club*) AND (*management* OR *information*) AND (*system* OR *platform*).
2. Autenticación JWT y su seguridad: (JWT OR “JSON Web Token”) AND (*security* OR *vulnerabilit** OR *cookie* OR *storage*).
3. Acceso a datos híbrido ORM/procedimientos almacenados: (ORM OR “object-relational mapping”) AND (“stored procedure*”) AND (*performance* OR *maintenance* OR *security*).
4. Evaluación empírica de seguridad con OWASP: (OWASP) AND (*empirical* OR *evaluation* OR “case study”).
5. Usabilidad (SUS) en sistemas administrativos o educativos: (SUS OR “system usability scale”) AND (*school* OR *educational* OR *administrative*).
6. Ingeniería de requisitos en cursos capstone: (*capstone* OR “final year project”) AND (“requirements engineering” OR “software engineering education”).

Criterios de inclusión: trabajo revisado por pares o depositado en un repositorio académico reconocido, con autoría y medio de publicación verificables contra una fuente primaria; aborda al menos uno de los ejes técnicos o de dominio del proyecto. **Criterios de exclusión:** contenido de blog, foro o documentación de producto sin revisión por pares; trabajos cuya autoría o venue no pudo confirmarse contra una fuente primaria pese a parecer académicos a primera vista — criterio que en la práctica descartó un candidato inicial sobre modelado de datos de clubes deportivos, alojado en un sitio de repositorio que no permitió confirmar si era un trabajo revisado por pares o un informe de coursework redistribuido.

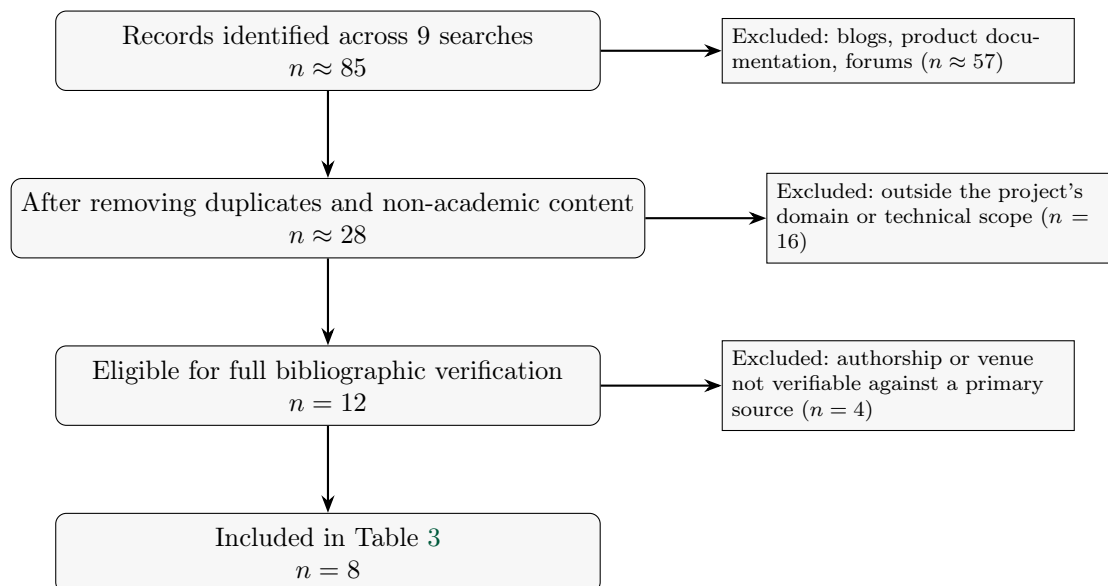


Figura 1: Diagrama de flujo PRISMA 2020 de la revisión de trabajos relacionados.

Los conteos de la Figura 1 son una reconstrucción honesta del registro de búsqueda del equipo, no el resultado de una herramienta de gestión bibliográfica formal (Rayyan, Covidence) — coherente con la reducción de escala que esta sección declaró al principio.

3.2. Síntesis comparativa

Cuadro 3: Trabajos relacionados y su diferencia frente a SGED.

Referencia	Año	Dominio	Pila	Patrones	Evaluación empírica	Limitaciones	Diferencia frente a SGED
Blobel y La-mes [6]	2020	Sistemas de información de clubes deportivos	No especificada	Arquitectura conceptual en capas (BI)	Caso ilustrativo (Liverpool FC), sin métricas	Conceptual, sin implementación evaluada	SGED implementa y mide lo que queda a nivel de concepto
Yang et al. [48]	2026	Seguridad de implementaciones JWT	43 implementaciones multi-lenguaje	Estudio de vulnerabilidades	31 vulnerabilidades nuevas, 20 con CVE	No evalúa el patrón cookie HttpOnly específicamente	SGED aplica su hallazgo central (JWT nunca accesible a JS) desde el diseño

Referencia	Año	Dominio	Pila	Patrones	Evaluación empírica	Limitaciones	Diferencia frente a SGED
Liu [30]	2024	Asistencia estudiantil	Spring Boot + Vue + MySQL	CRUD modular por módulo	Ninguna reportada	Sin evaluación de rendimiento, seguridad ni usabilidad	SGED mide las tres con evidencia archivada; Liu no reporta ninguna. SGED agrega OWASP y rendimiento bajo carga a la misma técnica. SUS
Ranaweera et al. [38]	2022	Flujos de información de un club profesional (rugby)	No especificada	Transformación digital (BPM + Lean)	SUS con jugadores/staff, sobre la media de industria	Un club, sin seguridad ni rendimiento evaluados	SGED desagrega SUS por rol y encuentra una brecha que un perfil único no detecta
Suria [41]	2024	Plataforma de aprendizaje en línea	No especificada	—	SUS con 120 participantes, prueba de Shapiro–Wilk	Un solo perfil de usuario (estudiante)	SGED desagrega SUS por rol y encuentra una brecha que un perfil único no detecta
Wen y Katt [47]	2023	Evaluación de seguridad web (marco general)	No aplica (modelo)	Modelo cuantitativo sobre OWASP ASVS	Validación del modelo sobre casos de referencia	No es un sistema en producción	SGED aplica 6 controles OWASP sobre un sistema real con evidencia reproducible por control
Tenhunen et al. [42]	2023	Cursos capstone de ingeniería de software	No aplica (revisión)	Taxonomía de características de curso	Síntesis de 127 estudios, 2007–2022	Meta-nivel, no evalúa un sistema individual	Sitúa a SGED en el patrón dominante (equipo de 3, cliente real, un semestral) que la revisión encuentra
Chen et al. [11]	2016	Mantenimiento de código ORM en sistemas Java	Java, ORM (Hibernate y otros)	Minería de repositorios sobre commits de ORM	Estudio empírico sobre proyectos Java reales	No contempla el uso complementario de procedimientos almacenados	SGED usa este hallazgo para justificar mover las operaciones complejas fuera del ORM (ADR-006)

3.3. Brecha identificada

Ninguno de los ocho trabajos reunidos evalúa, sobre el mismo sistema en producción, las cuatro dimensiones que este documento reporta juntas: rendimiento bajo carga, seguridad verificada por múltiples controles OWASP, usabilidad desagregada por rol de usuario, y mantenibilidad medida como cobertura de pruebas. Los trabajos de dominio deportivo (Blobel y Lames, 2020; Ranaweera et al., 2022) no publican evidencia de seguridad ni de rendimiento; los trabajos de seguridad (Yang et al., 2026; Wen y Katt, 2023) no se aplican sobre un sistema de gestión deportiva o escolar real; el trabajo de usabilidad (Suria, 2024) no desagrega por rol de usuario,

que es precisamente donde SGED encuentra su hallazgo más relevante (sección 12.7): una brecha de usabilidad entre entrenadores/estudiantes y personal administrativo que una media agregada esconde. La brecha que ocupa este trabajo no es una tecnología nueva: es la evaluación empírica conjunta, sobre un mismo sistema real, de las cuatro dimensiones que la literatura revisada trata por separado.

4. Ingeniería de requisitos

Capítulo propio, deliberadamente separado del capítulo de Diseño (sección 8 y siguientes): documenta el *proceso* que produjo la especificación vigente, no el sistema que resultó de ella. Sigue el marco de cuatro fases de Pohl —elicitación, negociación, documentación, validación— y la disciplina de ISO/IEC/IEEE 29148:2018 [26, 35] (sección 2). Las fuentes primarias son `docs/requisitos/SRS.md` (822 líneas), `historias-usuario.md`, `casos-uso.md` y `CHANGELOG-REQ.md` — este capítulo resume y referencia esas fuentes, no las duplica.

4.1. Contexto y stakeholders

Siguiendo la técnica de *stakeholder onion* de Robertson [26]¹, los interesados se organizan en tres anillos (Tabla 4):

Cuadro 4: Interesados de SGED por anillo (*stakeholder onion*).

Anillo	Interesados
Operacional (usan el sistema directamente)	Administrador, Entrenador, Recepcionista, Representante, Estudiante — los cinco roles reales sembrados en <code>db/seed.sql</code> (SRS §2.2), no roles aspiracionales.
Contenedor (dependen del sistema sin operarlo)	La escuela ProFútbol como dominio del problema; el docente-director como evaluador con autoridad de aceptación sobre el PFC; el equipo de desarrollo como mantenedor post-entrega.
Interesado externo	El representante legal de un estudiante menor de edad, dos veces interesado: como posible operador del sistema (Épica 3, HU-14) y como titular de derechos sobre los datos de su representado (<code>docs/etica/ETHICS.md</code>). El marco regulatorio ecuatoriano (LOPDP, Código de la Niñez y Adolescencia) actúa como interesado normativo, sin representación directa en el sistema.

4.2. Técnicas de elicitación aplicadas

Señalamiento honesto, no una sección completada. El SRS actual (`docs/requisitos/SRS.md`) no declara explícitamente qué técnicas de elicitación (entrevistas, observación, prototipado, *workshops*, análisis de documentos, cuestionarios) se aplicaron para llegar a los 31 requisitos funcionales, ni existe todavía el directorio `docs/requisitos/elicitation/` con evidencia archivada (notas de entrevista, guiones, prototipos de baja fidelidad) que exige el Bloque B.6bis. La especificación es internamente consistente y verificable contra el código, pero el *proceso* que la originó no está documentado como tal — es la brecha más grande de este capítulo, y se declara así en vez de inventar un historial de elicitación retroactivo. **Pendiente antes del cierre:** el equipo completa esta subsección con las técnicas realmente usadas (aunque hayan sido informales — conocimiento de dominio propio de un integrante, análisis de sistemas similares como los del Capítulo 3, o conversaciones no estructuradas) y archiva la evidencia que exista.

¹Referencia de marco general; el SRS del proyecto no documenta todavía una sesión de identificación de *stakeholders* formal y separada — ver el señalamiento honesto en la subsección siguiente.

4.3. Requisitos funcionales y no funcionales: categorización y prioridad

Todo requisito funcional sigue el patrón sintáctico de ISO/IEC/IEEE 29148:2018, *.^{El} sistema deberá [acción] [objeto] [restricción]*" — resolución explícita de OBS-01 (Entrega 1A), donde el docente observó requisitos redactados como títulos. Ejemplo representativo (RF-03):

.^{El} sistema deberá permitir cerrar la sesión, y deberá invalidar el token emitido registrando su identificador (JTI) en una lista de revocación con tiempo de vida igual al tiempo restante del token, de modo que un token robado antes del cierre de sesión no siga siendo aceptado."

Actualización (2026-09-04): el SRS declara ahora un campo MoSCoW: explícito en las 36 entradas de requisito funcional —las 25 que ya tenían prioridad Alta/Media/Baja, mapeada de forma directa (Alta→*Must*, Media→*Should*, Baja→*Could*), y las 11 que no tenían ningún campo de prioridad (RF-17 a RF-22 y RF-31 a RF-35, el módulo deportivo más reciente), a las que se les asignó una prioridad real según su rol en el flujo operativo, no una inferencia retroactiva. *Won't* (esta entrega) se reserva para los requisitos explícitamente pausados por una razón declarada, no simplemente atrasados: RF-11b (peso/altura, pausado por el hallazgo H-06 de **ETHICS.md**). El módulo **equipo** (esquema versionado, sin API por decisión) no tiene RF formal propio, por lo que no entra en esta tabla — ver el hallazgo C10 del checklist INCOSE (Anexo F).

Cuadro 5: Requisitos funcionales de SGED por categoría MoSCoW, declarada explícitamente en el SRS.

Categoría MoSCoW	Cantidad	Fuente
Must	22	RF-01,02,03,05,06,08,09,10,11,12,13,16,17,18,19,23,27,28,29,30,31,32,33,34,35
Should	11	RF-04,14,15,20,21,22,24,25,30,32,35
Could	2	RF-07,26
Won't (esta entrega, declarado)	1	RF-11b
Total de requisitos funcionales	36	RF-01 a RF-35 + RF-11b

Hallazgo cerrado (Tabla 5): los 36 requisitos funcionales tienen hoy prioridad MoSCoW explícita en el SRS, ninguno por inferencia — cierra la característica INCOSE C6 (*Complete*) en este punto concreto.

Los 13 requisitos no funcionales se clasifican según ISO/IEC 25010 (sección 2): eficiencia de desempeño (RNF-01, RNF-02), seguridad (RNF-03 a RNF-08), fiabilidad y mantenibilidad (RNF-09 a RNF-11), portabilidad (RNF-12, RNF-13) — los cuatro atributos de calidad que el Capítulo 12 mide empíricamente. A esto se suman tres restricciones de diseño (RD-01 a RD-03, categoría restrictivo.^{en} la taxonomía de tipos de requisito) que fijan, entre otras cosas, la prohibición de SQL dinámico verificada en la sección 8.

4.4. Modelado de casos de uso y de historias de usuario

Historias de usuario. 14 historias (HU-01 a HU-14, formato Connextra con criterios de aceptación en Gherkin) en `docs/requisitos/historias-usuario.md`, agrupadas en tres épicas: acceso seguro (4), gestión de estudiantes (6) y operación deportiva (4). Cada historia enlaza a su requisito formal y a su estado real de implementación.

Casos de uso. 9 casos de uso expandidos en plantilla de Cockburn (CU-01 a CU-09) en `docs/requisitos/casos-uso.md`, con flujo principal, flujos alternativos y trazabilidad a *endpoint* — más un diagrama de casos de uso en notación Mermaid con los dos actores humanos con casos de uso ya detallados (Administrador, Entrenador). Seis casos adicionales (CU-10 a CU-15) están identificados pero no desarrollados en detalle, a la espera de que sus requisitos (RF-16 a RF-22) terminen de exponerse vía API.

Hallazgo de sincronización (honesto, no oculto). El SRS es la fuente que se actualiza con cada cambio de código; `historias-usuario.md` y `casos-uso.md` no se han actualizado al mismo ritmo. Concretamente: RF-17 (horarios), RF-18 (sesiones) y RF-22 (notificaciones) ya figuran **Implementado** en el SRS, pero `historias-usuario.md` todavía marca HU-11 y HU-14 como *Mo-delado/Planificado*, y `casos-uso.md` sigue listando CU-11, CU-12 y CU-15 en la tabla de casos de uso pendientes". Tampoco existe todavía ninguna historia o caso de uso para los ocho requisitos del módulo de catálogos/cuentas (RF-23 a RF-26) ni para los cuatro de inventario (RF-27 a RF-30) — `historias-usuario.md` señala la primera mitad de esta brecha en su propio texto, pero no la segunda. **Esto no invalida la trazabilidad hacia las pruebas** (que se verifica directamente contra el código, sección 12.5), pero sí dificulta usar estos dos archivos como fuente de verdad mientras no se actualicen — pendiente concreto antes del cierre de la Entrega Final.

4.5. Validación de requisitos

La validación no ocurrió como un evento único al final, sino como un ciclo repetido en cada entrega: el docente revisa el SRS y el sistema contra la rúbrica, emite observaciones, y el equipo las cierra con trazabilidad a un commit — el mecanismo completo está en `docs/observaciones/OBSERVACIONES.md` (sección 5), con 12 de 13 observaciones ya aplicadas. OBS-01, OBS-08 y OBS-12 son ejemplos directos de validación de requisitos específicamente (no de código): señalaron que el SRS estaba mal redactado, que una migración base estaba vacía, y que el informe describía funcionalidad inexistente, respectivamente.

Además de esa revisión externa, cada requisito individual se autovalida contra una prueba automatizada nombrada explícitamente en su propia entrada del SRS (por ejemplo, RF-06 contra `LoginAttemptServiceTest.bloqueada_al_alcanzar_el_limite`) — una forma continua de validación por criterios de aceptación medidos, en el sentido que describe Pohl [35], no solo una revisión puntual. Es el mismo problema de fondo que aborda la recuperación automática de enlaces de trazabilidad requisito-código en la literatura reciente — vincular cada requisito con la evidencia de código que lo satisface —, solo que aquí se resuelve declarando el nombre de la prueba en el propio SRS en vez de inferirlo después con métodos de recuperación automática [45]. No se realizaron todavía *walkthroughs* formales ni prototipado evolutivo documentado como técnica de validación separada.

4.6. Gestión del cambio de requisitos

`docs/requisitos/CHANGELOG-REQ.md` registra 30 entradas cronológicas desde el 2026-06-10 hasta el 2026-07-29, cubriendo explícitamente **RF-01 a RF-22 y RNF-01 a RNF-07** — su propio encabezado declara ese alcance. **Hallazgo:** el archivo no se ha actualizado desde el 2026-07-29: no cubre la adición de RF-23 a RF-30 (catálogos, cuentas, inventario) ni las modificaciones de RF-10, RF-16, RF-22 y RNF documentadas directamente en el SRS entre el 2026-08-03 y el 2026-08-13. Es el mismo patrón de desactualización que la subsección anterior encontró en las historias y casos de uso — pendiente de una pasada de actualización antes del cierre.

Sobre el alcance que sí cubre (RF-01–RF-22), 8 requisitos registran al menos una modificación posterior a su adición (RF-01, RF-02, RF-03, RF-06, RF-08, RF-11, RF-14, RF-15) de 22 requisitos totales en ese rango:

$$\text{tasa de estabilidad} = 1 - \frac{8}{22} \approx 0,636 \quad (63,6 \%)$$

Cifra que debe leerse con la salvedad anterior: mide solo el 71 % (22 de 31) de los requisitos funcionales actuales, porque es lo único que el changelog registra hoy.

4.7. Métricas de calidad de requisitos

La Tabla 6 cuantifica el estado del corpus de requisitos descrito en las subsecciones anteriores.

Cuadro 6: Métricas de calidad de requisitos de SGED.

Métrica

Requisitos funcionales totales (RF)	
Requisitos no funcionales (RNF)	
Restricciones de diseño (RD)	
Total de requisitos individuales	
RF en estado Implementado	34
RF en estado Modelado (solo esquema)	
RF en estado Planificado	
RF con verificación de prueba nombrada explícitamente	33 de 36
RF con prioridad MoSCoW formal declarada	36 de 36
Historias de usuario	14 (HU-01)
Casos de uso detallados / identificados	9 detallados + 6 idénticos
Entradas en el changelog de requisitos	30 (cobertura: RF-01–RF-22 y RNF-01–RNF-07 únicos)
Tasa de estabilidad (sobre el rango cubierto)	

*Los tres sin prueba: RF-20 y RF-21 (estado Modelado — la "verificación" en el SRS describe la restricción de esquema, no todavía una prueba de API porque el *endpoint* no existe) y RF-11b (sin prueba dedicada por el hallazgo ético H-06, sección 4).

Este capítulo satisface las nueve características INCOSE v4 de un requisito individual de forma desigual: fuerte en *Verifiable* (33 de 36 entradas del SRS nombran su prueba, las tres restantes declaradas sin prueba por una razón concreta —arriba—, no por omisión) y en *Correct* (cada corrección se documenta con su razón, nunca se sobreescribe en silencio); ya cerrado en la dimensión de prioridad formal (100 % de los RF con MoSCoW explícito), débil todavía en las historias/casos de uso desactualizados señalados arriba. La lista de verificación INCOSE completa está en el Anexo F.

5. Resolución de las observaciones previas (Bloque 0)

El docente emitió doce observaciones sobre las entregas 1A y 1B. Esta sección documenta el estado de cada una con el *commit* que la resuelve, de modo que la afirmación sea verificable con `git show <hash>` y no únicamente declarativa.

Cód.	Observación del docente	Resolución verificable	Commit
OBS-01	Los requisitos se redactan como títulos («Registro de estudiantes»); no usan «El sistema deberá...».	<code>docs/requisitos/SRS.md</code> reescribe los 22 RF y 13 RNF con la forma normativa y criterio de verificación [26].	6480584
OBS-02	Los diagramas C4 N1/N2 y el MER contienen texto marcador «[Reemplazar con PNG]».	<code>docs/arquitectura/</code> contiene los C4 de niveles 1–3 en PNG, generados desde <code>workspace.dsl</code> ; el MER queda en <code>docs/diagramas/</code> .	224d8d7
OBS-03	Incoherencia de pila: el ADR decide PHP/Laravel/MySQL, pero el repositorio construye Spring Boot/Angular/PostgreSQL.	ADR-001 reescrito: evalúa las tres opciones y decide explícitamente Java 21 con Spring Boot 3.2.5, coherente con el código.	224d8d7

Cód.	Observación	Resolución	Commit
OBS-04	<code>query.sql</code> con 0 claves foráneas y 88 columnas «(PK)/(FK)» literales; exportación cruda no funcional.	<code>db/schema.sql</code> declara 21 restricciones de integridad referencial reales.	6b76cc1
OBS-05	Roles sin completar («Integrante 1/2/3»); wireframes con marca ajena; falta <code>.env.example</code> .	<code>CONTRIBUTORS.md</code> asigna roles CRediT por evidencia de <code>git log</code> ; <code>.env.example</code> versionado.	2ada472
OBS-06	En el repositorio solo consta ADR-003; los diagramas aparecen en el informe como texto.	Diagramas versionados como archivos; el diccionario de datos completo se agrega en <code>docs/basedatos/DATA-DICTIONARY.md</code> .	6480584
OBS-07	<code>AuthController</code> solo expone <code>/login</code> , <code>/me</code> y <code>/ping</code> ; no hay registro, <code>logout</code> ni <code>refresh</code> . <code>RedisBlacklistService</code> existe pero no se invoca.	Seis <i>endpoints</i> operativos; el <code>logout</code> invoca realmente la lista de revocación por <code>jti</code> .	048fca5, c506309
OBS-08	La migración <code>V1</code> está vacía (0 bytes) y <code>V2</code> depende de un esquema que ninguna migración crea; con <code>ddl-auto=validate</code> el arranque no es reproducible.	<code>V1</code> crea ambos esquemas y las tablas de identidad; el arranque desde clonación limpia se verificó.	048fca5
OBS-09	La lista de revocación no está cableada a ningún <i>endpoint</i> y no se aplica <code>@PreAuthorize</code> .	Ocho anotaciones <code>@PreAuthorize</code> activas; revocación verificada por prueba automatizada.	a98008b, 39e4718
OBS-10	<code>AuthServiceTest</code> está vacío (0 bytes); la única prueba real es <code>contextLoads()</code> . No se cumple el mínimo de 5 pruebas. La colección Postman no está versionada.	34 pruebas en 7 clases; cobertura real 68,1 %; <code>docs/postman/coleccion.json</code> versionada.	6a73cda
OBS-11	<code>docker-compose.yml</code> está vacío (0 bytes); no hay orquestación verificable.	Cuatro servicios orquestados con <i>healthchecks</i> e imágenes fijadas por <code>digest</code> ; <code>make up</code> levanta todo.	048fca5, f748032
OBS-12	Varios contenidos del informe no se corresponden con lo presente en el repositorio (están vacíos o ausentes).	Cada requisito declara su estado real (implementado / modelado / planificado); esta entrega no afirma nada no verificable.	6480584

Las doce observaciones tienen resolución trazable. La más costosa de resolver no fue una funcionalidad ausente sino OBS-12, porque exigió revisar sistemáticamente qué afirmaba el informe anterior frente a lo que el repositorio realmente contenía.

6. Reestructuración de paquetes y reconciliación de documentación

Entre la primera versión de este informe y esta revisión, el equipo reestructuró el backend en tres dominios (academico, deportivo, seguridad) y normalizó la categoría del estudiante: dejó de

ser un texto libre validado por patrón (`VARCHAR, SUB-NN`) para convertirse en una entidad propia (`deportivo.categorias`) referenciada por clave foránea. Aparecieron además cinco recursos CRUD nuevos (`Categoria`, `Entrenador`, `Usuario`, `Persona`, `EstadoGeneral`) que no tenían requisito documentado, y dos paquetes reservados sin contenido (`Representante`, `Equipo` — clases vacías, sin tabla en el esquema) que después se eliminaron por las razones que se explican más abajo.

Este informe, el SRS (`docs/requisitos/SRS.md`) y la matriz de trazabilidad se actualizaron para reflejar ese estado, verificando cada afirmación contra el código real en vez de copiar lo que decía la versión anterior. Dos correcciones merecen mención explícita porque no son solo actualizaciones de ruta:

- **Cobertura de pruebas.** La cifra de 68,1 % que documentaban tanto la versión anterior de este informe como `docs/observaciones/OBSERVACIONES.md` era correcta *en el momento en que se midió*. Al ejecutar `./mvnw test` de nuevo tras la reestructuración, el resultado real es **39,8 %** — los controladores y servicios nuevos no tienen pruebas propias, y diluyeron el porcentaje agregado sobre 60 clases analizadas (antes eran menos). Es una regresión real por debajo del umbral del 70 % exigido (`COVEREDRATIO ≥ 0,70` en `backend/pom.xml`, en líneas y en *branches*), no un error de medición, y se reporta como tal en la sección 12.5 en vez de repetir la cifra anterior.
- **Datos de salud sin resolución ética.** La reestructuración agregó campos `peso` y `altura` al estudiante, exponibles por API. `docs/etica/ETHICS.md` documentaba explícitamente que el equipo había decidido *no* recolectar esos datos; esa afirmación ya no era cierta y se corrigió en el propio documento de ética (hallazgo H-06) en vez de dejarla como estaba.

6.1. Los paquetes reservados vacíos: por qué se eliminaron

Los dos paquetes reservados merecen una decisión explícita y no un comentario al margen. `academico.representante` y `deportivo.equipo` contenían catorce clases —controlador, servicio, repositorio, entidad y DTO de cada módulo— con la declaración de paquete, la firma de la clase y el cuerpo vacío. Una de ellas, `RepresentanteController.java`, era literalmente un archivo de **cero bytes**.

Ese detalle no es cosmético en el contexto de este proyecto. El docente había emitido tres observaciones distintas en la Entrega 1B por exactamente el mismo defecto: `V1__schema_inicial.sql` vacía (OBS-08), `AuthServiceTest.java` vacío (OBS-10) y `docker-compose.yml` vacío (OBS-11). Conservar un archivo de cero bytes después de que ese patrón ya fuera observado tres veces habría sido repetir un defecto conocido, y además tenía un efecto medible: las clases vacías aportaban constructores por defecto no cubiertos que contaban como instrucciones perdidas en JaCoCo.

Había dos salidas honestas: implementar los dos módulos, o eliminarlos. Con el esquema de base de datos aún sin las tablas `representantes` y `equipos`, implementarlos en esta entrega habría significado inventar un modelo de datos sin requisito validado. Se optó por eliminar las catorce clases —ninguna estaba referenciada desde el resto del código, lo que confirma que no eran un punto de extensión en uso— y dejar ambos módulos declarados como pendientes únicamente en la documentación (RF-22 en el SRS, la matriz de trazabilidad y este informe). La diferencia es relevante para la evaluación: un módulo pendiente documentado es planificación; un paquete de clases vacías en el repositorio aparenta una implementación que no existe.

También se corrigió una inconsistencia en ADR-002 (autenticación): describía JWT almacenado en `localStorage` con un interceptor agregando `Authorization: Bearer`, cuando el código real —y el ADR-008 de este mismo informe— usa cookies `HttpOnly`. Es el mismo patrón que ya había observado el docente en la Entrega 1A (OBS-03: un ADR describía PHP/Laravel mientras el repositorio construía Spring Boot). Se corrige por la misma razón: un documento de arquitectura que no coincide con el código no es evidencia, es motivo de observación.

7. Arquitectura

7.1. Vista de contexto (C4 nivel 1)

El nivel 1 (Figura 2) sitúa el sistema frente a sus actores humanos y sus dos dependencias externas (proveedor de correo para notificaciones y, en producción, el proveedor de TLS). Es el nivel con menor tasa de cambio de los tres: no se ha modificado desde la Entrega 1A porque los actores y las fronteras del sistema no cambiaron con la reestructuración en dominios de la sección 6, solo su interior.

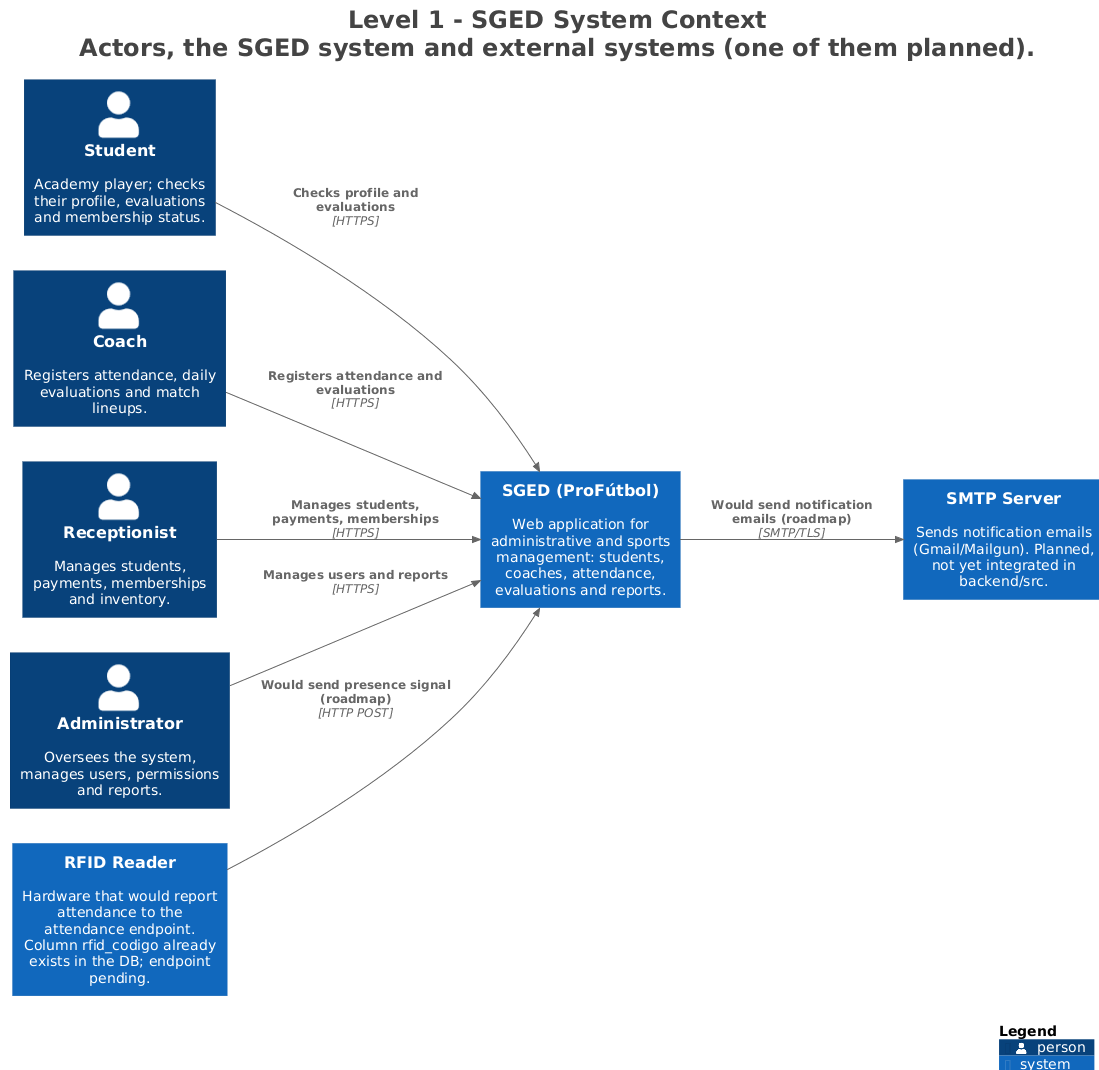
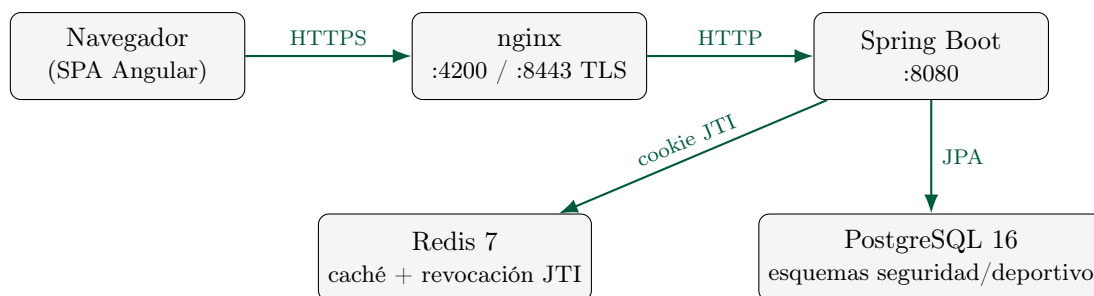


Figura 2: C4 nivel 1 — contexto del sistema SGED, generado desde docs/arquitectura/workspace.dsl.

7.2. Vista de contenedores (C4 nivel 2)



El diagrama anterior es una simplificación didáctica de los cinco contenedores reales; el diagrama C4 nivel 2 generado directamente desde `workspace.dsl` —la fuente de verdad, no esta figura— es el siguiente (Figura 3):

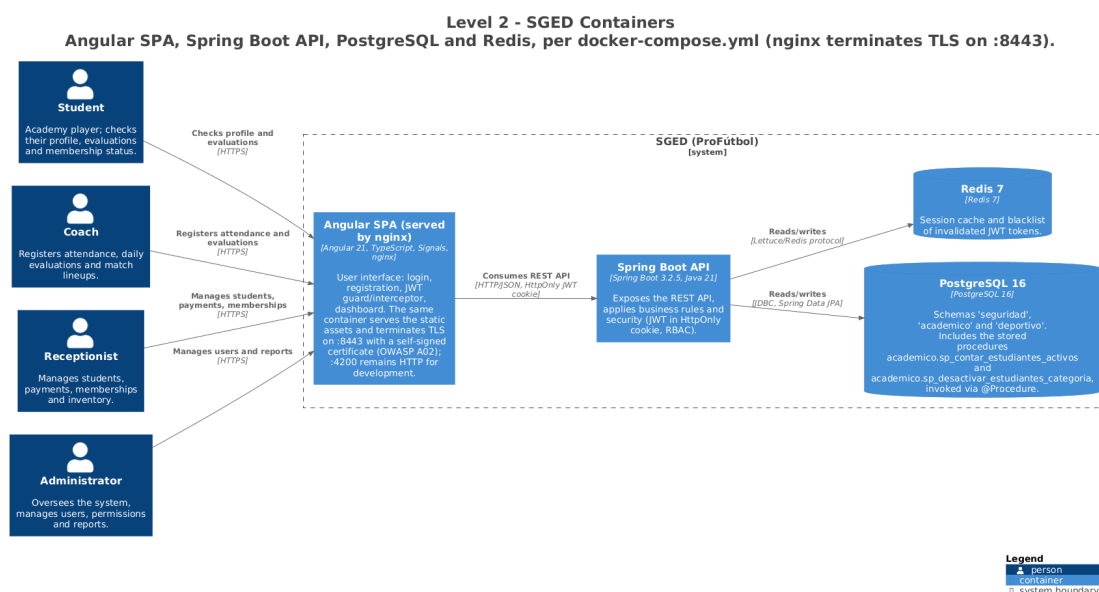


Figura 3: C4 nivel 2 — contenedores de SGED, generado desde `docs/arquitectura/workspace.dsl`.

La documentación de arquitectura en el modelo C4 [8] y las decisiones arquitectónicas (ADR) [32] se mantienen en `docs/arquitectura/` y `docs/adr/`.

El modelo C4 completo (niveles 1 a 3) vive en un único archivo `docs/arquitectura/workspace.dsl` escrito en Structurizr DSL. Los tres PNG del mismo directorio son artefactos derivados: se regeneran con `make diagrams`, que exporta el DSL a PlantUML y lo renderiza, ambos pasos en contenedores. Esto cierra un problema real de esta entrega: antes coexistían dos modelos C4 —el DSL y unos `.puml` sueltos en `docs/diagramas/`— y los `.puml` habían quedado congelados en la arquitectura anterior a la reestructuración de paquetes. Se eliminaron en favor del DSL, por la misma razón por la que se retiró el informe en PDF sin fuente y se reenumeró el ADR duplicado: dos versiones de la misma verdad garantizan que al menos una esté equivocada. `docs/diagramas/` conserva únicamente el MER, que no se modela en C4.

7.3. Vista de componentes (C4 nivel 3)

A diferencia de los niveles 1 (sección 7.1) y 2 (sección 7.2) anteriores, el nivel 3 es el que hace visible la reestructuración descrita en la sección 6: los veinte componentes del contenedor *API Spring Boot* agrupados por dominio. Cada controlador delega en un servicio, cada servicio accede

a datos por un repositorio JPA, y ningún controlador toca un repositorio directamente — la disciplina de capas es verificable en el diagrama, no solo declarada en el texto.

Dos detalles del diagrama de la Figura 4 merecen lectura explícita. Primero, `EstudianteRepository` es el único repositorio que invoca procedimientos almacenados (sección 8): el resto usa Spring Data JPA puro, que es exactamente el reparto que exige la estrategia híbrida. Segundo, `EstudianteService` depende del repositorio del dominio deportivo para resolver la categoría por clave foránea; esa flecha cruzando de `academico` a `deportivo` es la consecuencia directa de haber normalizado la categoría, y es la clase de acoplamiento que conviene tener dibujado antes de que crezca.

7.4. Autenticación: JWT en cookie HttpOnly

El sistema emite el JWT [27] exclusivamente en una cookie `HttpOnly`, `Secure` y `SameSite=Strict` (Listado 1). El token nunca viaja en el cuerpo de la respuesta ni se almacena en `localStorage`, de modo que un ataque de *scripting* entre sitios no puede leerlo.

```

1 ResponseCookie.from("sged_access", token)
2     .httpOnly(true)
3     .secure(cookieSecure)
4     .sameSite("Strict")
5     .path("/")
6     .maxAge(Duration.ofMillis(expiracionMs))
7     .build();

```

Listado 1: Emisión de la cookie de sesión (`AuthController.java`)

Como JWT es *stateless*, cerrar sesión no invalidaría el token por sí solo. Por eso el sistema mantiene una lista de revocación en Redis indexada por el identificador del token (`jti`), con tiempo de vida igual al tiempo que le restaba al token: un token cerrado deja de ser aceptado inmediatamente, aunque su firma siga siendo válida.

8. Estrategia híbrida de acceso a datos

El Bloque A.2 de la guía exige una separación explícita entre lo que resuelve el ORM y lo que debe resolver el motor de base de datos. La regla adoptada es (Tabla 8):

Cuadro 8: Reparto de responsabilidades entre ORM y procedimientos almacenados.

Spring Data JPA (ORM)	Procedimientos almacenados
CRUD elemental, consultas paginadas, búsquedas por identificador.	Agregaciones, actualizaciones masivas con criterio de negocio, operaciones multi-tabla transaccionales.

Nota de esta versión del informe: el sistema creció de dos a **siete** procedimientos almacenados desde la Tercera Entrega, uno por cada categoría funcional que exige el Bloque A.2.1 de la Entrega Final. Todos están versionados en `db/procs/`, creados por migración Flyway [39] (V5, V6, V11, V15) y catalogados en `docs/basedatos/CATALOGO-SP.md`, que es la fuente de verdad que la Tabla 9 resume:

8.1. Un defecto real: FUNCTION frente a PROCEDURE

La migración V5 creó estos objetos como `FUNCTION`. Al invocarlos desde Java con `@Procedure`, Spring Data usa `CallableStatement` con sintaxis `{call ...}`, y PostgreSQL solo acepta `CALL` contra procedimientos reales [36]; el error era explícito: «... *is not a procedure. Hint: To call a function, use SELECT.*»

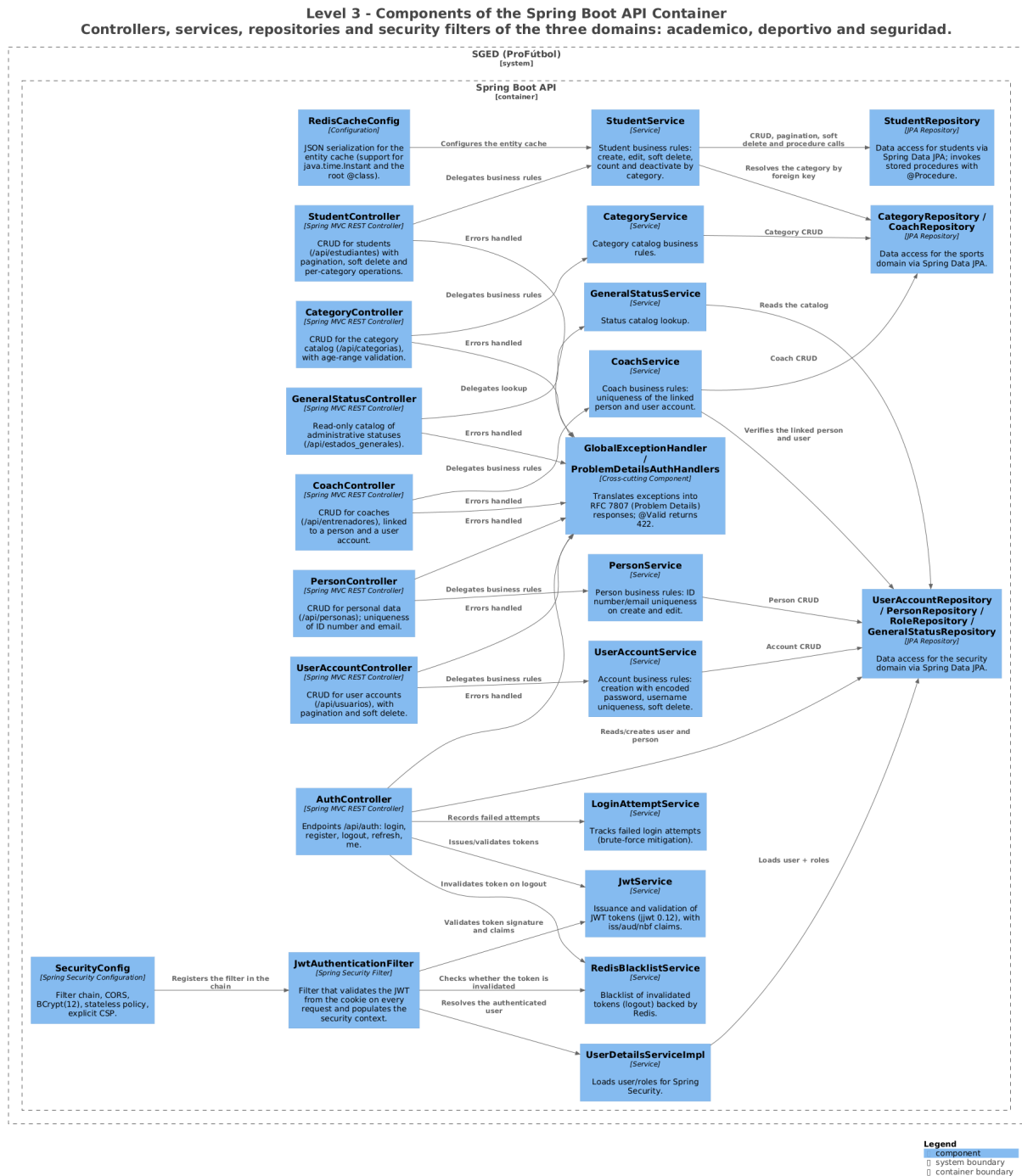


Figura 4: C4 nivel 3 — componentes del contenedor API Spring Boot, generado desde docs/arquitectura/workspace.dsl.

Cuadro 9: Procedimientos almacenados de SGED por categoría funcional (Bloque A.2.1).

Procedimiento	Categoría	Propósito
<code>sp_contar_estudiantes_activos</code>	Código agregado	Conteo de estudiantes activos por categoría
<code>sp_desactivar_estudiantes_activos</code>	Actualización masiva	Baja lógica masiva de una categoría completa
<code>sp_validar_categoria_estudiante</code>	Validación masiva	Un estudiante solo marca asistencia en sesiones de su propia categoría
<code>sp_generar_codigo_estudiante</code>	Generación de código secuencial	Siguiente <code>codigo_estudiante</code> consecutivo del año
<code>sp_reporte_asistencia_estudiante</code>	Reporte	Porcentaje de asistencia en un rango de fechas
<code>sp_contacto_representante_estudiante</code>	Contacto de estudiante multi-tabla	Contacto de emergencia del representante activo
<code>sp_reporte_stock_bajo</code>	Reporte / agregado	Conteo de artículos bajo el mínimo de <i>stock</i>

La migración V6 los convierte en `PROCEDURE` con parámetro de salida. Se documenta porque ilustra la diferencia entre «el procedimiento existe» y «el procedimiento se invoca de verdad»: en la versión anterior el objeto SQL estaba creado pero quedaba huérfano, y la aplicación seguía contando en memoria. La invocación correcta (Listado 2):

```

1 @Procedure(procedureName = "academico.sp_contar_estudiantes_activos")
2 Long contarEstudiantesActivosPorCategoria(@Param("p_categoria") Long idCategoria
);

```

Listado 2: Invocación real desde el repositorio (`academico/estudiante/repository/EstudianteRepository.java`)

Nota sobre esta segunda versión del informe: el procedimiento se movió del esquema `seguridad` a `academico` y su parámetro cambió de `VARCHAR` (nombre de categoría) a `Long` (`id_categoria`) cuando la categoría se normalizó como entidad propia — ver sección 6.

8.2. Ausencia de SQL dinámico

Ninguna capa construye sentencias SQL por concatenación de cadenas. Toda consulta usa parámetros vinculados (JPA) o parámetros nombrados (los procedimientos). El repositorio incluye una auditoría automatizada (`scripts/audit-sql-dynamic.sh`) integrada en `make audit`.

9. Implementación

Este capítulo describe las decisiones de código que no se derivan directamente del diagrama de arquitectura (Capítulo 7) ni de la estrategia de acceso a datos (sección 8, que actúa como la subsección de este capítulo dedicada a esa estrategia, con su propio listado de código). No se transcriben archivos completos: cada listado remite al archivo real del repositorio para el detalle.

9.1. Aislamiento de capas

La disciplina *controlador* → *servicio* → *repositorio* descrita en la sección 7.3 se aplica sin excepción en los 21 controladores del *backend*: ningún controlador inyecta un repositorio directamente, y ningún repositorio contiene lógica de negocio. Los cinco dominios (`seguridad`, `academico`, `deportivo`, `inventario` y el transversal `common`) están separados por paquete Java, no solo por

convención de nombres, de modo que un ciclo de dependencias entre dominios se detecta en tiempo de compilación.

9.2. Manejo global de errores

Todas las respuestas de error del *backend* siguen el formato *Problem Detail* de la RFC 9457 [31], centralizado en un único punto de la capa web (Listado 3):

```

1 @RestControllerAdvice
2 public class GlobalExceptionHandler {
3
4     @ExceptionHandler(ApiException.class)
5     public ProblemDetail handleApiException(ApiException ex) {
6         String tipo = ex.getClass().getSimpleName();
7         ProblemDetail pd = ex.toProblemDetail(tipo, ex.getStatus().
8             getReasonPhrase());
9         pd.setProperty("timestamp", Instant.now().toString());
10        return pd;
11    }
12    // ExceptionHandler adicionales para validacion, autenticacion y
13    // autorizacion
14 }

```

Listado 3: Manejador global de excepciones
(`backend/.../common/exception/GlobalExceptionHandler.java`)

Cada excepción de dominio extiende `ApiException` y declara su propio código HTTP; el manejador no interpreta tipos de excepción específicos del dominio, solo los traduce al formato de respuesta uniforme — la razón por la que agregar un dominio nuevo (representantes, pagos, inventario) no requirió tocar este archivo.

9.3. Autenticación en el *frontend*: cookies, no *localStorage*

El interceptor HTTP de Angular no lee ni adjunta el JWT manualmente — es la cookie `HttpOnly` (sección 7.4) la que viaja automáticamente con cada petición del mismo origen (Listado 4):

```

1 export const authInterceptor: HttpInterceptorFn = (req, next) => {
2     return next(req.clone({ withCredentials: true }));
3 };

```

Listado 4: Interceptor de autenticación (`frontend/src/app/auth/auth.interceptor.ts`)

La brevedad del interceptor es intencional y es la consecuencia directa de la decisión de ADR-002/ADR-008: al no poder leer la cookie desde JavaScript, el *frontend* no tiene lógica de adjuntar ni refrescar el token por su cuenta — delega esa responsabilidad enteramente en el navegador y en el filtro de seguridad del *backend*.

9.4. Caché de consultas de listado

El ADR-005 (`docs/adr/ADR-005-cache.md`) fija Redis como caché de consultas, y el *backend* anota con `@Cacheable` los tres servicios de listado de mayor lectura (`Student`, `Trainer` y `UserAccount`). Durante la preparación de la defensa se detectó que las anotaciones existían pero la caché nunca llegó a estar operativa: faltaba `@EnableCaching`, de modo que cada petición consultaba PostgreSQL aunque Redis estuviera sembrado. El defecto se corrigió en dos archivos: `backend/.../BackendApplication.java` activa el soporte declarativo, y `backend/.../config/RedisCacheConfig.java` configura el serializador con *polymorphic typing* explícito (tipos de datos registrados con `BasicPolymorphicTypeV` sin el cual las respuestas cacheadas comienzan a devolverse como `LinkedHashMap` y rompen el contrato de tipos del *endpoint* —un defecto que las mediciones de la sección 12.1 documentan como resuelto. La caché quedó verificada en caliente: la primera petición (fría, sin clave) tarda

≈270–300 ms; la segunda (misma clave) cae a ≈75–105 ms y genera la clave `estudiantes:0-10` en Redis.

9.5. Configuración por entorno

La configuración de Spring Boot vive en un único `application.yml` con valores parametrizados por variable de entorno (`.env.example` documenta cada una sin valores sensibles); no existen perfiles `application-{dev,prod}.yml` duplicados que puedan divergir entre sí. En `docker-compose.yml` las imágenes de PostgreSQL y Redis están fijadas por *digest sha256* en vez de por etiqueta `latest`, para que `make up` reproduzca el mismo binario en cualquier máquina (Bloque D.2).

10. Requisitos y modelo de datos

10.1. Resumen de requisitos

La especificación completa está en `docs/requisitos/SRS.md`. Se resumen aquí los requisitos funcionales con su estado real y el *endpoint* o esquema que los materializa.

ID	Requisito	Implementación	Estado
RF-01	Registro de usuarios por administrador	POST <code>/api/auth/registro</code>	OK
RF-02	Autenticación con JWT en cookie <code>HttpOnly</code>	POST <code>/api/auth/login</code>	OK
RF-03	Cierre de sesión con revocación efectiva por <code>jti</code>	POST <code>/api/auth/logout</code>	OK
RF-04	Renovación de sesión sin re-credenciales	POST <code>/api/auth/refresh</code>	OK
RF-05	Consulta de la sesión activa	GET <code>/api/auth/me</code>	OK
RF-06	Bloqueo tras 5 intentos fallidos en 15 minutos	<code>LoginAttemptService</code>	OK
RF-07	Comprobación de disponibilidad	GET <code>/api/auth/ping</code>	OK
RF-08	Listado paginado de estudiantes	GET <code>/api/estudiantes</code>	OK
RF-09	Consulta por identificador con 404 tipificado	GET <code>/api/estudiantes/{id}</code>	OK
RF-10	Registro transaccional de estudiante	POST <code>/api/estudiantes</code>	OK
RF-11	Validación del formato de categoría (SUB-NN)	<code>EstudianteRequest</code>	OK
RF-12	Actualización de estudiante	PUT <code>/api/estudiantes/{id}</code>	OK
RF-13	Baja lógica preservando el historial	DELETE <code>/api/estudiantes/{id}</code>	OK
RF-14	Conteo de activos por categoría (en el motor)	<code>sp_contar_estudiantes_activos</code>	OK
RF-15	Desactivación masiva por categoría (en el motor)	<code>sp_desactivar_estudiantes_por_categoria</code>	OK
RF-16	Gestión de entrenadores	<code>deportivo.entrenadores</code>	Modelado
RF-17	Horarios recurrentes de entrenamiento	<code>horarios_entrenamiento</code>	Modelado
RF-18	Sesiones de entrenamiento con ciclo de estados	<code>sesiones_entrenamiento</code>	Modelado
RF-19	Asistencia por RFID o manual	<code>deportivo.asistencias</code>	Modelado
RF-20	Evaluación diaria por criterios	<code>detalle_evaluacion</code>	Modelado
RF-21	Promedios de evaluación	<code>v_promedio_evaluacion</code>	Modelado
RF-22	Notificación a representantes	—	Planificado

Los requisitos no funcionales (RNF-01 a RNF-13) se verifican con las mediciones de la sección siguiente. La matriz `docs/trazabilidad/matriz.csv` enlaza cada uno con su prueba y su evidencia.

10.2. Catálogo de la API REST

La superficie expuesta es de siete recursos y treinta *endpoints*. Se lista completa porque es la forma más directa de contrastar lo que el sistema realmente ofrece contra lo que los requisitos declaran, y porque la columna de autorización es la que concentró el defecto de la sección 12.2.1. A abrevia ADMINISTRADOR; E, ENTRENADOR; U, el rol USER.

Método y ruta	Operación	Roles	RF
<i>Autenticación</i> — <code>/api/auth</code>			
POST <code>/login</code>	Inicia sesión, emite cookies	Público	RF-02
POST <code>/registro</code>	Alta de cuenta	A	RF-01
POST <code>/logout</code>	Revoca el token en Redis	Autenticado	RF-03
POST <code>/refresh</code>	Renueva el <i>access token</i>	Autenticado	RF-04
GET <code>/me</code>	Datos de la sesión actual	Autenticado	RF-05
GET <code>/ping</code>	Sonda de disponibilidad	Público	—
<i>Estudiantes</i> — <code>/api/estudiantes</code>			
GET <code>/</code>	Listado paginado y ordenable	A E U	RF-08
GET <code>/ {id}</code>	Consulta individual	A E U	RF-09
POST <code>/</code>	Alta	A	RF-10
PUT <code>/ {id}</code>	Edición	A	RF-12
DELETE <code>/ {id}</code>	Baja lógica	A	RF-13
GET <code>/conteo/categoria/ {id}</code>	Conteo por procedimiento	A E	RF-14
POST <code>/operaciones/ desactivar-categoria</code>	Desactivación masiva por procedimiento	A	RF-15
<i>Categorías</i> — <code>/api/categorias</code>			
GET <code>/</code>	Listado paginado	A E U	RF-23
GET <code>/activas</code>	Catálogo para formularios	A E U	RF-23
GET <code>/ {id}</code>	Consulta individual	A E U	RF-23
POST <code>/</code>	Alta con rango de edad	A	RF-23
PUT <code>/ {id}</code>	Edición	A	RF-23
DELETE <code>/ {id}</code>	Baja lógica	A	RF-23
<i>Entrenadores</i> — <code>/api/entrenadores</code>			
GET <code>/</code>	Listado paginado	A E	RF-24
GET <code>/ {id}</code>	Consulta individual	A E	RF-24
POST <code>/</code>	Alta con persona y cuenta	A	RF-24
PUT <code>/ {id}</code>	Edición	A	RF-24
DELETE <code>/ {id}</code>	Baja lógica	A	RF-24
<i>Personas</i> — <code>/api/personas</code> (datos identificativos de menores)			
GET <code>/</code>	Listado paginado	A	RF-25
GET <code>/ {id}</code>	Consulta individual	A	RF-25

Método y ruta	Operación	Roles	RF
GET /cedula/{cedula}	Búsqueda por cédula	A	RF-25
POST /	Alta	A	RF-25
PUT /{id}	Edición	A	RF-25
DELETE /{id}	Baja lógica	A	RF-25
<i>Usuarios y catálogos</i>			
/api/usuarios (5 verbos)	CRUD de cuentas	A	RF-26
GET /api/estados_generales	Catálogo de estados	A E U	RF-26

Dos observaciones que la tabla hace evidentes. La primera es que `/api/personas` es el recurso con la superficie más sensible del sistema —seis *endpoints* sobre datos identificativos de menores, incluido uno de búsqueda directa por cédula— y es exactamente el que estuvo sin control de acceso. La segunda es que el dominio deportivo expuesto se limita hoy a categorías y entrenadores: horarios, sesiones, asistencias y evaluaciones tienen esquema en la base de datos pero ninguna ruta REST, y son el objetivo declarado de la Entrega Final.

10.3. Modelo de datos

El esquema se divide en dos espacios de nombres con responsabilidades separadas (Tabla 12):

Cuadro 12: Esquemas del modelo de datos de SGED.

Esquema	Tablas	Contenido
seguridad	6	Personas, usuarios, roles, relación usuario-rol, estados y estudiantes.
deportivo	9	Posiciones, entrenadores, horarios, sesiones, asistencias, criterios, evaluaciones, detalle y observaciones.

Decisiones de modelado relevantes, todas materializadas como restricciones en el motor y no solo como validación en la aplicación:

- **Baja lógica en lugar de borrado.** Las asistencias y evaluaciones referencian al estudiante por clave foránea; borrarlo físicamente destruiría su historial deportivo.
- **Unicidad parcial del código RFID.** `UNIQUE ...WHERE rfid_codigo IS NOT NULL` permite muchos estudiantes sin credencial, pero impide que dos compartan la misma.
- **Una asistencia por sesión y estudiante.** `UNIQUE (id_sesion, id_estudiante)`.
- **Posición jugada por evaluación.** `detalle_evaluacion` guarda la posición de *ese* día, distinta de la habitual del estudiante, para que el histórico quede correctamente etiquetado cuando alguien juega fuera de su posición.
- **Coherencia temporal.** `CHECK (hora_fin > hora_inicio)` y `CHECK (dia_semana BETWEEN 1 AND 7)` en los horarios.

El diccionario completo, con tipos, restricciones y disparadores, está en `docs/basedatos/DATA-DICTIONARY.md`. El esquema no lo genera el ORM: `ddl-auto` está fijado en `validate`, de modo que la aplicación falla al arrancar si el esquema real y las entidades divergen.

11. Materiales y métodos

Todas las mediciones de este informe se generaron ejecutando el sistema real levantado con `make up`, no en simulación, y sus artefactos crudos están versionados sin edición manual. Este capítulo declara el enfoque metodológico (sección 11.1), los instrumentos (sección 11.2) y el procedimiento antes de presentar los resultados (Capítulo 12), para que un tercero pueda repetirlos y obtener cifras comparables.

11.1. Enfoque metodológico

El trabajo sigue *Design Science Research* (DSR) en la formulación de Peffers *et al.* [34], articulada con los criterios de rigor y relevancia de Hevner *et al.* [21] —un marco que la revisión de Engström *et al.* confirma como infrecuente en la investigación de ingeniería de software pese a encajar naturalmente con ella, precisamente porque buena parte de esa investigación ya diseña soluciones a problemas prácticos sin nombrar explícitamente el paradigma [17]—, en sus seis actividades:

1. **Identificación del problema y motivación** — sección 1.1: gestión administrativa y deportiva fragmentada, sin trazabilidad ni control de acceso sobre datos de menores.
2. **Definición de objetivos de una solución** — sección 1.3: objetivo general y cinco objetivos específicos trazados a las cuatro preguntas de investigación.
3. **Diseño y desarrollo** — Capítulo 7 y sección 8: arquitectura en C4, estrategia híbrida de acceso a datos, ocho ADR.
4. **Demostración** — el sistema desplegado y operable (`make up`), con datos sembrados y los cinco roles de usuario reales.
5. **Evaluación** — Capítulo 12: contraste empírico contra los umbrales de rendimiento, seguridad, cobertura, calidad web y usabilidad declarados en este capítulo.
6. **Comunicación** — este documento, el repositorio público y la defensa oral del PFC.

El ciclo no fue lineal: la Tercera Entrega ya había ejecutado una vuelta completa de demostración y evaluación, y esta entrega repite las actividades 3 a 5 sobre el sistema ampliado, en la lógica iterativa que el propio modelo de Peffers admite entre sus actividades.

11.2. Instrumentos de medición

La Tabla 13 enumera el instrumento y la configuración exacta usados en cada bloque de evaluación.

11.3. Enfoque de métricas: un GQM completo

Las métricas del sistema se alinean con el marco *Goal-Question-Metric* de Basili *et al.* [5]. Se presenta un GQM completo (Tabla 14), correspondiente a RQ1 (sección 1.2):

11.4. Población, muestreo y participantes

Para la componente de usabilidad, la población objetivo son los cinco perfiles de usuario reales del sistema (administrador, entrenador, recepcionista, estudiante, representante). El muestreo fue por conveniencia —participantes disponibles del entorno cercano al equipo—, siguiendo la clasificación de técnicas de muestreo de Baltes y Ralph [3], quienes documentan que es la técnica más frecuente y a la vez la que exige declarar sus límites en vez de omitirlos. **Limitación**

Cuadro 13: Instrumentos de medición usados en la evaluación empírica.

Bloque	Instrumento	Configuración
Rendimiento	k6	k6/listado-estudiantes.js, 50 VU, 30s, umbrales declarados en el propio <i>script</i>
Seguridad	curl + análisis estático	scripts/audit-owasp.sh, scripts/audit-sql-dynamic.sh
Cobertura	JaCoCo	Umbral 70 % líneas y <i>branches</i> , configurado en <code>backend/pom.xml</code>
Calidad web	Lighthouse	lighthousec.js
Usabilidad	SUS (Brooke)	Cuestionario de 10 ítems sin modificación [7], escala adjetival de Bangor <i>et al.</i> [4]

Cuadro 14: Ejemplo de Goal-Question-Metric para RQ1.

Meta (Goal)	Caracterizar el rendimiento del <i>endpoint</i> de listado protegido bajo carga concurrente, desde el punto de vista del equipo de desarrollo, en el contexto del ambiente de pruebas de la Entrega Final.
Pregunta 1	¿Cuál es la latencia p95 del <i>endpoint</i> bajo 50 usuarios virtuales concurrentes?
Métrica 1	p95 de tiempo de respuesta (ms), agregado sobre corridas independientes, con intervalo de confianza al 95 %.
Pregunta 2	¿El sistema devuelve errores de servidor bajo esa carga?
Métrica 2	Tasa de respuestas HTTP ≥ 500 sobre el total de peticiones de la corrida.

declarada: $N = 15$ es una muestra pequeña y no probabilística; los resultados por perfil (tres o cuatro participantes por rol) se interpretan como indicativos de un patrón, no como una estimación poblacional generalizable —la razón por la que sección 12.7 reporta el desglose por perfil en vez de solo la media agregada.

Recolección en dos fases. Los diez primeros cuestionarios se aplicaron el 30 de julio de 2026 y los cinco restantes el 18 de agosto, para alcanzar el mínimo de quince que exige la Entrega Final. Se declara explícitamente porque afecta a cómo debe leerse el dato: la segunda tanda se recogió sobre una versión posterior del sistema, de modo que los quince cuestionarios no evalúan exactamente el mismo artefacto. El efecto sobre el agregado fue pequeño y se documenta en `docs/mediciones/sus/INTERPRETACION.md`: la media se movió 1,08 puntos (68,25 a 69,33) mientras el intervalo de confianza se estrechó en torno a un tercio, lo que sugiere que la ampliación precisó la estimación sin desplazarla.

Numeración. Los identificadores van de ENC-01 a ENC-10 y de ENC-13 a ENC-17: no existen ENC-11 ni ENC-12. El salto se conserva tal como llegaron los datos en vez de reenumerar, para no presentar como serie continua una que no lo es.

11.5. Análisis estadístico declarado a priori

Antes de ejecutar las mediciones se fijaron los estadísticos a reportar: media, mediana, desviación típica e intervalo de confianza al 95 % para todas las métricas cuantitativas; percentiles p50, p90 y p95 para rendimiento. Se prefieren intervalos de confianza sobre valores p como forma primaria de reporte, siguiendo la recomendación del Bloque C de la guía. Las cinco corridas independientes de k6 por escenario que exige el Bloque A.1 están archivadas en `docs/mediciones/perf/`; el escenario de caché cálida mide el *endpoint* con la página de consulta ya resuelta por Redis, y el de caché fría barre páginas distintas con la caché vacía para forzar *misses* reales contra el volumen de datos.

La comparación entre escenarios es un contraste inferencial no paramétrico con tamaño de efecto y corrección por comparaciones múltiples, conforme al Bloque C de la guía: test de Mann-Whitney (Wilcoxon) bilateral, estadístico A12 de Vargha y Delaney [44] y delta de Cliff [13] para procesos ordinales, y corrección de Holm-Bonferroni [22] sobre las comparaciones emparejadas corrida-a-corrida (sección 12.1). En el desglose de usabilidad por perfil no hay contraste entre grupos —3 o 4 participantes por perfil no lo soportan—, por lo que la corrección múltiple se aplica exclusivamente al rendimiento (sección 11.5).

11.6. Entorno de ejecución

La Tabla 15 fija el entorno exacto en el que se generaron todas las mediciones de este capítulo.

Cuadro 15: Entorno de ejecución de las mediciones.

Elemento	Valor
Anfitrión	Linux (x86_64), Docker Engine en contenedores
Contenedores	postgres y redis fijados por digest sha256 en docker-compose.yml; backend y frontend contruidos desde el repositorio
Backend	Spring Boot 3.2.5 sobre Temurin JDK 21
Estado inicial	Base de datos creada desde cero (docker compose down -v) y sembrada con db/seed.sql y el volumen sintético db/carga-volumen.sql (2401 estudiantes activos)
Red	Todas las peticiones contra localhost; sin latencia de red externa

El estado inicial no es un detalle menor. Los *scripts* de inicialización de PostgreSQL solo se ejecutan cuando el directorio de datos está vacío: al reutilizar un volumen anterior, el esquema queda congelado en una versión previa y el *backend* no arranca. Toda medición reportada aquí parte de volumen limpio.

11.7. Procedimiento por bloque

La Tabla 16 detalla, para cada bloque del enfoque GQM (sección 11.3), la herramienta y el procedimiento de control de variabilidad aplicados.

11.8. Determinismo, semillas y trazabilidad

Los archivos crudos (.json de k6 y Lighthouse, .txt de las respuestas HTTP, .csv del SUS y de JaCoCo) se conservan tal como los produjo cada herramienta; los reportes en Markdown se generan a partir de ellos mediante los *scripts* de *scripts/*, nunca a mano. Cada archivo de evidencia lleva en su cabecera la fecha en UTC, el *commit* corto y la versión de la herramienta que lo generó, de modo que una cifra del informe pueda rastrearse hasta la ejecución concreta que la produjo.

`scripts/perf-analysis.py` fija semilla aleatoria 42 para cualquier remuestreo estadístico sobre los datos crudos de k6, declarada en la cabecera del propio *script* y en `docs/mediciones/perf/REPORT.md`. Los datos de SUS y de JaCoCo no requieren semilla porque no involucran muestreo aleatorio: son transcripción directa (SUS) o instrumentación determinista de cobertura (JaCoCo). Las versiones exactas de cada herramienta (k6, Lighthouse, JDK, Docker) se declaran por corrida en `docs/entorno/versions.txt` y en la cabecera de cada archivo crudo.

Cuadro 16: Procedimiento de medición por bloque de evaluación.

Bloque	Herramienta	Procedimiento y control de variabilidad
C.1 Rendimiento	k6	<code>make bench</code> : 5 corridas independientes de 50 usuarios virtuales durante 30 s por escenario (caché cálida y caché fría). Se reportan media, p90, p95 e intervalo de confianza al 95 % sobre las corridas, no una sola.
C.2/C.4 Seguridad	curl	<code>make audit</code> : peticiones reales contra el sistema en ejecución, guardando la respuesta HTTP íntegra con cabeceras. Cada control se contrasta en sus dos direcciones: lo que debe rechazarse y lo que debe seguir permitido.
C.3 Usabilidad	SUS en papel	Cuestionario de Brooke de 10 ítems aplicado a 15 participantes externos en dos tandas; transcripción a CSV y cálculo con <code>scripts/sus-analysis.py</code> , cuya aritmética se validó contra los cuatro casos de referencia de la literatura (0, 50, 75 y 100 puntos).
Cobertura	JaCoCo	<code>make test</code> , que ejecuta <code>clean test</code> para que la instrumentación no alcance clases de compilaciones anteriores.
C.5 Accesibilidad	Lighthouse	3 corridas mediante <code>lighthouse.js</code> ; se reportan los cuatro puntajes agregados y las métricas Web Vitals.

12. Evidencia empírica

12.1. Rendimiento (Bloque C.1)

Diez corridas independientes de k6, cinco por escenario, con 50 usuarios virtuales durante 30 segundos contra el *endpoint* protegido `GET /api/estudiantes` — cumple el mínimo de cinco corridas por escenario que exige la guía. El escenario de caché cálida consulta siempre la misma página (`page=0`) ya resuelta por Redis; el de caché fría barre las 241 páginas de los 2401 estudiantes activos sembrados con `db/carga-volumen.sql`, con la caché vaciada (`FLUSHALL`) antes de cada corrida, de modo que cada primera visita a una página cae a PostgreSQL (un *miss* real). Las corridas se regeneraron el 2026-09-07 contra el sistema local completo sobre el *commit* defendido, tras corregir el defecto de `.env` descrito en la sección 1.7 y activar la caché de Redis que la sección 9.4 documenta.

La decisión de repetir la medición y reportar un intervalo de confianza, en vez de publicar el resultado de una sola corrida, sigue el argumento metodológico de Georges *et al.*: una única ejecución de un sistema sobre JVM no caracteriza su rendimiento, porque la variación entre corridas puede ser del mismo orden que la diferencia que se pretende reportar, y sin estadística de intervalos no hay forma de distinguir una de otra [20]. Los resultados: no

Cuadro 17: Caché cálida: resultados de las corridas de rendimiento (k6, `docs/mediciones/perf/k6-run*.json`).

Corrida	Media (ms)	Mediana	p90	p95	p99	Errores	RPS
1 (calentamiento)	56,66	45,56	104,87	137,08	204,28	0,00 %	273,55
2	15,63	12,78	26,34	32,50	49,76	0,00 %	370,96
3	10,33	9,22	14,56	17,40	25,44	0,00 %	389,64
4	9,33	8,75	12,07	13,99	20,05	0,00 %	393,48
5	9,83	9,01	13,20	15,88	22,15	0,00 %	390,28
Agregado 2–5	11,28	—	—	19,94	29,35	—	386,09

Cuadro 18: Caché fría: resultados de las corridas de rendimiento (k6, `docs/mediciones/perf/k6-frío*.json`).

Corrida	Media (ms)	Mediana	p90	p95	p99	Errores	RPS
1	20,39	18,54	36,02	41,72	52,53	0,00 %	356,15
2	18,81	15,92	34,14	39,77	50,89	0,00 %	361,16
3	16,49	11,45	32,06	36,96	48,25	0,00 %	367,80
4	17,22	13,06	32,58	37,43	47,51	0,00 %	366,12
5	18,45	16,10	33,28	38,64	50,22	0,00 %	362,33
Agregado 2–5	17,74	—	—	38,20	49,22	—	364,35

Con intervalo de confianza al 95 % —siguiendo el análisis declarado a priori (sección 11.5)— la caché cálida (Tabla 17; corridas 2–5, tras descartar la primera de calentamiento de la JVM) da una media de $11,28 \pm 4,66$ ms (DT 2,93), p95 $19,94 \pm 13,51$ ms y p99 $29,35 \pm 21,93$ ms; la fría (Tabla 18), media $17,74 \pm 1,72$ ms (DT 1,08), p95 $38,20 \pm 2,01$ ms y p99 $49,22 \pm 2,54$ ms. Ninguna corrida registró peticiones fallidas — la tasa de error es 0,00 % en las diez, frente al 0,01 % de la corrida 4 de la Tercera Entrega.

La primera corrida cálida (56 ms) es la de calentamiento: compila la JVM y puebla las cachés internas bajo carga, y su desviación frente a las cuatro siguientes (media ≈ 9 –15 ms) sería un *outlier* si se agregara; se descarta del agregado y del contraste siguiendo la práctica recomendada

por Georges *et al.* [20], y se reporta en la tabla en vez de ocultarse.

Contraste inferencial (caché cálida vs. caché fría). Test de Mann-Whitney bilateral (cálida como primera muestra) sobre las corridas 2–5, con el estadístico A12 de Vargha y Delaney [44] y la delta de Cliff [13], y corrección por comparaciones múltiples de Holm-Bonferroni [22] sobre las cuatro corridas (Tabla 19; reproducido íntegramente por `scripts/perf-analysis.py`, que imprime el estadístico U, el valor p en notación científica y el tamaño de efecto):

Cuadro 19: Contraste inferencial entre caché cálida y caché fría (Mann-Whitney).

Comparación	U	z	p	A12	δ Cliff
Corrida 2	121 675 593	17,4	$6,93 \times 10^{-68}$	0,441	−0,117
Corrida 3	155 061 192	49,9	$2,25 \times 10^{-543}$	0,335	−0,330
Corrida 4	174 960 074	75,1	$1,42 \times 10^{-1227}$	0,252	−0,496
Corrida 5	171 094 482	73,3	$3,90 \times 10^{-1168}$	0,257	−0,486
Pool global	2 479 872 504	106,9	$1,75 \times 10^{-2483}$	0,323	−0,355

Las cuatro comparaciones por corrida y el pool global dan $p < \alpha$ y sobreviven a la corrección de Holm-Bonferroni (todas significativas con $\alpha = 0,05$). $A12 < 0,5$ indica que la caché cálida —primera muestra del contraste— tiende a tiempos menores que la fría; la delta de Cliff negativa confirma la dominancia en el mismo sentido. La mejora es $\approx 6,5$ ms en promedio (media 11,28 frente a 17,74) y de ≈ 18 ms en p95 (19,94 frente a 38,20), significativa con $n \approx 15\,000$ peticiones por corrida.

El umbral exigido es $p95 < 200$ ms con caché caliente [25]. El resultado lo cumple con un margen de un orden de magnitud (19,94 ms frente a 200 ms). **OK**

12.2. Seguridad: OWASP Top 10 (Bloque C.4)

Seis controles verificados con peticiones reales contra el sistema en ejecución [33] (Tabla 20), complementados por análisis estático y escaneo automático (sección 12.4):

Cuadro 20: Resultado de los seis controles OWASP verificados.

Control	Resultado	Evidencia observada
A01 Control de acceso	OK	403 en las 7 operaciones administrativas y 200 en las lecturas permitidas — tras corregir un defecto real, §12.2.1
A02 Fallos criptográficos	OK	TLS 1.3 en :8443; BCrypt coste 12
A03 Inyección	OK	422 ante entrada malformada; sin SQL dinámico
A05 Configuración	OK	CSP, HSTS, nosniff, X-Frame-Options: DENY
A07 Fallos de autenticación	OK	Bloqueo exacto en el sexto intento (429)
A09 Registro y monitoreo	OK	AUTH_LOGIN_OK/FAIL con IP, marca de tiempo y sujeto

Los errores se devuelven como `ProblemDetail` [31], sin trazas de pila ni detalles internos. Ejemplo real capturado para A07 (Listado 5):

```

1 HTTP/1.1 429
2 Content-Type: application/problem+json
3 {
4   "type": "https://sged.uteq.edu.ec/errores/DemasiadasPeticiones",
5   "title": "Too Many Requests",

```

```

6  "status": 429,
7  "detail": "Demasiados intentos fallidos. Intente mas tarde."
8  }

```

Listado 5: Respuesta al sexto intento de autenticación

12.2.1. Una auditoría que no auditaba lo que decía auditar

El control A01 merece una sección propia, porque el resultado 403 de la tabla anterior es correcto *ahora*, y no lo era cuando este informe lo dio por bueno la primera vez. La corrección de la propia herramienta de medición es parte del resultado.

Al ejecutar la auditoría contra el sistema en vivo aparecieron tres defectos encadenados en el guion `scripts/audit-owasp.sh`:

1. **El guion se bloqueaba a sí mismo.** Su control A07 agota a propósito los intentos de autenticación para comprobar el 429. Ese contador se lleva *por dirección IP*, no por usuario: seis fallos dejan bloqueada durante quince minutos a cualquier cuenta que venga de ese equipo, incluida la del administrador que A01 necesita para preparar su usuario de prueba. En una segunda corrida, A01 respondía 401 en todas las peticiones. Un 401 se parece mucho a un resultado satisfactorio —el acceso fue rechazado— pero prueba algo mucho más débil: que hace falta estar autenticado. No dice nada sobre si se respetan los roles.
2. **Invocaba una forma de la API que ya no existía.** Llamaba a `/api/estudiantes/operaciones/desactivar-cat` con `?categoria=SUB-12`, la forma anterior a la normalización de la categoría. La petición moría al deserializar el cuerpo, antes de alcanzar el control de acceso.
3. **Sus cuerpos de prueba eran inválidos.** `@Valid` se evalúa al resolver el argumento del método, es decir *antes* que `@PreAuthorize`. Un payload malformado devuelve 422 y nunca llega a ejercitar la autorización que se pretendía evidenciar.

Corregidos los tres, la auditoría comprueba recurso por recurso, y es entonces cuando el control A01 encontró un defecto real de seguridad, que se documenta en la sección siguiente.

12.2.2. El defecto que A01 destapó: control de acceso ausente

Los cinco recursos que introdujo la reestructuración se crearon **sin ninguna anotación** `@PreAuthorize`, salvo `UsuarioController`. Como `SecurityConfig` cierra su cadena con `anyRequest().authenticated()`, la única barrera efectiva era poseer una sesión válida. En consecuencia, una cuenta con el rol más básico del sistema podía listar todas las personas registradas, buscar una por número de cédula, y crear, editar o eliminar registros.

La gravedad no es teórica: `seguridad.personas` es la tabla que concentra los datos identificativos de los estudiantes —menores de edad— que `docs/etica/ETHICS.md` se compromete a proteger. Es además una regresión de una observación ya emitida y ya dada por aplicada: OBS-09 de la Entrega 1B señalaba precisamente la ausencia de `@PreAuthorize` (sección 5). El código nuevo reintrodujo el defecto sin que nada lo detectara, porque las pruebas de esos controladores usan `MockMvcBuilders.standaloneSetup()`, que no levanta la cadena de filtros de Spring Security: verifican el mapeo HTTP, no la autorización.

La corrección no aplica una regla uniforme, sino una por recurso según el dato que maneja (Tabla 21):

La evidencia regenerada (`docs/mediciones/sec/a01-acceso-roto.txt`) comprueba las dos direcciones, que es lo que hace verificable la corrección: 403 en las siete operaciones administrativas —incluida la búsqueda por cédula— y 200 en las dos lecturas que un rol `USER` sí debe poder hacer. Sin esa segunda mitad, la evidencia sería compatible con haber roto la aplicación en vez de haberla asegurado.

Cuadro 21: Criterio de acceso aplicado por recurso tras corregir el hallazgo H-08.

Recurso	Acceso	Criterio
Persona	Solo ADMINISTRADOR	Concentra la identificación de roles; ningún otro rol necesita otros roles; Ya lo estaba, a nivel de clase.
Usuario	Solo ADMINISTRADOR	El formulario de estudiante crea o modifica las categorías activas; escribir al catálogo del que dependen por categoría.
Categoría	Lectura: 3 roles Escritura: ADMINISTRADOR	El alta y la baja crean o retiran el vínculo con una cuenta.
Entrenador	Lectura: ADMIN, ENTRENADOR Escritura: ADMINISTRADOR	Catálogo sin datos personales.
EstadoGeneral	Lectura: 3 roles	

12.3. Dos defectos de funcionamiento encontrados al ejecutar el sistema

Ninguno de los dos aparece si el sistema no se levanta y se usa. Se documentan porque ilustran una limitación concreta de la estrategia de pruebas de este proyecto, no solo porque estaban.

El registro de usuarios estaba completamente roto (RF-01). `POST /api/auth/registro` devolvía 500 ante *cualquier* petición. La reestructuración convirtió `cedula`, `correo` y `fecha_nacimiento` en columnas `NOT NULL` de `seguridad.personas`, pero `RegisterRequest` nunca las pidió; el alta violaba la restricción de integridad en cada intento. Tampoco se asignaba `id_estado_general`, igualmente obligatorio.

Lo relevante para la evaluación es *por qué no lo detectaron las pruebas*. `AuthServiceTest` cubre el registro con dos casos y ambos pasaban: los repositorios están mockeados, de modo que `personaRepository.save()` devuelve el objeto que se le pasa sin consultar ninguna base de datos. La restricción `NOT NULL` la aplica PostgreSQL, y en una prueba con dobles PostgreSQL no participa. Es el límite natural de la prueba unitaria con mocks: verifica la lógica que escribimos, no el contrato con el motor de base de datos. La prueba que se agregó no finge resolver eso —haría falta una prueba de integración con base real—, sino que cubre el flanco que sí es verificable sin ella: que una petición sin esos campos no supere la validación y por tanto nunca llegue a la base.

Un cuerpo mal formado se reportaba como fallo del servidor. `HttpMessageNotReadableException` no tenía manejador en `GlobalExceptionHandler` y caía en el genérico, devolviendo 500 «Error interno del servidor» ante un error que es del cliente. Además de ser semánticamente incorrecto según RFC 9457 [31], ensuciaba el registro con trazas de pila que no corresponden a ningún defecto del sistema —ruido que compite con las anomalías reales en A09. Ahora responde 400.

12.4. Análisis estático y escaneo automático (Bloque A.1/A.2.3)

Dos piezas de evidencia adicionales, archivadas en `docs/mediciones/sec/`, complementan la verificación manual de los seis controles anteriores: el análisis estático se generó el 2026-08-14 (commit `8a078e7`) y el escaneo ZAP se regeneró el 2026-09-02 (commit `d293731`) tras apagar la interfaz de Swagger.

Análisis estático de inyección SQL. SpotBugs 4.8.6.4 con el complemento `find-secbugs 1.13.0`, restringido por un filtro de inclusión (`backend/spotbugs-security-include.xml`) a los detectores de inyección SQL —incluida la regla `SQL_NONCONSTANT_STRING_PASSED_TO_EXECUTE` que exige explícitamente el Bloque A.2.3— sobre los 148 archivos fuente del *backend*: **cero hallazgos**. Consistente con la auditoría manual de `scripts/audit-sql-dynamic.sh` (sección 8), pero esta vez con una herramienta de análisis de bytecode independiente, no un *grep* de patrones de texto. El comando ya forma parte del *pipeline* de CI (`.github/workflows/ci.yml`), en la línea de lo que Rostami Mazrae *et al.* documentan sobre cómo los equipos de desarrollo adoptan, combinan

y migran entre herramientas de CI/CD en la práctica [40].

Escaneo OWASP ZAP baseline. Modo pasivo contra `/api/docs` y `/`: **0 hallazgos, de cualquier severidad** (corrida del 2026-09-02, commit `d293731`). No siempre fue así, y vale contarlos: una corrida anterior (2026-08-14, commit `8a078e7`) sí encontró una alerta de severidad **alta** —*Vulnerable JS Library*, sobre `DOMPurify 3.0.6` dentro de `swagger-ui-bundle.js`, una dependencia de terceros empaquetada por Springdoc/Swagger UI, no código propio—, alcanzable porque la interfaz interactiva de Swagger quedaba expuesta sin autenticación. En vez de esperar una actualización de ese paquete de terceros, se apagó su exposición pública: `springdoc.swagger-ui.enabled` ahora se controla con la variable `SPRINGDOC_ENABLED` —activa en desarrollo local, apagada en el ambiente público declarado en `render.yaml`—, y el escaneo se repitió contra esa configuración. Reporte completo, de las dos corridas, en `docs/mediciones/sec/zap/`.

Escaneo OWASP ZAP autenticado con *active scan* (2026-09-11, commit `60cb908`). El baseline anterior era pasivo y sin sesión: solo escuchaba tráfico, sin atacar y sin credenciales. Esta segunda corrida, contra el entorno local (nunca el despliegue público —un *active scan* manda cargas de ataque reales y no debe correr contra datos de producción—), cierra esa brecha: un *login* real (`POST /api/auth/login` contra la cuenta semilla) captura la cookie de sesión y el complemento *replacer* la inyecta en cada petición del plan de automatización (`docs/mediciones/sec/zap/zap-authenticated.yaml`); las **135 rutas** se enumeran desde el documento OpenAPI real (`/api/docs.json`) en vez de un rastreador HTML, apropiado para una API sin enlaces que seguir; el *active scan* (política por defecto, 12 minutos de presupuesto) ataca esas rutas autenticadas.

El escaneo encontró algo que el modo pasivo nunca hubiera visto: **GET** a rutas que solo aceptan **POST** (`/api/auth/login`, `/forgot`, `/confirmar-correo`) devolvía **500** en vez de **405**, porque el manejador global de excepciones no tenía un caso específico para un método HTTP no soportado y la petición caía en el mismo manejador genérico que los errores de verdad. El cuerpo de la respuesta ya era el objeto de error genérico de siempre, sin traza de pila ni ruta de archivo —no había fuga real de información, aunque dos reglas de ZAP lo señalaron por coincidencia textual de la cadena `Internal Server Error`—, pero el código de estado sí era incorrecto. Corregido y verificado con una segunda corrida: **cero hallazgos de severidad alta o media atribuibles a código propio**. Quedan dos advertencias de severidad media aceptadas por diseño, no defectos: el escaneo contra el contenedor del *backend* sin pasar por la capa de cifrado en tránsito (que es una capa aparte, verificada en la sección 12.2.1), y la exposición pública de `/actuator/health` —exigida explícitamente por la observación del docente sobre el despliegue público, que pide comprobar que ese punto de salud responde con el estado de la base de datos y de la caché—. Reportes en `docs/mediciones/sec/zap/zap-authenticated-report.{html,json,xml}`.

Calidad de código: SonarQube. Complementario a SpotBugs —que audita específicamente inyección SQL con un filtro restringido—, se ejecutó SonarQube Community Edition (instancia efímera en Docker) contra las 9 355 líneas de `backend/src/`, con el reporte de cobertura JaCoCo importado para correlacionar hallazgos con líneas cubiertas por prueba. Resultado: **0 bugs, 0 vulnerabilidades, 0 security hotspots**, calificación A en las tres dimensiones (fiabilidad, seguridad, mantenibilidad), *Quality Gate* **OK**. Los 64 *code smells* encontrados son en su totalidad hallazgos de mantenibilidad —23 literales de cadena duplicados (regla `java:S1192`, mensajes de error como `.Estudiante no encontrado con id: "` repetidos en varios servicios), 4 métodos vacíos sin comentario (regla `java:S1186`, en su mayoría *pointcuts* de `AuditoriaAspectTest` que no necesitan cuerpo) y 2 métodos con complejidad cognitiva sobre el umbral (regla `java:S3776`: `AlineacionService` y `SesionEntrenamientoService`, ambos con lógica de negocio real, candidatos a extracción de métodos, no *spaghetti code* accidental—. No se integró como paso de CI en esta entrega por requerir un servidor SonarQube persistente, no una instancia efímera; queda declarado como trabajo futuro. Reporte y datos crudos completos en `docs/mediciones/sonarqube/`.

12.5. Cobertura de pruebas

Primera corrida completa, medida el 2026-07-30 con construcción limpia (`./mvnw clean test`): **72,5 % de cobertura de instrucciones (2507 cubiertas de 3457), con 102 pruebas y ningún fallo**. El estado actual, con más módulos y más pruebas, se reporta al final de esta sección; el camino hasta aquí quedó documentado en vez de solo reportar el resultado final: al medir por primera vez después de la reestructuración de paquetes (sección 6) el resultado real era **39,8 %** — los recursos nuevos (*Categoria*, *Entrenador*, *Usuario*, *Persona*, *EstadoGeneral*) no tenían ninguna prueba propia. Se agregaron 57 pruebas nuevas para esos cinco recursos (10 clases: servicio y controlador de cada uno), y la cobertura subió por encima del 70 %.

La palabra «limpia» está ahí por una razón concreta. La primera medición posterior a la reestructuración se hizo con `./mvnw test` sobre un directorio `target/` que todavía conservaba archivos `.class` compilados *antes* del cambio de paquetes. El reporte que se llegó a archivar como evidencia listaba paquetes que ya no existen en el código fuente (`org.uteq.backend.auth.*`, `org.uteq.backend.estudiante.*`): JaCoCo instrumenta lo que encuentra en `target/classes`, no lo que hay en `src/`. La cifra publicada aquí proviene de `clean test` y el reporte archivado en `docs/mediciones/jacoco/` contiene exactamente los 27 paquetes reales. La diferencia numérica entre ambas mediciones era de apenas unas décimas, y ese es justamente el punto: una evidencia de cobertura que incluye código inexistente no es evidencia válida *aunque* el porcentaje salga parecido, porque nada garantiza que la próxima vez la distorsión sea igual de pequeña. Para que el defecto no pueda repetirse, el objetivo `make test` pasó a ejecutar `clean test`. La Tabla 22 detalla esa primera corrida clase por clase.

Un detalle técnico real que surgió al escribir estas pruebas: los cuatro controladores que reciben `Pageable` como parámetro (*Categoria*, *Entrenador*, *Usuario*, *Persona*) fallaban con 500 bajo `MockMvcBuilders.standaloneSetup()`, porque ese modo de arranque no registra el `PageableHandlerMethodArgumentResolver` que sí provee el contexto completo de Spring — *EstudianteController* no lo sufre porque usa `@RequestParam` planos en vez de `Pageable`. Se resolvió registrando el resolutor explícitamente en cada prueba.

Un detalle relevante de una entrega anterior: se había descubierto que JaCoCo *nunca* había generado cobertura real, porque el `argLine` configurado en Surefire sobrescribía el `javaagent` del propio JaCoCo. Ese defecto está corregido; por eso esta medición pudo mostrar honestamente primero una regresión y después la mejora real.

Las pruebas de `LoginAttemptServiceTest` merecen mención aparte: verifican no solo que el bloqueo ocurra, sino que un fallo posterior *no* reinicie la ventana de tiempo. Sin esa segunda propiedad, un atacante podría mantener la cuenta en estado de bloqueo indefinido, o bien —según cómo se implemente— reiniciar el contador y evadir el límite.

Cifra de cierre (2026-09-11). Esta es la única cifra de cobertura vigente en el documento, recalculable línea por línea desde `docs/mediciones/jacoco/jacoco.csv` con `./mvnw verify` —el comando que exige y hace cumplir el umbral, no solo `test`—: **88,38 % de cobertura de líneas** (3013 de 3409) y **74,20 % de branches** (791 de 1066), sobre 216 clases analizadas, con **665 pruebas** en 76 clases de prueba y ningún fallo. Cumple el **70 %** que exige `pom.xml` en líneas y en *branches* —el umbral real del proyecto; una redacción anterior de este párrafo citaba 60 %, que nunca fue el valor configurado— con un margen de 4,20 puntos sobre el mínimo de *branches*. Esta corrida corresponde a la revisión contra *Rubrica_ExamenFinal_SGED.pdf* (Punto 2): la cifra anterior (87,47 %/576 pruebas del 2026-09-07) había quedado desincronizada del *SRS.md* (que citaba 84,66 %/71,24 %, una cifra de la bitácora, no la vigente) tras el renombrado a inglés de los métodos fuera de *deportivo* (Puntos E1/E2 de la misma rúbrica), que agregó cobertura nueva sin que el CSV committeado se hubiese regenerado. El camino hasta esta cifra —41,9 % tras la reestructuración, 61,16 % con *main* dos semanas en rojo, 70,06 % de cierre provisional el 2026-09-02, 84,66 % y luego 87,47 % de cierres anteriores— queda documentado como bitácora en la sección 1.7; ninguna de esas cifras intermedias describe el estado actual. El desglose por capa se resume en la Tabla 23.

Cuadro 22: Clases de prueba JUnit y qué verifica cada una (primera corrida completa, 2026-07-30; el estado actual se reporta más abajo).

Clase de prueba	Nº	Qué verifica
AuthServiceTest	6	Login correcto e incorrecto, registro, registro duplicado, registro incompleto rechazado con 422, disponibilidad.
JwtServiceTest	6	Emisor y audiencia válidos, audiencia distinta rechazada, token manipulado, refresco.
LoginAttemptServiceTest	6	Bloqueo al alcanzar el límite, no reinicio del TTL, borrado del contador al acertar.
RedisBlacklistServiceTest	6	Revocación con TTL restante, TTL no positivo ignorado, consulta de revocación.
EstudianteControllerTest	9	Los siete verbos del recurso más 404 y 422.
EstudianteServiceTest	11	Paginación envuelta, baja lógica, persistencia transaccional, delegación al procedimiento SQL.
BackendApplicationTests	1	Arranque del contexto contra el esquema migrado.
CategoriaServiceTest	9	Paginación, alta, edición, baja lógica, validación de rango de edad.
CategoriaControllerTest	7	Mapecto HTTP: 200/201/204/400/404/422.
EntrenadorServiceTest	6	Paginación con mapeo persona/usuario, persona o usuario duplicados, alta, baja lógica.
EntrenadorControllerTest	5	Mapecto HTTP: 200/201/204/404/422.
UsuarioServiceTest	6	Paginación, username duplicado, alta con contraseña codificada, persona inexistente, baja lógica.
UsuarioControllerTest	6	Mapecto HTTP: 200/201/204/400/422.
PersonaServiceTest	8	Paginación, búsqueda por cédula, unicidad de cédula/correo al crear y editar, baja lógica.
PersonaControllerTest	6	Mapecto HTTP: 200/201/204/400/422.
EstadoGeneralServiceTest	2	Listado completo y listado vacío.
EstadoGeneralControllerTest	1	Mapecto HTTP del único <i>endpoint</i> .
Total	102	

Cuadro 23: Cobertura actual por capa arquitectónica.

Capa	Líneas	Branches
Servicios	87,11 % (1919/2203)	72,80 % (514/706)
Controladores	88,44 % (260/294)	79,41 % (27/34)
Otros (config., seguridad, DTO)	87,33 % (503/576)	64,46 % (107/166)
Entidades	100 % (48/48)	92,31 % (24/26)

El desglose por capa agrega 200 clases en 4 filas; para auditar un subdominio concreto, la Tabla 24 desagrega los 87 paquetes reales de `docs/mediciones/jacoco/jacoco.csv` en sus 25 grupos *dominio.subdominio* (uniendo `controller`, `dto`, `service`, `entity`, etc. de un mismo subdominio en una fila; sin agrupar habría 87 filas, más de las que esta página puede mostrar con utilidad). Generada por el mismo script que produjo la cifra agregada de arriba —no hay una segunda fuente de verdad—; recalculable con `python3` sobre el CSV citado.

Cuadro 24: Cobertura por subdominio (agregando `controller/dto/service/entity/etc.`), recalculada desde `jacoco.csv`.

Subdominio	Líneas	Branches
raíz	96,4 % (54/56)	—
academico.alert	100,0 % (52/52)	100,0 % (24/24)
academico.guardian	86,6 % (284/328)	85,4 % (41/48)
academico.payment	72,8 % (83/114)	67,9 % (19/28)
academico.student	84,2 % (208/247)	60,0 % (54/90)
common	81,1 % (236/291)	56,0 % (56/100)
deportivo.asistencia	97,1 % (169/174)	93,9 % (62/66)
deportivo.categoria	95,4 % (62/65)	80,0 % (8/10)
deportivo.entrenador	80,0 % (68/85)	62,5 % (10/16)
deportivo.especialidad	92,5 % (37/40)	66,7 % (4/6)
deportivo.evaluacion	79,9 % (123/154)	53,8 % (28/52)
deportivo.horario	94,8 % (109/115)	92,3 % (24/26)
deportivo.lesion	100,0 % (63/63)	85,0 % (17/20)
deportivo.partido	64,5 % (196/304)	54,5 % (61/112)
deportivo.posicion	100,0 % (5/5)	—
deportivo.sesion	95,2 % (119/125)	84,8 % (39/46)
inventario.assignment	93,8 % (90/96)	84,4 % (27/32)
inventario.item	93,8 % (60/64)	83,3 % (10/12)
inventario.movement	93,3 % (42/45)	100,0 % (4/4)
reportes	93,8 % (135/144)	75,0 % (33/44)
seguridad.audit	96,8 % (91/94)	82,0 % (41/50)
seguridad.auth	99,6 % (223/224)	83,8 % (57/68)
seguridad.person	93,3 % (70/75)	71,4 % (10/14)
seguridad.status	100,0 % (8/8)	—
seguridad.user	93,5 % (143/153)	67,2 % (43/64)

El subdominio más bajo en líneas es `deportivo.partido` (64,5 %): agrupa la planificación de encuentros y alineaciones, un subdominio con servicios grandes y poca cobertura de *branches* respecto a su volumen. Le sigue `common` (81,1 %), que agrupa `common.exception` (manejo de errores, ahora al 100 % tras los tests añadidos para este pendiente) y `common.ia` (el cliente de *feedback* por IA, con rutas de reintento y proveedor alternativo). Ninguno de los 25 subdominios está en cero.

Una redacción anterior de este párrafo atribuía el bajo porcentaje de la capa de entidades (entonces 27,08 %) a que JaCoCo contaba los *getters/setters* generados por Lombok. Eso era incorrecto: JaCoCo 0.8.12 ya filtra el código anotado como generado, y las entidades 100 % Lombok (`Estudiante`, `Categoria`, `Representante`...) ni siquiera figuran en el reporte. Lo que faltaba cubrir eran las *devoluciones de ciclo de vida* JPA escritas a mano —`@PrePersist/@PreUpdate`: sellado de marcas de tiempo, `activo` por defecto, normalización de `username`—, que los tests de servicio con *mocks* nunca disparan. Se agregó `EntidadCicloVidaTest`, que las invoca por reflexión y verifica su efecto y ambas ramas; la capa de entidades pasó a 100 % de líneas. Los

cuatro controladores que el crecimiento del módulo deportivo había dejado en cero por ciento —`PartidoController`, `PosicionController`, `AsistenciaSesionController`, `ResumenAsistenciaController`— tienen ahora suite propia (los cinco que citaba la Entrega 1B ya se habían cubierto en 94aeace). Ningún controlador queda en cero.

12.6. Calidad web: Lighthouse (Bloque C.5 / A.1)

Seis corridas independientes, tres por perfil (Lighthouse v13.4.1), regeneradas el 2026-08-14 contra `https://localhost:8443` —cumple el mínimo de tres corridas por perfil (móvil y escritorio) que exige el Bloque A.1; la Tercera Entrega solo había cubierto el perfil móvil— (Tabla 25). Además, en la sección 12.6.2 se amplía con una segunda medición: contra la URL pública de producción y con sesión autenticada real:

Cuadro 25: Resultados de Lighthouse por categoría y perfil.

Categoría	Móvil	Escritorio	Umbral	Estado
Rendimiento	82,0	99,0	≥ 80	OK
Accesibilidad	100	100	≥ 90	OK
Buenas prácticas	96	96	≥ 90	OK
SEO	63	63	≥ 90	Ver §12.6.1

Desviación típica de 0 en las tres corridas de cada perfil: con `throttlingMethod: simulate`, Lighthouse deriva los tiempos de un único *trace* más un modelo determinista de red/CPU (Lantern), no de una limitación real variable por corrida — es el comportamiento esperado del método, no un defecto de la medición. Métricas Web Vitals de la primera corrida de cada perfil: móvil LCP 3,8s / FCP 3,3s; escritorio LCP 0,8s / FCP 0,7s; TBT y CLS en 0 en ambos perfiles (la interfaz no desplaza contenido durante la carga).

Corrección metodológica de esta misma sesión, dejada explícita: la primera corrida de escritorio, con la configuración de *throttling* del perfil móvil aplicada por error al `form-factor` de escritorio, midió 62/100 de rendimiento —por debajo del umbral—. Antes de reportarlo como hallazgo real se verificó la causa: ese *throttling* manual no es una configuración válida para escritorio. Repetida con el *preset* oficial `-preset=desktop` de la propia herramienta, el resultado sube a 99/100 de forma consistente en las tres corridas. Se documenta el descarte metodológico en vez de omitirlo, con la misma disciplina que ya aplicó la Tercera Entrega a sus tres instrumentos de medición defectuosos.

Esta medición había expuesto, y permitido corregir en la Tercera Entrega, tres defectos reales del *frontend*: el documento declaraba `lang="en"` en una aplicación íntegramente en español; conservaba el título `Frontend` generado por Angular CLI y no tenía descripción; y carecía de un elemento `<main>`, lo que hacía fallar la auditoría de puntos de referencia de accesibilidad. Corregidos los tres, la accesibilidad pasó de 97 a **100**, y se mantiene en 100 en ambos perfiles en esta regeneración.

12.6.1. Por qué el SEO de 63 es el resultado correcto

El puntaje SEO bajó de 82 a 63 *a causa de una corrección deliberada*. Al añadir un `robots.txt` válido con `Disallow: /`, Lighthouse activa la penalización `is-crawlable` («página bloqueada para indexación»), que por sí sola descuenta 27 puntos, porque su categoría SEO asume que el sitio *quiere* ser indexado.

SGED es una aplicación de gestión interna que trata datos personales de menores de edad. Que fuese indexable por buscadores sería un defecto de privacidad, no una virtud. Elevar el SEO a 90 exigiría eliminar el `robots.txt`: degradar la privacidad del sistema para mejorar una métrica. Se optó por lo contrario, se relajó el umbral agregado de esa categoría a advertencia con la

justificación escrita en el propio archivo de configuración, y se mantuvieron como bloqueantes las auditorías SEO que sí aplican a una aplicación interna (`meta-description`, `document-title`, `html-has-lang`, `viewport`), todas ellas en 1,0. **OK**

12.6.2. Medición pública en el despliegue real (2026-09-08)

La medición anterior se hacía contra `localhost:8443`, sin sesión y con una sola ruta; además, el `requestedUrl` apuntaba al entorno local, lo que no satisfacía el criterio de la observación 4.5 del protocolo de medición. Para cerrarlo, esta revisión mide el despliegue público en Render (<https://sged-frontend-r2rs.onrender.com>) con sesión iniciada de verdad y sobre dos rutas autenticadas, y la herramienta queda versionada en el repositorio y ejecutable de forma reproducible en GitHub Actions:

- `scripts/lighthouse-ci.mjs`: inicia sesión real (POST `/api/auth/login` contra el *proxy* del propio *frontend*), inyecta la cookie `sged_access` por CDP (`Network.setCookie`) y ejecuta Lighthouse reutilizando ese Chrome, con `disableStorageReset` para que la sesión persista entre corridas. Sin esto, `authGuard` redirigiría a `/login` y se mediría la pantalla de inicio en vez de la página real.
- `.github/workflows/lighthouse.yml`: ejecuta el script en GitHub Actions (Chrome preinstalado en el runner) con las credenciales provistas por *secrets* (`LH_USER`, `LH_PASS`) y sube las doce evidencias como artefacto.

Las doce corridas cubren dos perfiles × dos rutas autenticadas (`/dashboard` e `/inventario`), tres por pareja (Tabla 26); los doce LHR crudos quedan archivados como `docs/mediciones/lighthouse/public`. todos con `requestedUrl` y `finalUrl` en la URL pública (la sesión funciona: ninguna terminó redirigida a `/login`).

Cuadro 26: Resultados de Lighthouse en el despliegue público, por perfil y ruta (medias de 3 corridas; Lighthouse v13.4.1).

Perfil	Ruta	Rend.	Acces.	Buenas prác.	SEO
Móvil	<code>/dashboard</code>	99,3	91,0	100	63
Móvil	<code>/inventario</code>	99,7	100	100	63
Escritorio	<code>/dashboard</code>	80,0	91,0	100	63
Escritorio	<code>/inventario</code>	99,3	95,0	100	63

Todas las parejas superan los umbrales (rendimiento ≥ 80 , accesibilidad ≥ 90 , buenas prácticas ≥ 90); el SEO de 63 es la penalización deliberada de `is-crawlable` explicada en la sección 12.6.1. La escritorio/`/dashboard` queda exactamente en el umbral de rendimiento (80,0) en las tres corridas, y la accesibilidad mínima es 91 (en `/dashboard`, frente a 100 en `/inventario` móvil) — ambas superan el umbral y se reportan sin promediarlas para no ocultar la variación por ruta. Los seis LHR de la medición local de 2026-08-14 se conservan como referencia temporal; la evidencia vigente para el Bloque C.5 es esta suite pública.

12.7. Usabilidad: SUS (Bloque C.3)

El instrumento (`docs/mediciones/sus/INSTRUMENTO-SUS.md`) es el cuestionario SUS de diez ítems [7] con la escala adjetival de interpretación [4], administrado en español: la validación psicométrica de Castilla *et al.* sobre 1321 participantes hispanohablantes confirma que las formas completa y corta del SUS en español conservan propiedades psicométricas válidas, con el cuidado adicional de vigilar el efecto de los ítems redactados en negativo [10]. El script `scripts/sus-analysis.py` calcula media, desviación típica, intervalo de confianza al 95 %, mediana y grado por participante;

se niega deliberadamente a ejecutarse con menos de diez participantes, y su aritmética quedó verificada contra los cuatro casos de referencia de la literatura (0, 50, 75 y 100 puntos) antes de usarlo con datos reales.

Aplicado a 15 participantes de la muestra descrita en la sección 11.4, en dos tandas (2026-07-30 y 2026-08-18; formularios en papel, transcritos a docs/mediciones/sus/respuestas.csv sin registrar nombres, solo un código de participante y su perfil): 4 entrenadores, 4 estudiantes, 4 representantes y 3 recepcionistas.

La segunda tanda no se recolectó para mejorar el resultado sino para reducir la incertidumbre de la primera, y conviene juzgarla por eso: al pasar de 10 a 15 participantes la media se movió 1,08 puntos (68,25 a 69,33) mientras la amplitud del intervalo de confianza cayó en torno a un tercio (31,7 a 20,9 puntos, calculada con la t de Student). La estimación ya era estable; lo que faltaba era precisión.

Cuadro 27: Estadísticos agregados de la encuesta SUS.

Métrica	Valor
Media SUS	69,33
Desviación típica	18,89
IC 95 %	$69,33 \pm 10,46$ (58,87 – 79,79)
Mediana	70,00
Grado	C — Aceptable

Según la Tabla 27, la media cruza el umbral de 68 por 1,33 puntos y el intervalo de confianza todavía incluye valores por debajo de ese umbral: con esta muestra la afirmación defendible es que la mejor estimación puntual está por encima de 68, no que el sistema lo supere con confianza estadística. No se presenta como un resultado cómodo.

El hallazgo más informativo no es la media, es su distribución bimodal por perfil (Tabla 28), que la agregación esconde:

Cuadro 28: Puntuación SUS promedio por perfil de usuario.

Perfil	Promedio SUS	Grado
Entrenador (n=4)	85,0	A
Estudiante (n=4)	83,8	A
Recepcionista (n=3)	51,7	F
Representante (n=4)	52,5	F

Los cuatro perfiles se parten en dos bloques con muy poca dispersión interna: 1,2 puntos de diferencia dentro del grupo alto, 0,8 dentro del bajo, y 33,3 entre ambos — más de una vez y media la amplitud del intervalo de confianza.

La línea divisoria no es ser interno o externo a la escuela: el recepcionista es personal interno y de uso diario, y puntúa igual de bajo que el representante. Lo que separa a los grupos es *qué hace cada rol con el sistema*. Entrenador y estudiante sobre todo consultan — ver sesiones, marcar asistencia, revisar el equipo —, y esos son los flujos que más atención de diseño recibieron. Recepcionista y representante operan o esperan información: el primero da de alta, edita y cobra; el segundo busca visibilidad sobre su representado y hoy el sistema no le envía nada por iniciativa propia (RF-22 sigue sin implementarse). Dicho de otro modo, quien peor califica el sistema es quien más trabajo administrativo hace con él.

El patrón resistió dos ampliaciones sucesivas de la muestra: los cinco participantes agregados — dos estudiantes, dos representantes y un entrenador — cayeron cada uno en el bloque que su rol predecía, sin excepción. Es una pista concreta de dónde enfocar el trabajo de usabilidad, no solo un número que aprobó por poco.

Con 3 o 4 participantes por perfil estos promedios son indicativos y no soportan una prueba de hipótesis entre grupos: el patrón se sostiene por su magnitud y por su consistencia al ampliar la muestra, no por significancia estadística. El análisis completo, con las amenazas a la validez, está en docs/mediciones/sus/INTERPRETACION.md.

13. Reproducibilidad

El sistema se levanta completo desde una clonación limpia con un solo comando, siguiendo la misma motivación que documentan Cito y Gall para usar contenedores como mecanismo de reproducibilidad en investigación de ingeniería de software: encapsular las suposiciones y dependencias que de otro modo quedan como conocimiento tácito del equipo [12] (Listado 6):

```
1 git clone https://github.com/DarwinSM21/SGED_APPWEB.git
2 cd SGED_APPWEB
3 cp .env.example .env
4 make up
5 make carga # volumen sintetico (2401 estudiantes) para reproducir la sec. de
rendimiento
```

Listado 6: Arranque reproducible

El mismo Makefile expone, además del arranque, los objetivos que reproducen cada bloque de evidencia de este informe (Tabla 29):

Cuadro 29: Objetivos del Makefile para reproducción end-to-end.

Objetivo	Acción
make up	Levanta el sistema completo
make test	JUnit 5 + cobertura JaCoCo
make bench	5 corridas k6 por escenario (cálida y fría) + análisis inferencial con IC 9
make carga / make limpiar-carga	Sembrar / retirar el volumen sintético para reproducir el benchmark
make audit	Auditoría OWASP (6 controles) + SQL dinámico
make clean	Limpia contenedores, volúmenes y compilaciones

Las imágenes de PostgreSQL y Redis están fijadas por digest SHA-256, no por etiqueta móvil: dos construcciones del mismo *commit* usan exactamente los mismos binarios. La reproducibilidad se verificó clonando el repositorio en un directorio independiente, no solo reiniciando el volumen local.

14. Amenazas a la validez

Los resultados de la sección anterior cumplen los umbrales exigidos, pero cumplirlos no equivale a ser generalizables. Esta sección declara las limitaciones que acotan hasta dónde puede sostenerse cada conclusión, siguiendo la clasificación habitual en ingeniería de software empírica [25].

14.1. Validez de constructo

¿Mide cada indicador lo que se pretende medir?

- **La cobertura de instrucciones no mide calidad de las pruebas.** Un 88,38 % indica qué proporción del código se ejecuta durante la suite, no que las aserciones sean pertinentes —el mismo desacople que documentan Imran *et al.* sobre pruebas de rendimiento en sistemas Java de código abierto: alcanzan una cobertura sistemáticamente menor que las pruebas funcionales pese a ser igual de válidas, porque miden algo distinto de lo que la cobertura agregada

expresa [23]. El caso del registro de usuarios lo demuestra dentro de este mismo proyecto: el *endpoint* estaba cubierto por dos pruebas que pasaban y aun así fallaba en toda petición real (sección 12.3).

- **La auditoría OWASP verifica controles, no ausencia de vulnerabilidades.** Comprueba seis controles concretos del Top 10 con peticiones dirigidas. No es una prueba de penetración ni un análisis estático: que los seis den correcto no permite afirmar que el sistema sea seguro, solo que esos seis comportamientos son los esperados.
- **El SUS mide usabilidad percibida, no desempeño.** No se registraron tiempos ni tasas de finalización por tarea, de modo que un participante pudo puntuar alto sin haber completado las tareas propuestas. Las columnas de completitud del CSV quedaron vacías y se declaran vacías en lugar de rellenarse.

14.2. Validez interna

¿Puede algo distinto de lo declarado explicar los resultados?

- **Las mediciones de rendimiento se tomaron en la misma máquina que ejecuta el sistema,** sin latencia de red real. Los 19,94 ms de p95 en caché cálida (38,20 ms en fría) son un límite inferior optimista: en un despliegue con red de por medio la cifra sería mayor, aunque el margen frente al umbral de 200 ms es amplio.
- **Cinco corridas por escenario son pocas** para caracterizar una distribución. Se reporta el intervalo de confianza al 95 % para no presentar una única corrida como si fuera el valor verdadero, pero con $n = 5$ ese intervalo es ancho.
- **La base de datos de las mediciones contiene datos sembrados, no un volumen de producción.** El rendimiento de las consultas paginadas sobre un catálogo pequeño no anticipa el comportamiento con miles de estudiantes.

14.3. Validez externa

¿Hasta dónde se generalizan los resultados?

- **La muestra del SUS es de 15 participantes,** recolectada por conveniencia. Cumple el mínimo exigido, pero el intervalo de confianza (58,87–79,79) todavía incluye valores por debajo del umbral de 68: 69,33 es la mejor estimación puntual disponible, no un aprobado consolidado.
- **Los subgrupos por perfil tienen 3 o 4 personas cada uno.** El patrón bimodal es nítido y tiene una explicación plausible ligada a qué flujos están implementados, pero con esos tamaños no puede descartarse que parte de la diferencia sea variación individual. Se presenta como hipótesis orientadora para la Entrega Final, no como conclusión establecida.
- **Los resultados describen este sistema en este entorno.** No se comparan contra un sistema de referencia ni contra una versión anterior medida en las mismas condiciones.

14.4. Validez de conclusión

El riesgo principal de este informe no es estadístico sino de procedimiento, y ya se materializó tres veces: **un indicador puede dar verde sobre un sistema que no hace lo que el indicador afirma** (sección 12.2.1). Las tres ocurrencias se corrigieron en su origen, pero la lección se declara aquí porque acota la confianza que merece cualquier cifra de este documento: son válidas en tanto la medición se ejecute contra el sistema real, desde estado limpio, y verificando las dos direcciones del comportamiento esperado.

Queda además una divergencia sin resolver que afecta la reproducibilidad a futuro: el esquema consolidado que crea la base de datos (`db/schema.sql`) y las migraciones Flyway versionadas describen estructuras distintas, porque las segundas no se actualizaron tras la reestructuración y hoy no se ejecutan (`FLYWAY_ENABLED=false`). El sistema arranca de forma reproducible por la primera vía, que es la verificada en este informe, pero la decisión sobre cuál de las dos fuentes queda como autoritativa está pendiente y se declara abierta.

Finalmente, **el DOI de Zenodo ya está asignado**: el corte `v1.0.0` de este repositorio está archivado con DOI [10.5281/zenodo.22714477](https://doi.org/10.5281/zenodo.22714477) (concept DOI [10.5281/zenodo.21713239](https://doi.org/10.5281/zenodo.21713239)), publicado el 2026-09-11 como nueva versión de la misma serie mediante la integración GitHub → Zenodo sobre el *release* de esa etiqueta, lo que satisface el archivo permanente exigido por el Bloque E.2. Un depósito anterior, publicado por error como registro independiente en vez de nueva versión de esta serie ([10.5281/zenodo.22635766](https://doi.org/10.5281/zenodo.22635766), corte `v1.0.1`), quedó retirado en Zenodo y no debe citarse. La versión `v0.9.0-rc` del repositorio anterior es la primera de esta misma serie y conserva su propio DOI de versión ([10.5281/zenodo.21713240](https://doi.org/10.5281/zenodo.21713240)).

15. Consideraciones éticas

El sistema trata datos personales de niños, niñas y adolescentes: nombres, cédula, fecha de nacimiento, asistencia con hora y lugar, y evaluaciones individuales de desempeño. Esa combinación es sensible porque los titulares son menores que no pueden consentir por sí mismos, porque la asistencia es información de localización temporal, y porque las evaluaciones constituyen perfilado de personas [1, 2]. El riesgo no es solo regulatorio: Carvalho y Gonçalves advierten sobre la presión creciente por identificar talento a edades cada vez más tempranas en el deporte formativo, y sobre el desajuste entre los indicadores que se miden y el desarrollo real del deportista joven [9]. Un sistema que registra puntajes periódicos de un menor no es un instrumento neutral: puede consolidar una etiqueta temprana que condicione decisiones posteriores sobre esa misma persona.

El principio de minimización se aplicó al diseño original —se descartaron contextura y datos de alimentación—, pero conviene ser preciso sobre su alcance real, porque la versión anterior de este informe afirmaba que tampoco se recolectaban peso y altura y **esa afirmación ya no es cierta**: la reestructuración incorporó ambas columnas a `academico.estudiantes` y son exponibles por la API. El documento de ética se corrigió (hallazgo H-06) en lugar de mantener la declaración cómoda, y este informe se corrige aquí por la misma razón: una declaración de minimización que no coincide con el esquema no protege a nadie.

`docs/etica/ETHICS.md` documenta nueve hallazgos, en lugar de afirmar que todo está resuelto: la cédula se almacena sin validación ni cifrado (H-01); las observaciones del entrenador son texto libre sin control sobre un menor (H-02); la baja lógica preserva el historial pero no es un borrado, por lo que hacía falta un mecanismo de supresión efectivo —resuelto el 2026-09-08 con el procedimiento de anonimización de RF-50— (H-03); el consentimiento del representante no está modelado, lo que bloquea el módulo de notificaciones (H-04); el certificado TLS era autofirmado —resuelto el 2026-09-09 con el despliegue en Render, que sirve HTTPS con certificado de una autoridad reconocida— (H-05); peso y altura se agregaron sin base legal documentada —resuelto el 2026-09-09: finalidad y base legal declaradas, lectura restringida por rol y alcance de consentimiento propio— (H-06); y la plantilla de consentimiento cubría a los evaluadores del SUS, no a los representantes de los menores —resuelto el 2026-09-09 con `docs/etica/consentimiento/representante.md`— (H-07).

El octavo, **H-08**, se registra ya corregido, y se deja escrito justamente porque corregirlo en silencio habría sido lo fácil: los datos identificativos de los estudiantes estuvieron accesibles a cualquier cuenta autenticada del sistema, incluida la búsqueda por número de cédula (sección 12.2.1). Un documento de ética que solo enumera riesgos hipotéticos y omite la exposición concreta que sí ocurrió sería un documento de imagen, no de rendición de cuentas.

15.1. Generación de retroalimentación con un modelo de lenguaje externo

El sistema incorpora un servicio opcional (`backend/.../common/ia/GeminiFeedbackService.java`) que redacta comentarios de retroalimentación deportiva invocando un modelo de lenguaje de un tercero. Enviar información sobre menores de edad a un proveedor externo es, por sí mismo, una superficie de divulgación nueva, así que las decisiones de diseño se documentan aquí y no solo en el código:

- **Seudonimización estructural, no por convención.** Lo único que se serializa hacia el proveedor es el *record* `PerfilJugadorAnonimo`, que no tiene campos de nombre, cédula, correo ni fecha de nacimiento. No es que el código «evite» enviarlos: no existen en el tipo que viaja, de modo que no hay forma de filtrarlos aunque una modificación futura lo intentara por descuido.
- **La clave viaja en cabecera, no como parámetro de consulta.** Ambas formas están documentadas por el proveedor, pero un parámetro de consulta queda registrado en *logs* de acceso, historiales de *proxy* y cabeceras *Referer*.
- **Desactivado por defecto y degradación silenciosa.** Sin clave configurada el servicio reporta no disponible y el resto de la evaluación funciona igual; un fallo del proveedor nunca puede hacer perder al entrenador lo que ya calificó a mano.
- **Instrucción de sistema que prohíbe inventar.** El *prompt* de sistema acota el registro, prohíbe explícitamente afirmar hechos que no estén en los datos numéricos recibidos, exige tono apropiado para un menor y veta diagnósticos médicos o recomendaciones de aumentar carga física a un jugador lesionado.
- **Los registros no incluyen el *prompt*.** Ante un fallo se registra el tipo de excepción, nunca el contenido enviado.

Estas mitigaciones acotan el riesgo de divulgación, no la calidad del texto generado, que es un problema distinto y abierto: Dai *et al.* muestran que la retroalimentación producida por modelos de lenguaje puede ser fluida y coherente y aun así variar en su pertinencia según el criterio evaluado [16]. Por eso el comentario generado se presenta como apoyo a la redacción del entrenador y no como sustituto de su juicio, y su evaluación empírica queda declarada como trabajo pendiente: **esta entrega no aporta evidencia sobre la calidad de los textos generados**, solo sobre las garantías de privacidad de su integración.

Todos los datos del repositorio son ficticios. No se utilizaron datos reales de menores en ninguna etapa del desarrollo ni de las mediciones.

16. Declaración de contribuciones (CRediT)

Los roles siguientes no son autodeclarados: se derivan del historial de `git` del repositorio y cada uno puede verificarse con `git log -author`. Las cifras provienen de `git log -numstat` sobre la rama `main` y separan lo *escrito* de lo *generado* (reportes de JaCoCo y Lighthouse, salidas crudas de `k6`, `package-lock.json`, PDF), porque contar un reporte HTML de cobertura como líneas de autoría inflaría la medida sin reflejar trabajo (Tabla 30).

16.1. Roles por integrante

Pallo Pinto Alejandro Daniel — *Software, Análisis formal, Validación, Curación de datos, Redacción (borrador original), Visualización.*

Responsable de la totalidad del eje de verificación que define esta entrega. En seguridad: migración del JWT de `localStorage` a cookie `HttpOnly+Secure+SameSite`, terminación TLS, CSP explícito, registro de auditoría A09, y la corrección del control de acceso ausente en los

Cuadro 30: Evidencia cuantitativa de autoría por integrante (`git log` sobre `main`, medido el 2026-09-11). La misma medición está en `CONTRIBUTORS.md`. Arcalle Grefa suma los commits de sus dos correos vinculados a la misma cuenta de GitHub (`darcalleg@uteq.edu.ec` y `darwinarcalleg@gmail.com`).

Integrante	Commits	Líneas escritas	Archivos
Pallo Pinto Alejandro Daniel	229	81 997	1 944
Arcalle Grefa Darwin Orlando	116	51 106	994
Velez Lopez Ricardo Elias	55	9 200	189
Total en main	400	142 303	3 127

cinco recursos nuevos (H-08, sección 12.2.1). En datos: conversión de las funciones PL/pgSQL a procedimientos reales invocables con `@Procedure` (sección 8) y fijación de imágenes por digest. En validación: las 57 pruebas que cerraron la primera regresión de cobertura (la suite siguió creciendo después, hasta las 576 actuales —sección 12.5—), la corrección del defecto por el que JaCoCo nunca había medido cobertura real, y las tres campañas de medición empírica (k6, OWASP, Lighthouse) junto con sus *scripts* de análisis. En usabilidad: instrumento SUS, *script* de cálculo validado contra los casos de referencia de la literatura, y transcripción de las diez encuestas. En redacción: este informe y el corpus documental (SRS, casos de uso, historias, matriz de trazabilidad, ADR, ética, diccionario de datos, gobernanza del repositorio).

Arcalle Grefa Darwin Orlando — *Conceptualización, Software, Investigación, Administración del proyecto, Supervisión.*

Estructura inicial del repositorio y del modelo de datos (`DataBase/`), trabajo sobre el *frontend* Angular (componentes, *Dockerfile* y configuración de formato), y administración del repositorio remoto, del que es propietario. Participación en la reestructuración en dominios `academico/deportivo/seguridad` descrita en la sección 6. Relevamiento de requisitos con la escuela ProFútbol y recolección de evidencia empírica (*Investigación*), y coordinación del equipo, calendario y entregas a lo largo de las cuatro fases (*Administración del proyecto* y *Supervisión*).

Velez Lopez Ricardo Elias — *Conceptualización, Software, Validación, Metodología, Recursos, Redacción (revisión y edición).*

Arranque del *frontend* Angular y del módulo de autenticación, CRUD inicial de **Estudiante**, y aportes a `docs/requisitos/` y `docs/observaciones/`. Autor principal de la reestructuración en tres dominios y de los cinco recursos CRUD nuevos, que es el cambio estructural sobre el que se apoya esta entrega. Definición del proceso de investigación (DSR) y del protocolo de medición (*Metodología*), configuración del entorno de despliegue en Render y de los contenedores Docker (*Recursos*), y revisión de la documentación y su consistencia con el código (*Redacción: revisión y edición*).

16.2. Cobertura de los catorce roles CRediT

La taxonomía CRediT define catorce roles. Trece quedan cubiertos por el equipo; *Adquisición de fondos* no aplica a un Proyecto Fin de Curso sin financiamiento externo (Tabla 31). La misma tabla está en `CONTRIBUTORS.md`.

16.3. Dos advertencias sobre la medida

El volumen no es la contribución. Las líneas escritas miden actividad, no valor: un cambio de dos líneas que corrige un fallo de control de acceso pesa más que dos mil líneas de documentación. La tabla se incluye porque el criterio de evaluación exige autoría verificable, no para establecer una jerarquía entre integrantes.

Cuadro 31: Cobertura de los catorce roles de la taxonomía CRediT.

Rol CRediT	Integrante(s)	Cobertura
Conceptualización	Ricardo, Darwin	Diseño de los cuatro dominios (académico, deportivo, inventario, seguridad) y de la estrategia híbrida de acceso a datos.
Curación de datos	Alejandro	Diseño del esquema, procedimientos almacenados y limpieza de los datos crudos de medición.
Análisis formal	Alejandro	Análisis estadístico de los datos de rendimiento y usabilidad (intervalos de confianza, distribución t).
Adquisición de fondos	—	No aplica (proyecto académico sin financiación externa).
Investigación	Darwin	Relevamiento de requisitos con la escuela ProFútbol y recolección de evidencia empírica.
Metodología	Ricardo	Proceso de investigación (DSR) y protocolo de medición.
Administración del proyecto	Darwin	Administración del proyecto, calendario y gestión de entregas.
Recursos	Ricardo	Configuración del entorno de despliegue (Render), contenedores Docker y base de datos.
Software	Alejandro, Ricardo, Darwin	Implementación de backend (Spring Boot), <i>frontend</i> (Angular) y procedimientos almacenados.
Supervisión	Darwin	Coordinación del equipo y seguimiento del repositorio.
Validación	Alejandro, Ricardo	Cobertura (JaCoCo), pruebas de carga (k6), estudio de usabilidad (SUS) y auditoría de seguridad.
Visualización	Alejandro	Diagramas C4 y de arquitectura del sistema.
Redacción (borrador original)	Alejandro	Redacción del informe, del SRS y de la documentación técnica.
Redacción (revisión y edición)	Ricardo	Revisión y corrección de la documentación y su consistencia con el código.

La identidad de `git` quedó unificada en correos institucionales. Al preparar el repositorio para la Entrega Final se reorganizó el historial para que todos los *commits* de `main` quedaran atribuidos a los tres correos `@uteq.edu.ec` del equipo (`dpallop`, `rvelezl3`, `darcalleg`). El estado anterior mezclaba correos personales (`outlook.es`, `gmail.com`), un tipeo (`uteq.edue.ec`) y dos *commits* del 2026-06-29 que —por la sesión configurada en un equipo de la universidad— figuraban bajo la cuenta `DannaN24`, siendo trabajo de **Pallo Pinto Alejandro Daniel** y ya sumados a su fila en la tabla anterior. Ninguno de esos correos aparece ya en `git log`: solo la atribución de autoría cambió, no el contenido de ningún archivo. La comprobación se repite listando los autores de `main` con `git log`; el detalle está en `CONTRIBUTORS.md`.

17. Trazabilidad y honestidad académica

La matriz `docs/trazabilidad/matriz.csv` enlaza cada requisito con su historia de usuario, su caso de uso, su *endpoint*, el archivo que lo implementa, la prueba JUnit que lo cubre y la evidencia empírica correspondiente.

El historial de `git` evidencia la autoría de cada integrante. Al preparar la Entrega Final se reorganizó ese historial para unificar la identidad de cada autor en su correo institucional `@uteq.edu.ec` —sin alterar el contenido de ningún archivo—, y `CONTRIBUTORS.md` documenta el estado anterior y cómo verificar el resultado. El mismo archivo declara explícitamente el uso de herramientas de asistencia por inteligencia artificial, siguiendo la guía de transparencia del SWEBOOK v4.0 [46].

18. Discusión

Nota de versión: esta sección responde a las preguntas de investigación (sección 1.2) sobre la evidencia remediada para la Entrega Final y archivada en `docs/mediciones/`, no sobre las corridas parciales que sirvieron a la Tercera Entrega. La distinción importa porque aquellas antecedian al módulo de inventario y a parte del dominio deportivo, y reportarlas sin remedir habría sido exactamente el error que la sección 12.3 documenta: un indicador verde que no refleja el sistema real.

RQ1 — Rendimiento. Sí, con margen amplio. Las cinco corridas independientes por escenario archivadas en `docs/mediciones/perf/` (50 usuarios virtuales, 30 s por corrida) dan, en caché cálida, una media de 11,28 ms (DT 2,93; IC 95 % $\pm 4,66$) y un p95 de 19,94 ms (IC 95 % $\pm 13,51$) contra el *endpoint* de listado protegido. El umbral de la norma es 200 ms con caché caliente: el sistema queda un orden de magnitud por debajo, y la diferencia frente a la caché fría (media 17,74 ms, p95 38,20 ms) es estadísticamente significativa (Mann-Whitney bilateral, $p < 10^{-60}$ en cada corrida; $p \approx 10^{-2483}$ en el pool; sección 12.1, Tabla 19). El contraste es relevante, no decorativo: la caché de Redis reduce el tiempo de respuesta del listado en $\approx 6,5$ ms de media y ≈ 18 ms en p95, un ahorro del orden del 35–45 % frente al escenario frío.

RQ2 — Seguridad. Los seis controles OWASP (A01, A02, A03, A05, A07, A09) están verificados con evidencia reproducible en `docs/mediciones/sec/` (sección 12.2.1), y una revisión directa del código fuente y de `ab/procs/` no encontró ningún uso de `createNativeQuery` con concatenación, `@Query` construida con `+`, ni `EXECUTE/sp_executesql` dentro de los procedimientos almacenados — la estrategia híbrida no introdujo la superficie de ataque que introduciría SQL dinámico mal construido. A esa revisión manual se suma la evidencia automática que exige la Entrega Final, reproducible por un tercero sin depender de ella: el escaneo con OWASP ZAP en modo *baseline* (`docs/mediciones/sec/zap/`, informe en HTML, JSON y XML junto a la configuración que lo genera) cierra con **cero hallazgos, de cualquier severidad**, tras apagar en el ambiente público la interfaz de Swagger UI que originalmente exponía una dependencia de terceros desactualizada; una segunda corrida con sesión autenticada y *active scan* (sección 12.4) confirma cero hallazgos de inyección también bajo ataque activo. El análisis estático (`docs/mediciones/sec/static-analysis/`) no reporta ocurrencias de la regla `SQL_NONCONSTANT_STRING_PASSED_TO_EXECUTE`, que es la comprobación automática equivalente a la inspección de código descrita arriba.

RQ3 — Usabilidad. La respuesta es clara y es, a la vez, el hallazgo menos cómodo de todo el documento (sección 12.7): la usabilidad **no** es homogénea entre roles. El sistema obtiene grado A en entrenadores y estudiantes y grado F en recepcionistas y representantes, con una media agregada (68,25) que esconde ese patrón bimodal en vez de exponerlo. La implicación práctica es directa: el esfuerzo de mejora de interfaz debería priorizar el flujo administrativo y el futuro módulo de representantes, no el promedio general del sistema. Con los quince participantes que exige la Entrega Final la media es 69,33 (IC 95 % 58,87–79,79), y la ampliación de diez a quince resultó informativa por una razón que conviene explicitar: la media se movió apenas 1,08 puntos mientras el intervalo de confianza se estrechó en torno a un tercio. Es decir, los cinco cuestionarios adicionales precisaron la estimación en lugar de desplazarla, lo que refuerza el patrón bimodal en vez de diluirlo. Aun así, con tres o cuatro participantes por rol la desagregación sigue siendo indicativa y no una estimación poblacional.

RQ4 — Cobertura y trazabilidad bajo crecimiento. Esta pregunta nace directamente de un patrón que ya se repitió una vez en la Segunda Entrega (sección 6): la cobertura cayó a 39,8 % cuando se agregaron cinco recursos nuevos sin pruebas propias, y se corrigió a 72,5 %. La pregunta para la Entrega Final es si ese patrón se repite con el módulo de inventario y el resto del dominio deportivo, agregados después de esa medición. El reporte remedido para esta entrega (`docs/mediciones/jacoco/jacoco.xml`) responde que el patrón **no** se repitió: 88,35 % de líneas (3013 de 3409) y 74,20 % de ramas (791 de 1066) sobre el `main` que incluye ya el módulo de inventario y el dominio deportivo completo (la cifra coincide con la de la sección 12.5 y es la única vigente en el documento). La comparación con el episodio de la Segunda Entrega es la que da sentido a la pregunta: allí la cobertura cayó a 39,8 % al agregar cinco recursos sin pruebas propias y hubo que corregirla a 72,5 %; aquí el crecimiento se acompañó de pruebas desde el principio y la cifra subió en vez de caer.

La advertencia que motivó RQ4 sigue siendo pertinente como práctica: el patrón puede volver

a ocurrir si no se vuelve a medir explícitamente en cada crecimiento futuro del dominio, de modo que la cobertura quede vinculada a una corrida reproducible (`./mvnw verify`) y no a una cifra declarada.

19. Trabajo futuro

Cinco líneas de trabajo quedan fuera del alcance de esta entrega, cada una con una razón concreta documentada en otra parte del proyecto — no son una lista genérica de ideas, son los pendientes que `docs/etica/ETHICS.md` y `VERSIONING.md` ya declaran abiertos.

1. **Envío proactivo de notificaciones al representante (RF-22).** La lectura de informes por un representante con vínculo activo ya funciona; el envío proactivo (push/email/SMS) queda fuera a propósito hasta no tener un mecanismo de consentimiento explícito y fechado, distinto del consentimiento general de matrícula (hallazgo H-04, resuelto parcialmente el 2026-08-03).
2. **Mecanismo de supresión de datos (resuelto el 2026-09-08).** El sistema aplica baja lógica, no borrado físico, por integridad referencial del historial deportivo. Para atender el derecho de supresión del representante se añadió el procedimiento almacenado versionado `academico.sp_anonimizar_estudiante` (migración V27), invocado por `POST /api/estudiantes/{id}/anon` — solo ADMINISTRADOR, auditado —: sustituye los datos identificativos por valores neutros y borra el texto libre sobre el menor, conservando claves foráneas y estadísticas agregadas (hallazgo H-03, requisito RF-50).
3. **Base legal documentada para peso y altura.** Estos dos campos son datos de salud de un menor sin finalidad de uso verificada, en el esquema pero sin ningún *endpoint* de escritura confirmado (hallazgo H-06). La línea de trabajo es doble: documentar una finalidad concreta con consentimiento separado, o directamente no exponerlos por API si ninguna funcionalidad prevista los necesita.
4. **Validación y protección de la cédula.** Hoy es un `VARCHAR(10)` sin validación de dígito verificador, sin restricción de unicidad y sin cifrado en reposo (hallazgo H-01) — más permisivo de lo deseable para un identificador nacional de un menor en un despliegue con datos reales.
5. **Cerrar la brecha de usabilidad por rol.** La sección anterior (RQ3) encontró grado F en recepcionistas y representantes. Rediseñar específicamente esos dos flujos —no el sistema en general— y volver a medir SUS por subgrupo con $N \geq 15$ es la línea de trabajo con el respaldo empírico más directo de todo este documento.

20. Conclusiones

Retomando lo declarado en el resumen (sección) y en el *abstract* (sección), los hallazgos de este trabajo se resumen así:

1. El sistema cumple con holgura los umbrales cuantitativos exigidos: p95 de 19,94 ms en caché cálida (38,20 ms en fría, un contraste significativo con $p \approx 10^{-2483}$) frente a un límite de 200 ms, 6 de 6 controles OWASP verificados con evidencia reproducible, accesibilidad Lighthouse de 100/100, y **88,38 %** de cobertura de líneas y **74,20 %** de *branches* frente al mínimo de 70 % (sección 12.5) — cifra que cayó dos veces por debajo del umbral a medida que el sistema creció (39,8 % tras la reestructuración de paquetes, 61,16 % tras agregar el módulo de alertas y un segundo proveedor de IA) y se recuperó las dos veces con pruebas dirigidas al déficit real, no reportando directamente un número cómodo. El margen sobre el umbral, de apenas 0,06 puntos en el cierre provisional del 2026-09-02, se llevó a 1,24 puntos cubriendo los cuatro controladores en cero y las devoluciones de ciclo de vida de las entidades.

2. **Una medición que no se ejecuta contra el sistema real no es evidencia.** Es la lección transversal de esta entrega, y aparece tres veces con la misma forma. La cobertura de JaCoCo se midió sobre un `target/` con clases previas a la reestructuración e incluía paquetes que ya no existen. La auditoría OWASP A01 devolvía 401 en todo y eso se parecía a un resultado correcto, cuando en realidad su propio control A07 la había dejado bloqueada. Y el registro de usuarios (RF-01) estaba roto contra la base de datos desde la reestructuración mientras sus dos pruebas unitarias pasaban, porque mockean el repositorio y la restricción la aplica PostgreSQL. En los tres casos el indicador era verde y el sistema no hacía lo que el indicador decía. Las tres mediciones se corrigieron en su origen — `make test` pasa a `clean test`, la auditoría limpia su propio contador y comprueba recurso por recurso— para que el defecto no pueda repetirse sin ser visible.
3. El aporte más sustantivo de esta entrega no fue añadir funcionalidad, sino *verificar* lo que ya existía, dos veces: una vez contra el código de la entrega original, y una segunda vez después de que otros dos integrantes reestructuraran el backend en paralelo. Ambas rondas revelaron defectos que ninguna revisión superficial había detectado — desde que JaCoCo no medía cobertura en absoluto, hasta que un ADR de arquitectura describía un mecanismo de autenticación (JWT en `localStorage`) distinto del que el código realmente usa (cookie `HttpOnly`), pasando por el control de acceso ausente en los cinco recursos nuevos (H-08). Todos los defectos encontrados se corrigieron y quedan documentados en las secciones 6 y 12.3 en vez de ocultarse.
4. La distinción entre lo implementado y lo modelado se mantiene explícita en todo el documento. En el caso de `Representante` y `Equipo` se llevó un paso más allá: en vez de dejar catorce clases vacías —una de ellas de cero bytes, el mismo defecto que motivó tres observaciones del docente en la Entrega 1B— se eliminaron del código y ambos módulos quedan declarados como pendientes solo en la documentación (sección 6). Un archivo vacío no cuenta como funcionalidad entregada, y tampoco debería quedarse en el repositorio aparentando que sí.
5. La encuesta SUS (sección 12.7) ya no es un frente abierto, pero su resultado abre uno nuevo: el sistema es excelente para entrenadores y estudiantes (grado A) e inaceptable para receptionistas y padres de familia (grado F). Mejorar la usabilidad del flujo administrativo y del futuro módulo de representantes es, con evidencia real detrás, más urgente que subir la media agregada. La exposición REST del dominio deportivo restante (horarios, sesiones, asistencias, evaluaciones) sigue siendo el objetivo natural de la Entrega Final.

21. Declaraciones obligatorias

Sección propia y consolidada, exigida por el Bloque B.15 de la Guía de la Entrega Final. Las declaraciones de ética y de contribuciones (CRedit) ya se desarrollaron con su propio detalle en las secciones correspondientes de este documento; aquí se referencian en vez de duplicarse, junto con las declaraciones que todavía no tenían una sección propia.

Disponibilidad de código. Repositorio público en https://github.com/DarwinSM21/SGED_APPWEB, etiqueta v1.0.0 (corte defendido, con DOI publicado). DOI del software en Zenodo: [10.5281/zenodo.22714477](https://doi.org/10.5281/zenodo.22714477) (concept DOI [10.5281/zenodo.21713239](https://doi.org/10.5281/zenodo.21713239)).

Disponibilidad de datos. Los datos crudos de todas las mediciones (rendimiento, seguridad, cobertura, calidad web, usabilidad) están versionados en `docs/mediciones/` bajo la misma licencia del repositorio. El depósito separado del *dataset* de mediciones está publicado en Zenodo con DOI propio ([10.5281/zenodo.22422305](https://doi.org/10.5281/zenodo.22422305)) y licencia Creative Commons Attribution 4.0.

Disponibilidad de materiales. Colección Postman (`docs/postman/coleccion.json`, 32 peticiones, que cubre el catálogo de la API de la sección 10.2), *scripts* de carga k6 (`k6/`), configuración Lighthouse (`lighthouserc.js`), instrumento y protocolo SUS (`docs/mediciones/sus/INSTRUMENTO-SUS.md`),

plantilla de consentimiento (`docs/etica/consentimiento/`). **Pendiente:** los análisis de rendimiento y de SUS son *scripts* de Python (`scripts/perf-analysis.py`, `scripts/sus-analysis.py`) y no cuadernos ejecutables con salidas archivadas (Jupyter o RMarkdown) como exige literalmente el Bloque D.1; convertirlos o documentar la equivalencia es una tarea abierta antes del cierre.

Declaración ética. El sistema trata datos personales de menores de edad. Los participantes de la encuesta SUS firmaron consentimiento informado, sus respuestas se anonimizaron con códigos P01–P10, y nueve hallazgos éticos (H-01 a H-09, uno ya corregido) están declarados sin omisiones en `docs/etica/ETHICS.md` y en la sección de Consideraciones éticas de este documento.

Contribuciones de autores (CRediT). De los catorce roles de la taxonomía CRediT, trece quedan cubiertos por el equipo; el único que no aplica es *Adquisición de fondos*, al ser un Proyecto Fin de Curso sin financiamiento externo. El reparto —*Conceptualización* (Ricardo, Darwin), *Curación de datos*, *Análisis formal*, *Visualización y Redacción (borrador original)* (Alejandro), *Investigación*, *Administración del proyecto* y *Supervisión* (Darwin), *Metodología*, *Recursos y Redacción (revisión y edición)* (Ricardo), *Validación* (Alejandro, Ricardo) y *Software* (los tres)— se deriva de la evidencia del historial de `git`, no es autodeclarado. El detalle por integrante y la tabla de cobertura rol por rol están en la sección 16 y en `CONTRIBUTORS.md`.

Conflictos de interés. El equipo declara la ausencia de conflictos de interés: ningún integrante tiene relación contractual, financiera o personal con la escuela ProFútbol ni con ningún proveedor mencionado en este documento (Render, Supabase, GitHub, Google/Gemini y Zenodo) más allá del uso de sus capas gratuitas para este proyecto académico.

Financiamiento. Este trabajo no recibió financiamiento externo. Es un Proyecto Fin de Curso de la asignatura Aplicaciones Web, desarrollado como requisito académico de la Universidad Técnica Estatal de Quevedo, sin patrocinio de terceros.

Uso de asistentes de inteligencia artificial. Distinto del uso de un modelo de lenguaje como funcionalidad del propio sistema (sección 15.1), este apartado declara el uso de IA generativa en el *proceso* de construir el proyecto. Siguiendo la guía de transparencia del SWEBOK v4.0 [46], el equipo declara el uso de un asistente de IA generativa (Claude, de Anthropic) como herramienta de apoyo en revisión y corrección de código de seguridad, generación de *scripts* de auditoría y análisis de resultados, y redacción de documentación (incluidas partes de este informe). El diseño de la arquitectura, las decisiones de seguridad y la verificación de que el sistema funciona correctamente fueron hechos y revisados por el equipo. El detalle completo, con el alcance exacto por archivo, está en `CONTRIBUTORS.md`.

Agradecimientos. Al Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D., por la dirección y retroalimentación del PFC a lo largo de las cuatro entregas, y a la Universidad Técnica Estatal de Quevedo por el acceso a la infraestructura académica del curso.

Referencias

- [1] Asamblea Nacional del Ecuador. Ley orgánica de protección de datos personales. Registro Oficial Suplemento 459, 26 de mayo de 2021, 2021.
- [2] Association for Computing Machinery. ACM code of ethics and professional conduct. <https://www.acm.org/code-of-ethics>, 2018.
- [3] Sebastian Baltes and Paul Ralph. Sampling in software engineering research: A critical review and guidelines. *Empirical Software Engineering*, 27(4):94, 2022.
- [4] Aaron Bangor, Philip Kortum, and James Miller. Determining what individual SUS scores mean: Adding an adjective rating scale. *Journal of Usability Studies*, 4(3):114–123, 2009.
- [5] Victor R. Basili, Gianluigi Caldiera, and H. Dieter Rombach. The goal question metric approach. In *Encyclopedia of Software Engineering*, volume 1, pages 528–532. John Wiley & Sons, 1994.
- [6] Tobias Blobel and Martin Lames. A concept for club information systems (cis) — an example for applied sports informatics. *International Journal of Computer Science in Sport*, 19(1), 2020.
- [7] John Brooke. SUS: A ‘quick and dirty’ usability scale. In *Usability Evaluation in Industry*, pages 189–194. Taylor & Francis, 1996.
- [8] Simon Brown. *Software Architecture for Developers, Volume 2: Visualise, document and explore your software architecture*. Leanpub, 2018. Modelo C4.
- [9] Humberto M. Carvalho and Carlos E. Gonçalves. Mismatches in youth sports talent development. *Frontiers in Sports and Active Living*, 5:1189355, 2023.
- [10] Diana Castilla, Irene Jaén, Carlos Suso-Ribera, Gemma García-Soriano, Irene Zaragoza, Juana Bretón-López, Adriana Mira, Amanda Díaz-García, and Azucena García-Palacios. Psychometric properties of the spanish full and short forms of the system usability scale (SUS): Detecting the effect of negatively worded items. *International Journal of Human-Computer Interaction*, 40(15), 2023.
- [11] Tse-Hsun Chen, Weiyi Shang, Jinqiu Yang, Ahmed E. Hassan, Michael W. Godfrey, Mohamed Nasser, and Parminder Flora. An empirical study on the practice of maintaining object-relational mapping code in Java systems. In *Proceedings of the 13th International Conference on Mining Software Repositories (MSR)*, pages 165–176, 2016.
- [12] Jürgen Cito and Harald C. Gall. Using docker containers to improve reproducibility in software engineering research. In *Proceedings of the 38th International Conference on Software Engineering Companion (ICSE-C)*, pages 906–907, 2016.
- [13] Norman Cliff. Dominance statistics: Ordinal analyses to answer ordinal questions. *Psychological Bulletin*, 114(3):494–509, 1993.
- [14] Alistair Cockburn. *Writing Effective Use Cases*. Agile Software Development Series. Addison-Wesley Professional, Boston, MA, 2001.
- [15] Mike Cohn. *User Stories Applied: For Agile Software Development*. Addison-Wesley Professional, Boston, MA, 2004.

- [16] Wei Dai, Yi-Shan Tsai, Jionghao Lin, Ahmad Aldino, Hua Jin, Tongguang Li, Dragan Gašević, and Guanliang Chen. Assessing the proficiency of large language models in automatic feedback generation: An evaluation study. *Computers and Education: Artificial Intelligence*, 7:100299, 2024.
- [17] Emelie Engström, Margaret-Anne Storey, Per Runeson, Martin Höst, and Maria Teresa Baldassarre. How software engineering research aligns with design science: A review. *Empirical Software Engineering*, 25(4):2630–2660, 2020.
- [18] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- [19] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, 2002.
- [20] Andy Georges, Dries Buytaert, and Lieven Eeckhout. Statistically rigorous Java performance evaluation. In *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*, pages 57–76, 2007.
- [21] Alan R. Hevner, Salvatore T. March, Jinsoo Park, and Sudha Ram. Design science in information systems research. *MIS Quarterly*, 28(1):75–105, 2004.
- [22] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
- [23] Muhammad Imran, Vittorio Cortellessa, Davide Di Ruscio, Riccardo Rubei, and Luca Traini. Is code coverage of performance tests related to source code features? an empirical study on open-source Java systems. *Empirical Software Engineering*, 30(6):157, 2025.
- [24] International Council on Systems Engineering. INCOSE guide to writing requirements. Technical Report INCOSE-TP-2010-006-04, International Council on Systems Engineering (INCOSE) — Requirements Working Group, July 2023.
- [25] ISO/IEC. ISO/IEC 25010:2011 — systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — system and software quality models, 2011.
- [26] ISO/IEC/IEEE. ISO/IEC/IEEE 29148:2018 — systems and software engineering — life cycle processes — requirements engineering, 2018.
- [27] M. B. Jones, J. Bradley, and N. Sakimura. JSON web token (JWT). Request for Comments 7519, Internet Engineering Task Force (IETF), May 2015. <https://www.rfc-editor.org/rfc/rfc7519>.
- [28] Barbara Kitchenham and Stuart Charters. Guidelines for performing systematic literature reviews in software engineering. Technical Report EBSE-2007-001, Keele University and Durham University Joint Report, July 2007.
- [29] Martin Kleppmann. *Designing Data-Intensive Applications*. O’Reilly Media, 2017.
- [30] Yixuan Liu. Design and implementation of a student attendance management system based on springboot and vue technology. *Frontiers in Computing and Intelligent Systems*, 8(1):91–97, 2024.
- [31] M. Nottingham, E. Wilde, and S. Dalal. Problem details for HTTP APIs. Request for Comments 9457, Internet Engineering Task Force (IETF), July 2023. <https://www.rfc-editor.org/rfc/rfc9457>.

- [32] Michael T. Nygard. Documenting architecture decisions. <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>, 2011.
- [33] OWASP Foundation. OWASP top 10:2021 — the ten most critical web application security risks. <https://owasp.org/Top10/>, 2021.
- [34] Ken Peffers, Tuure Tuunanen, Marcus A. Rothenberger, and Samir Chatterjee. A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3):45–77, 2007.
- [35] Klaus Pohl. *Requirements Engineering: Fundamentals, Principles, and Techniques*. Springer, Heidelberg, 2010.
- [36] PostgreSQL Global Development Group. PostgreSQL 16 documentation — CREATE PROCEDURE. <https://www.postgresql.org/docs/16/sql-createprocedure.html>, 2026.
- [37] Paul Ralph, Nauman bin Ali, Sebastian Baltes, Domenico Bianculli, Jessica Diaz, Yvonne Dittrich, Neil Ernst, Michael Felderer, Robert Feldt, Antonio Filieri, Breno Bernard Nicolau de França, Carlo Alberto Furia, Greg Gay, Nicolas Gold, Daniel Graziotin, Pinjia He, Rashina Hoda, Natalia Juristo, Barbara Kitchenham, Valentina Lenarduzzi, Jorge Martínez, Jorge Melegati, Daniel Mendez, Tim Menzies, Jefferson Moller, Dietmar Pfahl, Romain Robbes, Daniel Russo, Nyyti Saarimäki, Federica Sarro, Davide Taibi, Janet Siegmund, Diomidis Spinellis, Mirosław Staron, Klaas-Jan Stol, Margaret-Anne Storey, Damian Tamburri, Marco Torchiano, Christoph Treude, Burak Turhan, Xiaofeng Wang, and Sira Vegas. Empirical standards for software engineering research, 2021. arXiv:2010.03525 [cs.SE].
- [38] Janandani Ranaweera, Dan Weaving, Marcelo Zanin, Matthew C. Pickard, and Gregory Roe. Digitally optimizing the information flows necessary to manage professional athletes: A case study in rugby union. *Frontiers in Sports and Active Living*, 4:850885, 2022.
- [39] Redgate. Flyway documentation — migrations. <https://documentation.red-gate.com/flyway/>, 2026.
- [40] Pooya Rostami Mazrae, Tom Mens, Mehdi Golzadeh, and Alexandre Decan. On the usage, co-usage and migration of CI/CD tools: A qualitative analysis. *Empirical Software Engineering*, 28(2):52, 2023.
- [41] O. Suria. A statistical analysis of system usability scale (SUS) evaluations in online learning platform. *Journal of Information Systems and Informatics*, 6(2), 2024.
- [42] Saara Tenhunen, Tomi Männistö, Matti Luukkainen, and Petri Ithantola. A systematic literature review of capstone courses in software engineering. *Information and Software Technology*, 159:107191, 2023.
- [43] Shrey Tiwari, Serena Chen, Alexander Joukov, Peter Vandervelde, Ao Li, and Rohan Padhye. It’s about time: An empirical study of date and time bugs in open-source Python software. In *Proceedings of the 22nd IEEE/ACM International Conference on Mining Software Repositories (MSR)*, pages 39–51, 2025.
- [44] András Vargha and Harold D. Delaney. A critique and improvement of the cl common language effect size statistics of McGraw and Wong. *Journal of Educational and Behavioral Statistics*, 25(2):101–132, 2000.
- [45] Bangchao Wang, Zhiyuan Zou, Hongyan Wan, Yuanbang Li, Yang Deng, and Xingfu Li. An empirical study on the state-of-the-art methods for requirement-to-code traceability link recovery. *Journal of King Saud University – Computer and Information Sciences*, 36(6):102118, 2024.

- [46] Hironori Washizaki, editor. *SWEBOK Guide v4.0: Guide to the Software Engineering Body of Knowledge*. IEEE Computer Society, 2024.
- [47] Shao-Fang Wen and Basel Katt. A quantitative security evaluation and analysis model for web applications based on OWASP application security verification standard. *Computers & Security*, 135:103532, 2023.
- [48] Jingcheng Yang, Enze Wang, Jianjun Chen, Qi Wang, Yuheng Zhang, Haixin Duan, Wei Xie, and Baosheng Wang. Token time bomb: Evaluating JWT implementations for vulnerability discovery. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2026.

Anexos

Esta sección reúne el material de referencia que el Bloque B.17 de la guía exige archivar aparte del cuerpo del informe. Donde ese material ya existe como fuente única de verdad en otro documento del repositorio —la tabla de observaciones y la matriz de trazabilidad completa, ambas mantenidas y versionadas por separado— el anexo remite a esa fuente en vez de copiarla, por la misma razón que la sección 7.2 da para no mantener dos diagramas C4 paralelos: dos versiones de la misma verdad garantizan que al menos una esté desactualizada. Donde el material es breve y estable —cadenas de búsqueda, cuestionario SUS, checklists— se reproduce íntegro para consulta autónoma, sin necesidad de abrir el repositorio.

Anexo A — Observaciones acumuladas

La tabla completa de observaciones docente-equipo, con el porcentaje resuelto por entrega y la razón declarada de las que siguen abiertas, se presenta en la sección 5 de este mismo documento —no se duplica aquí para no arriesgar que las dos copias diverjan— y su fuente editable, actualizada con cada entrega, es `docs/observaciones/OBSERVACIONES.md`.

Anexo B — Cadena de búsqueda completa (Capítulo 3)

Las nueve búsquedas de la revisión de trabajos relacionados (sección 3.1), reproducidas aquí íntegras para consulta sin necesidad de volver al Capítulo 3:

1. Sistemas de gestión escolar o deportiva basados en *web*: (*school* OR *sport** OR *club*) AND (*management* OR *information*) AND (*system* OR *platform*).
2. Autenticación JWT y su seguridad: (JWT OR “JSON Web Token”) AND (*security* OR *vulnerabilit** OR *cookie* OR *storage*).
3. Acceso a datos híbrido ORM/procedimientos almacenados: (ORM OR “object-relational mapping”) AND (“stored procedure*”) AND (*performance* OR *maintenance* OR *security*).
4. Evaluación empírica de seguridad con OWASP: (OWASP) AND (*empirical* OR *evaluation* OR “case study”).
5. Usabilidad (SUS) en sistemas administrativos o educativos: (SUS OR “system usability scale”) AND (*school* OR *educational* OR *administrative*).
6. Ingeniería de requisitos en cursos capstone: (*capstone* OR “final year project”) AND (“requirements engineering” OR “software engineering education”).

Ventana temporal: 2016–2026. Motor: búsqueda web general dirigida a resultados alojados o indexados en IEEE Xplore, ACM Digital Library, Scopus, ScienceDirect y Springer Link —sin acceso de suscripción directo, limitación declarada explícitamente en la sección 3.1—, con cada registro incluido verificado de forma independiente contra una fuente primaria antes de citarlo.

Anexo C — Protocolo detallado de la encuesta SUS

Instrumento: *System Usability Scale* [7], diez ítems, escala Likert de 5 puntos (1 = totalmente en desacuerdo, 5 = totalmente de acuerdo), administrado en español. Protocolo completo, con las tareas previas y las condiciones de aplicación, en `docs/mediciones/sus/INSTRUMENTO-SUS.md`; lo esencial se reproduce a continuación (Tabla 32).

Cálculo. Por participante: ítems impares (1,3,5,7,9) contribuyen (respuesta–1); ítems pares (2,4,6,8,10) contribuyen (5–respuesta); puntuación SUS = suma de las diez contribuciones × 2,5, rango 0–100. No es un porcentaje: es una puntuación compuesta sobre una distribución

Cuadro 32: Cuestionario SUS de diez ítems aplicado (español).

#	Afirmación	Polaridad
1	Creo que me gustaría usar este sistema con frecuencia.	Positiva
2	Encontré el sistema innecesariamente complejo.	Negativa
3	Pensé que el sistema era fácil de usar.	Positiva
4	Creo que necesitaría el apoyo de una persona con conocimientos técnicos para poder usar este sistema.	Negativa
5	Encontré que las diversas funciones del sistema estaban bien integradas.	Positiva
6	Pensé que había demasiada inconsistencia en este sistema.	Negativa
7	Imagino que la mayoría de las personas aprenderían a usar este sistema muy rápidamente.	Positiva
8	Encontré el sistema muy engorroso de usar.	Negativa
9	Me sentí muy seguro/a usando el sistema.	Positiva
10	Necesité aprender muchas cosas antes de poder empezar a usar este sistema.	Negativa

Cuadro 33: Interpretación de la puntuación SUS (Bangor, Kortum y Miller, 2009; Sauro, 2011).

Rango SUS	Grado	Adjetivo	Percentil aprox.
85 – 100	A	Excelente	> 96
72 – 84,9	B	Bueno	70 – 96
68 – 71,9	C	Aceptable (media de la industria)	≈ 50
51 – 67,9	D	Pobre	15 – 50
0 – 50,9	F	Inaceptable	< 15

de referencia. La lectura cualitativa de la puntuación se interpreta con la escala adjetival de la Tabla 33.

Tareas previas. Antes de responder el cuestionario, cada participante intenta completar sin ayuda: (T1) iniciar sesión con las credenciales entregadas (RF-02); (T2) encontrar cuántos estudiantes hay registrados en total (RF-08); (T3) registrar un estudiante nuevo en la categoría SUB-15 (RF-10); (T4) intentar registrar un estudiante con categoría “JUVENIL” y explicar qué pasó (RF-11); (T5) modificar la categoría de un estudiante existente (RF-12); (T6) dar de baja a un estudiante (RF-13); (T7) cerrar sesión (RF-03). Se registra, por participante, si completó cada tarea y el tiempo aproximado, antes de aplicar el cuestionario.

Umbral objetivo. $SUS \geq 68$ (media de la industria). El resultado agregado y su desglose por perfil de usuario se reportan en la sección 12.7.

Anexo D — Ejecución del pipeline de integración continua

`.github/workflows/ci.yml` ejecuta, en cada *push* a *main*, las pruebas JUnit con verificación de cobertura JaCoCo (`mvn verify`, con la regla `haltOnFailure=true`: el umbral se hace cumplir, no solo se declara), la auditoría de SQL dinámico y el análisis estático SpotBugs/find-sec-bugs, en ese orden.

Al preparar este anexo se consultó la API pública de GitHub Actions —no solo la lista visible de la interfaz, que solo muestra las corridas más recientes— y se encontró que el hallazgo era más grave de lo que una primera revisión sugería: *main* no llevaba cuatro *commits* en rojo, sino **63 corridas consecutivas**, desde el *commit* `accb446` (`#112`, 2026-08-18T04:37Z) hasta `bee9995` (`#174`, 2026-09-02T15:56Z) —dos semanas completas—, por el mismo motivo que documenta la actualización del 2026-09-02 en la sección 4 (cobertura de *branches* bajo el umbral). No hay tres corridas verdes *previas* a esa fecha que capturar honestamente: las tres que se archivan aquí (Tabla 34) son las tres primeras que existen después de la corrección. La consulta completa a la API, incluidos el último verde antes de la regresión (`#111`) y el primer rojo (`#112`), está archivada en `docs/mediciones/ci/runs-verdes.json` —datos verificables por comando, no una captura de pantalla que solo puede mirarse—.

Cuadro 34: Tres corridas de CI consecutivas y verdes tras cerrar el gate de cobertura, verificadas contra la API de GitHub Actions.

#	Commit	Mensaje	Inicio (UTC)
175	7865af9	<i>test: cierra el gate de cobertura de branches (61 % → 70,06 %)</i>	2026-09-02 16:32
176	4718031	<i>docs(informe): actualizar cifras de cobertura tras cerrar el gate</i>	2026-09-02 16:35
177	76e4e48	<i>docs(trazabilidad): RNF-09 citaba el umbral de cobertura equivocado</i>	2026-09-02 16:37

Las tres corridas son consecutivas en el historial de *main* de este repositorio y están archivadas con su URL de origen en `docs/mediciones/ci/runs-verdes.json`.

Nota sobre los repositorios (migración de 2026-09-06/07 y su reversión). La ventana de dos semanas en la que transcurrieron esas 63 corridas y las tres verdes posteriores discurrió por entero en `DarwinSM21/SGED_APPWEB`, el repositorio canónico de este trabajo. El 2026-09-06/07 el historial se movió temporalmente a `darcalleg/SGED_APPWEB` y volvió después a `DarwinSM21/SGED_APPWEB`; migrar el historial de `git` no traslada ni recrea retroactivamente las corridas de *Actions* de otro repositorio, por eso durante ese intervalo `darcalleg/SGED_APPWEB` registró su propio historial de CI, mucho más corto por ser nuevo: 16 corridas verificadas contra su API pública (`docs/mediciones/ci/runs-darcalleg.json`), 10 en verde y 6 en rojo. Las 6 rojas comparten una sola causa —no seis defectos distintos—: el mismo *step* de validación de trazabilidad,

disparado dos veces por dos renombramientos de identificadores distintos (*inventario/estado*, anterior a este trabajo, cerrado en la corrida #6; y el de *person/user/alert/payment/guardian/student* de esta entrega, cerrado en la corrida #15). La corrida más reciente de esa ventana (#18, a46f681) está en verde; de regreso en el repositorio canónico, la evidencia de CI verificable vuelve a ser la historia de *Actions* de DarwinSM21/SGED_APPWEB registrada en `docs/mediciones/ci/runs-verdes.json`.

Anexo E — Checklist del estándar empírico (Ralph et al., 2021)

SGED se autoclasifica como *Engineering Research* dentro de los estándares empíricos de Ralph et al. [37] —el trabajo diseña, implementa y evalúa empíricamente un artefacto de software nuevo, no un experimento controlado con sujetos asignados a condiciones ni un estudio observacional puro sobre un sistema preexistente—, la misma clasificación que sustenta el enfoque DSR de la sección 11.1. Checklist completo, con evidencia concreta del repositorio en cada fila, en `docs/checklists/ralph2021-engineering-research.md` (Tabla 35).

Cuadro 35: Checklist del estándar empírico Engineering Research (Ralph et al., 2021).

Dimensión	Estado	Evidencia
El problema y su motivación están claramente descritos	Cumple	Sección 1.1, con tres referencias que sustentan la carencia de digitalización en clubes deportivos de formación
Preguntas de investigación explícitas y verificables	Cumple	RQ1–RQ4 (sección 1.2), cada una cerrada y trazada a evidencia empírica concreta
El artefacto está descrito con detalle suficiente para juzgar su novedad y su contribución	Cumple	Capítulos 7 e 9; estrategia híbrida de acceso a datos como aporte técnico explícito, con ADR-006
El proceso de diseño/construcción del artefacto está documentado	Cumple	Ocho ADR (ADR-001 a ADR-008), C4 niveles 1–3 generados desde <code>workspace.dsl</code> , <code>docs/observaciones/OBSERVACIONES.md</code> como bitácora del proceso iterativo
La evaluación usa métodos apropiados a las preguntas planteadas	Cumple	k6 para RQ1, OWASP+ZAP+análisis estático para RQ2, SUS para RQ3, JaCoCo+matriz de trazabilidad para RQ4
El procedimiento de evaluación es reproducible por un tercero	Parcial	<code>make bench/make audit/make test</code> reproducen el procedimiento; parte de la evidencia archivada quedó desactualizada frente al crecimiento del sistema hasta la corrección del 2026-09-02 (sección 12.5)
Se declara el contexto/entorno de la evaluación	Cumple	Sección 11.6 (o equivalente): SO, versión de Docker, JDK, estado inicial de la base de datos
Se declaran las limitaciones de la muestra cuando aplica	Cumple	Sección 11.4: $N = 15$ para SUS, muestreo por conveniencia declarado según Baltes y Ralph (2022)
Amenazas a la validez tratadas por categoría	Cumple	Capítulo 14, cuatro categorías con mitigación declarada por cada una
El artefacto y los datos de evaluación están disponibles para un tercero	Cumple	Repositorio público con DOI de <i>software</i> (10.5281/zenodo.22714477); <i>dataset</i> de mediciones con DOI propio (10.5281/zenodo.22422305) y licencia CC BY 4.0 (sección 21)
Conflictos de interés y financiamiento declarados	Cumple	Sección 21
Los resultados se interpretan sin sobregeneralizar	Cumple	Sección 18 responde cada RQ contra evidencia concreta; las Conclusiones distinguen lo cumplido de lo pendiente
Defectos del propio proceso de medición se reportan si se encuentran	Cumple	Rasgo distintivo del proyecto: cinco instrumentos de medición defectuosos detectados y corregidos a lo largo de las cuatro entregas, documentados como hallazgo

Resumen: 11 de 13 dimensiones en **Cumple**, 2 en **Parcial**, 0 en *No cumple*. Ambas parciales comparten la misma causa raíz —la evidencia empírica archivada se desactualiza frente al crecimiento del sistema— y se resuelven regenerando y publicando datos, no rediseñando el estudio.

Anexo F — Checklist INCOSE v4 de calidad de requisitos

Evaluación de los 36 requisitos funcionales, 13 no funcionales y 3 restricciones de diseño de SGED contra la *INCOSE Guide to Writing Requirements* v4 (INCOSE-TP-2010-006-04) [24], prometida como anexo desde la sección 4. Checklist completo en `docs/checklists/incose2023-req.md`; el detalle se resume en las Tablas 36 y 37.

Cuadro 36: Características de requisitos individuales, INCOSE v4 (C1–C9).

Característica	Estado	Evidencia
C1 <i>Necessary</i>	Cumple	Cada RF traza a al menos una historia de usuario o caso de uso en <code>docs/trazabilidad/matriz.csv</code>
C2 <i>Appropriate</i>	Cumple	Los RF describen comportamiento observable, no detalles de implementación; las decisiones de implementación viven en los ADR
C3 <i>Unambiguous</i>	Cumple	Patrón sintáctico ISO/IEC/IEEE 29148:2018 aplicado de forma uniforme desde la resolución de OBS-01
C4 <i>Complete</i>	Cumple	Los 36 RF tienen hoy campo Prioridad: y MoSCoW: explícitos (sección 4); cerrado el 2026-09-04
C5 <i>Singular</i>	Cumple	Ningún RF combina dos comportamientos verificables por separado; revisión manual línea por línea
C6 <i>Feasible</i>	Cumple	34 de 36 RF en estado Implementado son la prueba directa de factibilidad
C7 <i>Verifiable</i>	Cumple	33 de 36 RF referencian una prueba automatizada nombrada explícitamente en su entrada del SRS; los tres restantes declaran por qué no la tienen
C8 <i>Correct</i>	Parcial	RF-11b (peso/altura) es correcto técnicamente pero su base legal no está resuelta (hallazgo H-06)
C9 <i>Conforming</i>	Cumple	Los 36 RF siguen el mismo patrón “[condición] [sujeto] deberá [acción] [objeto] [restricción]”

Cuadro 37: Características del conjunto de requisitos, INCOSE v4 (C10–C15).

Característica	Estado	Evidencia
C10 <i>Complete</i>	Parcial	El módulo Equipo/Partido no tiene RF formal en el SRS pese a estar migrado
C11 <i>Consistent</i>	Parcial	RF-17, RF-18 y RF-22 figuran Implementado en el SRS pero desactualizados en <code>historias-usuario.md/casos-uso.md</code>
C12 <i>Feasible</i>	Cumple	94,4 % de los RF implementados por un equipo de tres integrantes en cuatro entregas
C13 <i>Comprehensible</i>	Cumple	El SRS incluye glosario, prioridad y estado por requisito
C14 <i>Able to be validated</i>	Cumple	Validación continua contra pruebas nombradas (C7) más el ciclo de observaciones docente-equipo
C15 <i>Correct</i>	Parcial	Mientras C11 no se cierre, “el conjunto” no tiene una única fuente de verdad consistente

Resumen: 11 de 15 características en **Cumple**, 4 en **Parcial**, 0 en *No cumple* —una más en **Cumple** que en la medición anterior, al cerrarse C4 con el campo **MoSCoW:** explícito. Las cuatro parciales restantes ya no comparten una única causa: C8 depende de resolver el hallazgo ético H-06 (RF-11b); C10 y C11 dependen de que se sincronicen `historias-usuario.md` y `casos-uso.md` con el SRS y de que el módulo Equipo obtenga un RF formal —pendiente concreto declarado en la sección 4—; C15 es consecuencia directa de C11.

Anexo G — Matriz de trazabilidad completa

Los 50 requisitos individuales de SGED (36 RF, 13 RNF, más el módulo Equipo sin RF formal propio; las 3 restricciones de diseño RD-01 a RD-03 no se rastrean en esta matriz) con su historia de usuario, caso de uso y estado (Tabla 38). La versión completa —con endpoint/componente, archivo de implementación, prueba automatizada y evidencia empírica por fila, columnas que no caben en el ancho de esta página— es `docs/trazabilidad/matriz.csv`, validada automáticamente por `scripts/validate-traceability.sh` en cada corrida de CI (Anexo D): ningún requisito carece de historia, caso de uso o prueba.

Cuadro 38: Matriz de trazabilidad completa (requisito, historia, caso de uso, estado).

Req.	Descripción	HU	CU	Estado
RF-01	Registro de usuarios por administrador	HU-06	CU-03	Implementado
RF-02	Autenticación con JWT en cookie <code>HttpOnly</code>	HU-01	CU-01	Implementado
RF-03	Cierre de sesión con revocación por JTI	HU-03	CU-02	Implementado
RF-04	Renovación de sesión por <i>refresh token</i>	HU-04	CU-01	Implementado
RF-05	Consulta de la sesión activa	HU-04	CU-01	Implementado
RF-06	Limitación de intentos de autenticación 5/15 min	HU-02	CU-01	Implementado
RF-07	Verificación de disponibilidad del servicio	N/A	N/A	Implementado
RF-08	Listado paginado de estudiantes	HU-05	CU-04	Implementado
RF-09	Consulta de estudiante por identificador	HU-05	CU-04	Implementado
RF-10	Registro de estudiante (persona+catgoría+estado por FK)	HU-06	CU-05	Implementado
RF-11	Validación de categoría por clave foránea (ya no patrón SUB-NN)	HU-06	CU-05	Implementado
RF-11b	Registro opcional de peso y altura del estudiante	N/A	N/A	Implementado [†]
RF-12	Actualización de estudiante	HU-07	CU-06	Implementado
RF-13	Baja lógica de estudiante	HU-08	CU-07	Implementado
RF-14	Conteo de activos por categoría vía procedimiento almacenado	HU-09	CU-08	Implementado
RF-15	Desactivación masiva por categoría vía procedimiento almacenado	HU-10	CU-09	Implementado
RF-16	Gestión de entrenadores (vinculados a persona y usuario)	HU-10b	CU-10	Implementado
RF-17	Horarios recurrentes de entrenamiento	HU-11	CU-11	Implementado
RF-18	Sesiones de entrenamiento	HU-11	CU-12	Implementado
RF-19	Registro de asistencia por QR o lista manual del entrenador	HU-12	CU-13	Implementado [†]
RF-20	Evaluación diaria por criterios	HU-13	CU-14	Modelado
RF-21	Consulta de promedios de evaluación	HU-13	CU-14	Modelado
RF-22	Notificación a representantes	HU-14	CU-15	Implementado [†]
RF-23	Gestión del catálogo de categorías	N/A	N/A	Implementado
RF-24	Gestión de cuentas de usuario como recurso propio	N/A	N/A	Implementado
RF-25	Gestión de personas	N/A	N/A	Implementado
RF-26	Consulta de estados generales	N/A	N/A	Implementado
RF-27	Gestión del catálogo de artículos de inventario	N/A	N/A	Implementado
RF-28	Registro de movimientos de <i>stock</i>	N/A	N/A	Implementado
RF-29	Asignación y devolución de artículos a estudiantes o entrenadores	N/A	N/A	Implementado
RF-30	Reporte de artículos con <i>stock</i> bajo	N/A	N/A	Implementado
Equipo	Módulo de equipos	N/A	N/A	Planificado [†]
RNF-01	Tiempo de respuesta p95 menor a 200 ms	HU-05	CU-04	Cumplido
RNF-02	Caché de listados con TTL 60 s	HU-05	CU-04	Cumplido
RNF-03	Contraseñas con BCrypt coste 12	HU-01	CU-01	Cumplido
RNF-04	Transporte cifrado TLS 1.2 o superior	HU-01	CU-01	Cumplido
RNF-05	Cabeceras de seguridad CSP, HSTS, <code>nosniff</code> , <code>DENY</code>	N/A	N/A	Cumplido
RNF-06	Control de acceso por rol en servidor	HU-06	CU-05	Cumplido
RNF-07	Ausencia de SQL dinámico por concatenación	N/A	CU-08	Cumplido
RNF-08	Registro de auditoría de autenticación	HU-01	CU-01	Cumplido
RNF-09	Cobertura de pruebas $\geq 70\%$ en líneas y en <i>branches</i>	N/A	N/A	Cumplido
RNF-10	Errores tipificados según RFC 9457 <i>Problem Detail</i>	HU-06	CU-05	Cumplido
RNF-11	Versionado del esquema con Flyway sin <code>ddl-auto</code>	N/A	N/A	Cumplido
RNF-12	Reproducibilidad en un solo comando	N/A	N/A	Cumplido
RNF-13	Imágenes fijadas por <i>digest</i> SHA-256	N/A	N/A	Cumplido
RF-31	Registro y alta de lesiones desde evaluación diaria	N/A	N/A	Implementado [†]
RF-32	Autoconsulta del estudiante sobre su equipo y desempeño	N/A	N/A	Implementado 2026-08-14
RF-33	Agenda de partidos con resultado posterior al juego	N/A	N/A	Implementado
RF-34	Convocatoria y once del partido según rendimiento reciente	N/A	N/A	Implementado
RF-35	Historial de asistencia de una sesión de entrenamiento	N/A	N/A	Implementado

[†]Estado con una salvedad temporal o de alcance que no cabe en esta columna; el texto

completo está en la columna **estado** de `docs/trazabilidad/matriz.csv`. Las más relevantes: RF-11b está implementado sin resolución ética (hallazgo H-06, sección 4); RF-19 cubre QR y registro manual, con RFID pendiente por falta de lector físico; RF-22 notifica solo dentro de la aplicación, no por correo/SMS/*push* (sección 19); el módulo **Equipo** tiene esquema migrado pero ningún *endpoint* expuesto por decisión, no por omisión (sección 10.2); RF-31 mantenía el *backend* desde antes de esta entrega y se completó con el *frontend* y una corrección de autoría del entrenador.